Contents

Abstract **vii**

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Problem Statement 2
1.3 Objectives 2
1.4 Scope of the Project 3
1.5 Report Organisation 4

2 Literature Review **5**
2.1 Existing Interview Preparation Platforms 5
2.2 Large Language Models in Education 6
2.3 Adaptive Learning Systems 7
2.4 Research Gap 7

3 System Analysis **8**
3.1 Feasibility Study 8
3.1.1 Technical Feasibility 8
3.1.2 Economic Feasibility 8
3.1.3 Operational Feasibility 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 11
3.4 Technology Stack with Justification 11
3.4.1 Why Next.js over plain React? 11
3.4.2 Why TypeScript over JavaScript? 11
3.4.3 Why FastAPI over Django or Flask? 12
3.4.4 Why PostgreSQL over MongoDB? 12
3.4.5 Why Gemini + Groq Fallback over a Single Provider? 12
3.4.6 Why EMA over Simple Average? 12

4 System Design **13**
4.1 System Architecture Overview 13
4.2 Module Division 14
4.2.1 Authentication Module 14
4.2.2 Curriculum Module 14

4.2.3 Session Management Module 14
4.2.4 Question Generation Module 14
4.2.5 Quiz Interface Module 15
4.2.6 Answer Evaluation Module 15
4.2.7 Skill Tracking Module 15
4.2.8 Analytics and Dashboard Module 15
4.3 Database Design 15
4.3.1 ER Diagram Explanation 15
4.3.2 Complete Database Schema 18
4.3.3 Relational Schema Dependencies 27
4.4 API Design 29
4.5 Authentication Flow 30
4.6 Question Generation Pipeline 31
4.7 Answer Evaluation Flow 33
4.7.1 EMA Skill Update Logic 35
4.8 Data Flow Diagrams 37
4.8.1 DFD Level 0 — Context Diagram 37
4.8.2 DFD Level 1 — Internal Process Logic 38
4.9 Sequence Diagrams 39
4.9.1 Session Start Sequence 39
4.9.2 Session End Sequence 41
4.10 System Class Diagram 43
4.11 Quiz Interface State Machine 45
4.12 Page Flow Diagram 46
4.13 Curriculum Taxonomy 47

5 Implementation **50**
5.1 Development Environment 50
5.2 Project Directory Structure 51
5.3 Authentication Implementation 53
5.3.1 JWT Token Lifecycle and Request Authentication 55
5.4 Question Generation Implementation 57
5.5 Quiz Interface Implementation 61
5.6 Answer Evaluation Implementation 63
5.7 Skill Tracking Engine 64
5.8 Dashboard Implementation 65
5.9 Export Implementation 67
5.10 Session Feedback Implementation 68
5.11 Prompt Engineering Strategy and Prompt Structure 69

5.12 Frontend-Backend Data Communication 75
5.13 Question Lifecycle — Session Storage and Database Storage 78
5.14 Database Table Usage — Current and Planned 80
5.15 Score Computation — From Raw LLM Output to Skill Vector 83

6 Testing **86**
6.1 Testing Approach 86
6.2 Authentication Test Results 86
6.3 Question Generation Test Results 87
6.4 Evaluation Test Results 88
6.5 End-to-End User Journey Test 88

7 Results and Discussion **90**
7.1 System Performance 90
7.2 Question Quality Assessment 91
7.3 Evaluation Quality Assessment 92
7.4 Skill Vector Evolution 92
7.5 Feedback Analysis 93

8 Limitations and Future Work **94**
8.1 Current Limitations 94
8.2 Planned Phase B Features — Subject Activation 95
8.3 Machine Learning Prediction Layer 95
8.4 Real-Time AI Panel Interview Simulation 96
8.5 ClawBot Integration 96
8.6 Resume Parser Integration 96
8.7 Deployment Plan 97
8.8 Modularity and Extensibility Analysis 97
8.9 Production Deployment Strategy 99
8.10 Database Maintenance and Data Retention 100
8.11 Security Architecture 101

9 Conclusion **103**
9.1 Technical Contributions 103
9.2 Final Reflections 104
9.3 The Path Ahead 104

List of Figures

4.1 Three-Tier System Architecture of MockAI, highlighting the internal flow and external LLM API fallback pipeline. 13
4.2 Entity Relationship Diagram of MockAI database using Crow's Foot cardinality. 16
4.3 Relational Schema showing Foreign Key to Primary Key dependencies. Typographical glitch in Primary Key identifiers has been resolved. 28
4.4 UML Sequence Diagram defining the timeline and interactions between the Client, FastAPI backend, and PostgreSQL database during Authentication. 30
4.5 Question Generation Pipeline flowchart demonstrating the LLM interaction cycle, dynamic fallback system, and strict JSON validation layers. . 32
4.6 Answer Evaluation Flowchart displaying the per-question loop processing, AI rubric evaluation, subtopic tagging, and final performance aggregation. 34
4.7 DFD Level 0 (Context Diagram) mapping the high-level data boundaries between the user, the core MockAI Platform, and External LLM providers. 37
4.8 DFD Level 1 isolating the six core operational processes, internal data stores, and external cross-system routing. Line paths are mathematically anchored to prevent overlap and maximize visibility. 38
4.9 UML Sequence Diagram defining the timeline and interactions between the Client, Frontend, Backend, Database, and LLM API during session initialization. 40
4.10 UML Sequence Diagram displaying the complex iterative evaluation loop, external LLM interactions, and EMA metrics calculation triggered during session termination. 42
4.11 UML Class Diagram for MockAI, illustrating the relationship between core data entities and logic services. 44
4.12 UML State Machine Diagram representing the question lifecycle with transparent text transitions. 45
4.13 Application Page Flow Diagram illustrating user navigation paths, route triggers, and dynamic session routing logic. 46

5.1 MockAI Monorepo Directory Structure representing the complete technical stack layout. 53
5.2 MockAI Registration Page showing email validation requirement. 55
5.3 MockAI Registration Page showing password strength requirements. . . . 55

5.4 JWT Lifecycle Flow diagram detailing the initial token generation pipeline and the subsequent middleware validation layer for protected requests. . 57
5.5 Python implementation of the prompt engineering strategy within `question_generator.py`, detailing the dynamic assembly of role, difficulty, taxonomy, and schema constraints. 60
5.6 MockAI Quiz Interface showing global countdown timer, per-question timer, MCQ rendering, split MCQ/Theory sidebar, and Mark for Review functionality. 62
5.7 MockAI Dashboard displaying the domain radar chart, detailed skill breakdown by subject, recent sessions log, and dynamically recommended focus areas based on recent performance. 66
5.8 Sample exported PDF report card detailing the summary statistics table and the comprehensive per-question feedback breakdown. 68
5.9 Prompt Construction Flow Diagram illustrating the vertical assembly of the context layers into a single prompt string, culminating in the resilient API routing sequence. 73
5.10 Evaluation Processing Flowchart demonstrating the divergence between deterministic MCQ string matching and generative Theory rubric grading. 75
5.11 Frontend-Backend Communication Pattern. The U-shaped flow illustrates the stateless, RESTful decoupling of the presentation layer from the core business logic, accompanied by three specific feature lifecycle examples. . 77
5.12 Storage Lifecycle Timeline illustrating the divergence and synchronization between the temporary browser cache and the permanent backend database across the five primary phases of an assessment. 79
5.13 Database Table Interaction Map detailing the core Entity-Relationship flows for active components, alongside the architectural provisioning for Phase B and C expansions. 82
5.14 Score Journey Flowchart detailing the sequential algorithmic progression from raw LLM output to permanent database persistence and frontend hydration. 85

8.1 Modularity Layer Stack demonstrating the isolated extensibility of the MockAI platform. 99
8.2 Production Deployment Architecture Diagram mapping the transition from localized development to managed cloud infrastructure. 100
8.3 Security Architecture Layered Defense, illustrating the "Onion" model where requests must penetrate multiple validation and authentication barriers. 102

List of Tables

2.1 Feature Comparison of Existing Platforms vs. MockAI 6

3.1 Cost Analysis and Tier Limits of MockAI Infrastructure 9
3.2 Functional Requirements of MockAI 10
3.3 Non-Functional Requirements of MockAI 11

4.1 MockAI Database Entities and Primary Keys 17
4.2 Relationships and Cardinality Matrix 18
4.3 Schema for users table 19
4.4 Schema for colleges table 19
4.5 Schema for sessions table 21
4.6 Schema for questions table 22
4.7 Schema for answers table 22
4.8 Schema for evaluations table 23
4.9 Schema for skill_vectors table 25
4.10 Schema for subtopic_scores table 25
4.11 Schema for session_feedback table 26
4.12 Schema for cheat_events table 27
4.13 Schema for conversation_logs table 27
4.14 MockAI REST API Endpoint Specification and Routing Table 29
4.15 Step-by-step EMA progression demonstrating how volatile session scores are smoothed into a stable proficiency vector. 36
4.16 MockAI Curriculum Taxonomy detailing the three-level hierarchy (Domains, Subjects, Subtopics) across current and future development phases. 48

6.1 Authentication and Authorization Test Results verifying the JWT middleware and Pydantic schema validation. 87

7.1 System Performance Metrics detailing execution times across internal and external network boundaries. 91

8.1 Targeted Deployment Infrastructure mapping platform components to managed cloud providers. 97

Abstract

The transition from academic learning to professional technical interviews represents one of the most significant challenges faced by engineering students today. Existing preparation platforms rely on static question banks, provide binary feedback with no conceptual explanation, and offer no mechanism to track how a student's proficiency evolves over time.

This paper presents **MockAI**—an adaptive, AI-powered interview preparation platform that dynamically generates domain-specific technical questions using Large Language Models (LLMs), evaluates candidate responses with contextual AI feedback, and tracks skill progression through an Exponential Moving Average (EMA) based scoring engine.

The system is built on a full-stack architecture comprising a Next.js frontend, FastAPI backend, PostgreSQL database, and a dual-provider LLM pipeline using Google Gemini as the primary engine and Groq-hosted LLaMA 3.3 70B as an automatic fallback. **MockAI** covers a comprehensive curriculum of 27 subjects across 7 domains including Core Computer Science, Placement Preparation, Programming Languages, AI and Machine Learning, Systems, Mathematics, and Specialized Domains.

The platform supports Multiple Choice Questions (MCQ), theory, and mixed question formats with difficulty calibration, a five-state quiz navigation system, dual timers, AI-generated per-question feedback with strengths and weakness analysis, skill radar visualization, session export in PDF and CSV formats, and post-session feedback collection.

Keywords: Large Language Models, Adaptive Learning, Interview Preparation, Skill Assessment, Exponential Moving Average, FastAPI, Next.js, PostgreSQL, Gemini API, LLaMA, Question Generation, Answer Evaluation, Skill Vector.

Chapter 1

Introduction

1.1 Background and Motivation

The modern software engineering job market demands that candidates demonstrate practical competence across a broad and interconnected set of technical domains. A placement interview at a product-based company will test data structures and algorithms, operating system concepts, database fundamentals, computer network theory, object-oriented design, and increasingly, machine learning concepts—often within a single session. Yet the preparation journey for most students remains fragmented, unstructured, and one-dimensional.

Students today rely on a combination of static PDF notes, YouTube playlists, GeeksForGeeks articles, and coding platforms like LeetCode. While each of these resources has individual merit, none of them together constitute an adaptive, feedback-rich preparation system. A student who solves 200 LeetCode problems still may not know which conceptual areas they are weakest in, because LeetCode provides no conceptual evaluation—only test case pass/fail. A student who reads OS notes does not know whether their understanding of process scheduling is at an interview-ready level without someone to test and evaluate them.

The emergence of Large Language Models (LLMs)—particularly instruction-following models like Google Gemini, Anthropic Claude, and Meta LLaMA—creates an entirely new paradigm for educational technology. These models can generate novel, domain-specific questions that have never appeared in any static question bank. They can evaluate a student's written answer with the nuance of a human expert—identifying correct reasoning, incomplete arguments, missing examples, and conceptual errors. They can do this at scale, without human scheduling, and at near-zero marginal cost.

MockAI operationalises this paradigm into a production-grade web application. It is not a chatbot. It is not a simple Q&A interface. It is a structured, curriculum-aligned, session-based preparation platform where every question is generated fresh by AI, every answer is evaluated by AI, and every session updates a persistent skill model for that user.

1.2 Problem Statement

Existing interview preparation tools fail to address the following critical gaps:

- **Gap 1 — Static Content:** Questions are pre-authored and repeated across users. Students share questions from these platforms, reducing them to memorisation exercises rather than genuine understanding tests.
- **Gap 2 — No Conceptual Feedback:** Automated scoring marks MCQ answers right or wrong with no explanation of why. Theory questions either go unscored or require a human evaluator who is not available on-demand.
- **Gap 3 — No Skill Tracking:** There is no longitudinal view of a student's proficiency. A student cannot look at their preparation history and say with confidence, "my OS is at 7/10, my DSA is at 4/10, and I need to focus on graph algorithms."
- **Gap 4 — No Adaptivity:** Question difficulty, topic selection, and question type are not adapted based on what the student has practised before or how they have performed.
- **Gap 5 — Curriculum Disconnect:** Most platforms do not align with university syllabi. Topics covered in competitive platforms often do not match what UIET or similar universities teach, creating a gap between classroom learning and interview practice.

MockAI directly addresses all five gaps through dynamic generation, AI evaluation, EMA-based skill tracking, and a curriculum taxonomy built from standard university syllabi.

1.3 Objectives

The specific objectives of this project are:

1. To design and implement a dynamic AI question generation pipeline using LLMs that produces subject-aware, subtopic-specific, difficulty-calibrated technical interview questions across multiple question types.
2. To build an AI-powered answer evaluation system capable of scoring both MCQ and open-ended theory answers with contextual feedback, identification of strengths and weaknesses, and explanation of the correct answer.
3. To implement a longitudinal skill tracking engine using Exponential Moving Average that reflects current proficiency and responds to improvement over time.

4. To develop an interactive quiz interface with dual timers, free question navigation, five-state question tracking, mark-for-review functionality, and auto-submission on timer expiry.
5. To construct a comprehensive three-level curriculum taxonomy covering 27 subjects across 7 domains and 200+ subtopics, aligned with standard university placement syllabi.
6. To build a real-time dashboard with domain and subject level skill visualisation, session history, and personalised study recommendations.
7. To lay the architectural foundation for future phases including real-time AI panel interview simulation, anti-cheat mechanisms, ML-based readiness prediction, and ClawBot integration.

1.4 Scope of the Project

Currently Implemented (Phase A)

The current version of the platform includes:

- User authentication with JWT and bcrypt.
- AI question generation for all Phase A subjects.
- MCQ and theory answer evaluation with AI feedback.
- Five-state quiz navigation with dual timers.
- EMA-based skill vector tracking at subject level.
- Domain and subject skill visualisation using radar and bar charts.
- Session log, result report card, PDF and CSV export.
- Post-session feedback collection with star rating, qualitative questions, and comments.

Out of Scope — Planned for Future Phases

The following features are slated for subsequent development phases:

- Real-time AI panel interview simulation with multiple AI agents.
- ClawBot integration for project code fetching and agent orchestration.

- Anti-cheat computer vision system.
- ML-based interview readiness prediction.
- Phase B/C subject activation.
- Coding IDE with execution engine.
- Resume parser integration and refinement.
- Voice interface for AI interview.

1.5 Report Organisation

This report is structured as follows: **Chapter 2** reviews existing platforms and relevant literature. **Chapter 3** presents system analysis including feasibility study and requirements. **Chapter 4** covers complete system design including architecture, all database schemas, data flow diagrams, sequence diagrams, class diagram, and curriculum taxonomy. **Chapter 5** describes the implementation of each module with code-level detail. **Chapter 6** covers testing. **Chapter 7** presents results and discussion. **Chapter 8** covers current limitations and future work. Finally, **Chapter 9** concludes the report.

Chapter 2

Literature Review

2.1 Existing Interview Preparation Platforms

The current landscape of interview preparation tools is fragmented, with different platforms solving distinct parts of the problem but none providing a holistic, adaptive solution.

LeetCode

LeetCode is the most widely used algorithmic practice platform. It provides a large collection of coding problems categorised by difficulty and topic, with community-contributed solutions. However, it has no coverage of theoretical subjects such as Operating Systems (OS), Database Management Systems (DBMS), or Computer Networks (CN). All questions are pre-authored, feedback is limited to test case pass/fail, and no conceptual explanation is provided. Skill tracking is limited to a simple difficulty distribution chart with no subject-level intelligence.

InterviewBit

InterviewBit covers a broader curriculum including system design and core CS subjects, but relies entirely on a static question bank. Questions are pre-authored, answers are prewritten, and the system does not adapt to user performance. No AI evaluation exists.

GeeksForGeeks

GeeksForGeeks functions primarily as a reference platform rather than an adaptive learning system. It provides articles, practice problems, and company-specific question collections, but there is no session-based evaluation, no feedback mechanism, and no skill tracking.

Pramp

Pramp offers peer-to-peer mock interviews, matching users with each other for live practice sessions. While the feedback quality can be high, it depends entirely on human

availability, is inconsistent in quality between sessions, and does not scale. Users must schedule sessions in advance and depend on another student's availability.

IndiaBix

IndiaBix and similar aptitude platforms cover quantitative aptitude and verbal reasoning but have no coverage of technical CS domains and provide no AI-based evaluation.

Platform Comparison Summary
The critical observation is that no existing platform combines all four necessary pillars: dynamic question generation, AI-based contextual evaluation, longitudinal skill tracking, and curriculum-aligned subject coverage.

Platform	**Dynamic Qs**	**AI Evaluation**	**Skill Tracking**	**Theory Coverage**
LeetCode	No	No	Limited	No
InterviewBit	No	No	Limited	Yes (Static)
GeeksForGeeks	No	No	No	Yes (Reference)
Pramp	No	Human	No	Yes (Verbal)
MockAI (Proposed)	**Yes**	**Yes**	**Yes (EMA)**	**Yes (Dynamic)**

Table 2.1: Feature Comparison of Existing Platforms vs. MockAI

2.2 Large Language Models in Education

The application of LLMs in educational technology has grown rapidly since the public release of GPT-3 in 2020. Brown et al. (2020) demonstrated that LLMs exhibit strong few-shot learning capabilities—they can perform tasks given only a prompt description and a few examples, without parameter fine-tuning. This property is central to MockAI's question generation approach, where the prompt itself encodes the domain, difficulty, format, and constraints.

Research specifically evaluating LLM-generated educational questions (Elkins and Chun, 2023) found that GPT-4 generated MCQs were comparable in quality to expert-authored questions when prompts specified domain context, difficulty level, and distractor plausibility constraints. MockAI's prompt engineering directly incorporates these findings; the prompt specifies subject, subtopic list, difficulty standard, and explicit rules for distractor quality.

For automated answer evaluation, neural approaches have shown accuracy comparable to human graders for well-defined technical domains (Galhardi and Brancher, 2018). The key challenge is partial credit and semantic equivalence—a student who describes Breadth-First Search (BFS) correctly but uses different terminology than the model answer should still receive credit. MockAI addresses this using LLM evaluation rather than keyword matching, allowing the model to reason about semantic content rather than surface form.

2.3 Adaptive Learning Systems

Adaptive learning systems modify educational content based on learner performance data. The theoretical foundation is Item Response Theory, which models the probability of a correct response as a function of item difficulty and learner ability. More recent systems use Deep Knowledge Tracing—a recurrent neural network approach to modelling latent knowledge state as it evolves over practice sessions.

MockAI implements a simplified but highly effective adaptive mechanism using Exponential Moving Average (EMA) based skill tracking. While less computationally heavy than full knowledge tracing, EMA provides an interpretable score that weights recent performance more heavily than historical performance. This is crucial for interview preparation: a student who was weak in DSA three months ago but has since practised extensively should have their current score reflect their recent competence, rather than being dragged down by historical sessions. The learning rate ($\alpha = 0.3$) achieves this balance.

2.4 Research Gap

The survey above reveals a clear and actionable research gap. No single platform combines dynamic LLM-based question generation, contextual AI evaluation with explanatory feedback, longitudinal skill vector tracking, and curriculum-aligned subject coverage in one integrated system. **MockAI** is designed as a direct response to this gap.

Chapter 3

System Analysis

3.1 Feasibility Study

3.1.1 Technical Feasibility

Every component of MockAI's stack is mature, production-grade, and well-documented. Next.js is used in production by companies including Vercel, TikTok, and Twitch. FastAPI is used by Uber, Netflix, and Microsoft for internal services. PostgreSQL has been the world's most advanced open-source relational database for over 25 years. Google Gemini and Groq provide accessible, well-documented APIs with Python Software Development Kits (SDKs).

The development team has successfully implemented all planned Phase A features, confirming technical feasibility empirically. The LLM fallback chain was validated by intentionally exhausting the Gemini API quota and confirming that Groq takes over the inference load transparently.

3.1.2 Economic Feasibility

All development infrastructure currently runs on free tiers. Docker Desktop is free for educational use. Python and Node.js are open-source. The Gemini API provides a substantial free tier of 1,500 requests per day (or 15 requests per minute / 1 million tokens per day), while the Groq API provides a generous free tier of 14,400 requests per day.

For deployment, Vercel hosts the frontend for free, Railway or Render can host the backend for free, and Supabase provides managed PostgreSQL free up to 500MB—which is sufficient for storing thousands of users and session logs. A detailed breakdown of the development infrastructure costs is provided in Table 3.1.

Component	**Tier Used & Limits**	**Monthly Cost**
Google Gemini API	Free Tier (1,500 req/day, 15 req/min, ~1M tokens/min)	$0
Groq API (LLaMA 3)	Free Tier (14,400 req/day, 30 req/min, 500,000 tokens/day)	$0
PostgreSQL	Self-hosted via Docker / Managed Free Tier (500MB)	$0
React (Next.js)	Open-source / Vercel Hobby Tier	$0
FastAPI + Python	Open-source	$0
Docker Desktop	Free for personal/educational use	$0
Total Development & Operational Cost		**$0**

Table 3.1: Cost Analysis and Tier Limits of MockAI Infrastructure

Therefore, the total operational and development cost at the current project scale is conclusively zero.

3.1.3 Operational Feasibility

The system is designed for minimal operational overhead. Docker Compose brings up the complete local development stack with a single command. Alembic manages all database schema changes as versioned migration files, ensuring consistency across environments.

The automatic LLM fallback mechanism ensures high availability for the question generation pipeline, even when the primary provider experiences latency or quota issues. Furthermore, the stateless JSON Web Token (JWT) architecture means no session storage is required on the backend servers, improving scalability. The system has been tested end-to-end by multiple users successfully.

3.2 Functional Requirements

Functional requirements define the core capabilities and operations the system must perform to satisfy the user's needs. The functional requirements for MockAI are detailed in Table 3.2.

Req. ID	Description	Priority
FR-01	Users shall register with name, email, and password with strength validation	High
FR-02	Users shall authenticate via email and password and receive a JWT token	High
FR-03	System shall generate questions for a given subject, subtopics, difficulty, count, and type	High
FR-04	System shall evaluate MCQ answers by fuzzy string matching with AI explanation	High
FR-05	System shall evaluate theory answers using LLM scoring on a 0-10 scale with feedback	High
FR-06	System shall present a dual-timer quiz interface with countdown and per-question timers	High
FR-07	Users shall navigate questions freely, mark for review, skip, and clear answers	High
FR-08	System shall auto-submit session when countdown timer expires	High
FR-09	System shall evaluate all answers and return a scored report with per-question breakdown	High
FR-10	System shall update the user's skill vector using EMA after each completed session	High
FR-11	Dashboard shall display real scores, session history, and skill visualisation	High
FR-12	Users shall export session results as PDF and CSV	Medium
FR-13	Users shall submit feedback after each session	Medium
FR-14	Session log shall show all completed sessions with filter and bulk export	Medium
FR-15	System shall fallback to secondary LLM when primary quota is exceeded	High

Table 3.2: Functional Requirements of MockAI

3.3 Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance goals, and constraints under which the system must operate. These are defined in Table 3.3.

Req. ID	Description	Target
NFR-01	Question generation response time	Under 15 seconds
NFR-02	Answer evaluation response time per question	Under 8 seconds
NFR-03	Frontend page load time	Under 2 seconds
NFR-04	Database query response time	Under 200 milliseconds
NFR-05	All passwords must use one-way hashing	bcrypt cost factor 12
NFR-06	All endpoints except auth must require JWT	Enforced by middleware
NFR-07	System must remain functional when primary LLM fails	Groq fallback active
NFR-08	Data must persist across container restarts	Docker volume mounts

Table 3.3: Non-Functional Requirements of MockAI

3.4 Technology Stack with Justification

3.4.1 Why Next.js over plain React?

Next.js provides file-based routing, server-side rendering, automatic code splitting, and Vercel deployment with zero configuration. Plain React would require setting up React Router, a build tool, and a separate deployment pipeline. The App Router in Next.js 16 also provides layouts that wrap all pages, which MockAI uses to manage the Navbar and AuthContext efficiently.

3.4.2 Why TypeScript over JavaScript?

TypeScript adds static type checking at compile time. In a project of this size with complex data structures like `SessionData`, `Question`, `EvaluationResult`, and `SkillVector`, TypeScript catches type mismatches before they reach the browser. When the backend API shape changes, TypeScript immediately flags all frontend code that depends on the old shape. JavaScript would only surface these errors at runtime, often in production.

3.4.3 Why FastAPI over Django or Flask?

FastAPI is async-native, meaning it can handle multiple simultaneous LLM API calls without blocking. Django and Flask are synchronous by default—they would block the thread for the 8-15 seconds each LLM call takes. FastAPI also auto-generates OpenAPI documentation at `/docs`, which was used extensively during development and testing. Furthermore, Pydantic integration provides automatic request validation with clear error messages.

3.4.4 Why PostgreSQL over MongoDB?

MockAI's data is fundamentally relational—users have sessions, sessions have questions, questions have answers, and answers have evaluations. These relationships are best expressed as foreign keys with referential integrity. MongoDB would require application-level enforcement of these constraints. However, PostgreSQL's `JSONB` column type is utilized where data is genuinely semi-structured (e.g., subtopics list, evaluation strengths and weaknesses arrays, session summaries), providing the best of both worlds.

3.4.5 Why Gemini + Groq Fallback over a Single Provider?

No LLM API has 100% uptime or unlimited free quota. Using only Gemini means every quota exhaustion causes complete feature failure. The fallback chain ensures the system continues working seamlessly. Groq was chosen as the fallback because its inference speed is exceptional—often faster than Gemini—and its free tier is generous.

3.4.6 Why EMA over Simple Average?

A simple average drags historical performance indefinitely. A student who performed poorly six months ago but has significantly improved will still show a low average. The Exponential Moving Average (EMA) with $\alpha = 0.3$ gives 30% weight to the most recent session and 70% to the existing score—ensuring recent improvement is reflected meaningfully without ignoring history entirely. The α value was chosen based on the intuition that approximately 10 sessions should bring a score close to a student's current true proficiency.

Chapter 4

System Design

4.1 System Architecture Overview

The architecture of **MockAI** is designed for scalability, modularity, and high availability. It follows a modern three-tier application architecture separating the client interface, application logic, and data persistence layers, while integrating external AI services.

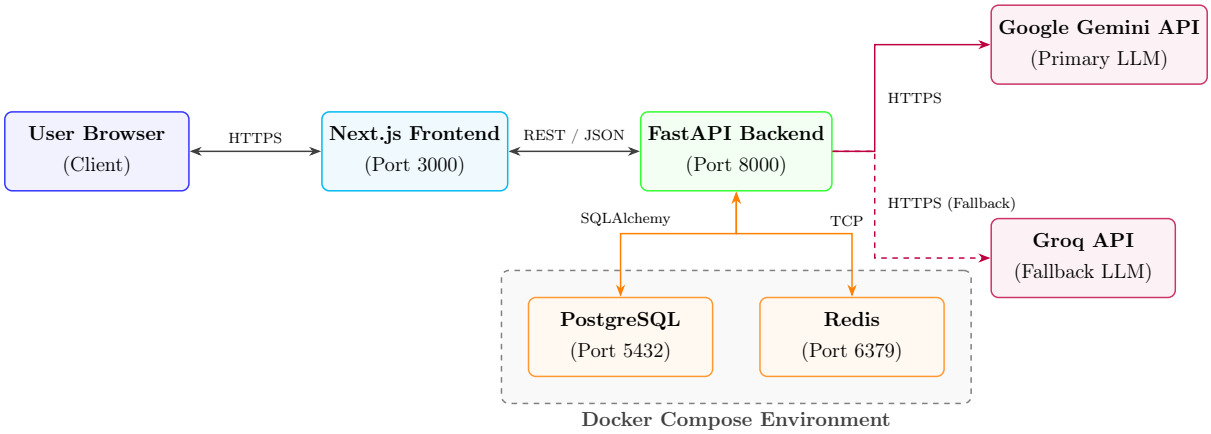

Figure 4.1: Three-Tier System Architecture of MockAI, highlighting the internal flow and external LLM API fallback pipeline.

Description of the Architecture:
The user's browser communicates exclusively with the Next.js frontend, which is responsible for rendering the interface and managing client-side state. The frontend never directly accesses the database or LLM APIs—all data passes through the FastAPI backend, which acts as the sole application layer.

The backend validates all requests via JWT middleware, orchestrates LLM API calls, persists data via SQLAlchemy to PostgreSQL, and communicates with Redis for caching operations. It returns structured JSON responses back to the frontend. The LLM providers are external services accessed over secure HTTPS connections. To ensure reproducible environments and simplified local development, Docker Compose manages the local PostgreSQL and Redis containers within an isolated boundary.

4.2 Module Division

The **MockAI** system is structured into eight distinct logical modules. Each module has a strictly defined responsibility boundary to ensure maintainability, scalability, and clean separation of concerns.

4.2.1 Authentication Module

This module handles secure user registration with input validation, password hashing using bcrypt, and login mechanisms that issue stateless JSON Web Tokens (JWT). It also includes the bearer token middleware that authenticates all protected backend requests.

- **Frontend Components:** `LoginPage`, `RegisterPage`.
- **Backend Components:** `auth_router`, `auth_middleware`.

4.2.2 Curriculum Module

The curriculum is implemented as a static TypeScript data structure defining the complete three-level taxonomy of domains, subjects, and subtopics. It is a compile-time constant rather than a database table, ensuring extremely fast load times. A `comingSoon` boolean flag on each domain and subject controls user interface visibility. Switching a flag to `false` instantly activates that subject for end-users without requiring further code modifications.

4.2.3 Session Management Module

This module orchestrates the complete lifecycle of a practice interview session—from initialization to submission. It acts as the central coordinator, triggering question generation, storing temporary answer states, initiating the evaluation pipeline upon submission, and firing the event to update the user's skill vector.

4.2.4 Question Generation Module

A dedicated Python service responsible for dynamic content creation. It dynamically constructs structured system prompts based on the requested subject and difficulty, invokes the primary LLM API, and automatically triggers the fallback sequence to the secondary LLM if necessary. It is also responsible for parsing and validating the LLM's JSON response to ensure it returns a clean, strongly-typed array of question objects.

4.2.5 Quiz Interface Module

This is the most state-intensive frontend module. It is responsible for rendering the interactive testing environment. It manages the dual-timer system (overall session countdown and per-question time tracking), allows free question navigation, tracks the five-state question status (unvisited, visited, answered, marked for review, answered and marked), and handles the final payload submission to the backend.

4.2.6 Answer Evaluation Module

A core Python backend service that routes answers to the appropriate evaluation strategy. For MCQs, it performs fuzzy string matching combined with AI-generated explanations. For theory questions, it constructs rigorous evaluation prompts, calls the LLM for deep semantic analysis, and returns structured objects containing a 0–10 score, explicit strengths, weaknesses, and a model answer.

4.2.7 Skill Tracking Module

This module houses the core mathematical logic for adaptive learning. It implements the Exponential Moving Average (EMA) algorithm. Immediately after a session is evaluated, this module retrieves the user's current skill vector, applies the EMA update formula specifically for the tested subject, and securely persists the newly calibrated score to the database.

4.2.8 Analytics and Dashboard Module

This data-aggregation module generates the user's personalized homepage. It compiles completed session data, computes summary statistics (e.g., total questions answered, average score), and retrieves the latest skill vector. It classifies subjects as strong or weak to generate actionable study recommendations. The frontend subsequently visualizes this data using interactive radar charts, bar graphs, and recent session logs.

4.3 Database Design

4.3.1 ER Diagram Explanation

The Entity Relationship (ER) diagram for MockAI is modelled using a hybrid notation: rectangles represent entities, diamonds denote relationships, and Crow's Foot line endings indicate cardinality. The primary key of each entity is strictly underlined.

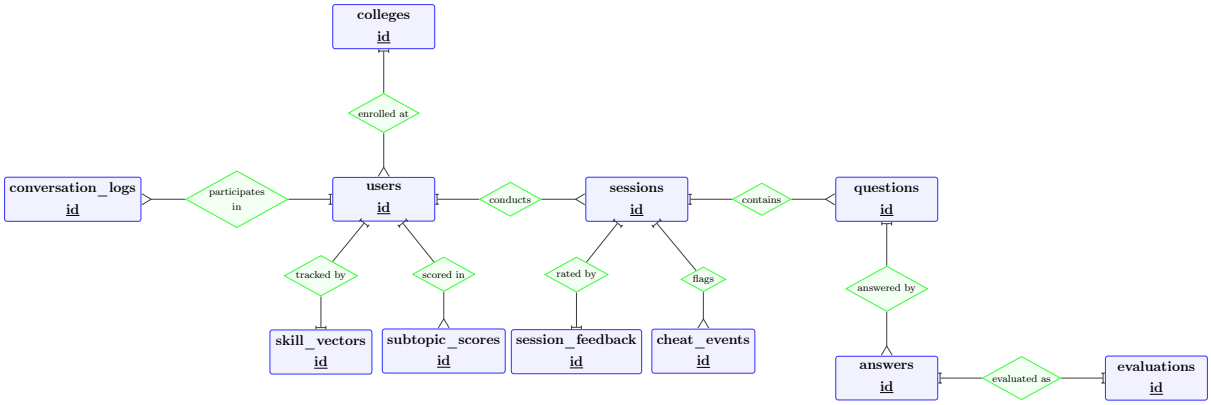

Figure 4.2: Entity Relationship Diagram of MockAI database using Crow's Foot cardinality.

Database Flow and Conceptual Description:

The MockAI database is structured around a central **users** table, which acts as the core entity linking identity, analytics, and user activity. A user's academic context is captured via the **colleges** relation, while their overarching interactions with the system are tracked in **conversation_logs**.

The primary operational workflow of the platform revolves around the **sessions** table. When a user initiates a practice interview, a new session record is created. This session acts as a parent container for multiple **questions** dynamically generated by the LLM. As the user navigates the quiz interface, their text inputs and option selections are recorded in the **answers** table. Because the relationship allows multiple answers per question, the system is future-proofed for feature expansions like question re-attempts.

Once a session is submitted, the evaluation module processes each answer, generating a corresponding 1-to-1 record in the **evaluations** table containing the quantitative score, identified strengths, weaknesses, and the AI's model explanation.

Alongside the core quiz content, the session entity also manages crucial metadata. Any suspicious or unauthorized activity detected during the test is logged in the **cheat_events** table, tied directly to that specific session for integrity audits. Upon completion of the test, the user's subjective experience is captured in the **session_feedback** table.

Finally, the system closes the feedback loop through its adaptive learning engine. After a session is evaluated, the aggregated results trigger an update to the user's longitudinal performance metrics. These updates are permanently stored in the **skill_vectors** (tracking broad domain proficiency) and **subtopic_scores** (tracking granular, topic-level mastery) tables, powering the user's analytics dashboard.

Database Entities

Table 4.1 outlines the core entities within the MockAI database alongside their respective primary keys.

Entity Name	**Primary Key**
`users`	`id`
`sessions`	`id`
`questions`	`id`
`answers`	`id`
`evaluations`	`id`
`skill_vectors`	`id`
`subtopic_scores`	`id`
`session_feedback`	`id`
`cheat_events`	`id`
`conversation_logs`	`id`
`colleges`	`id`

Table 4.1: MockAI Database Entities and Primary Keys

Entity Relationships and Cardinality

Table 4.2 defines the mapping, cardinality, and interaction flow between the database entities.

Source Entity	**Cardinality**	**Target Entity**	**Relationship Description**
`users`	1 : M	`sessions`	One user conducts many sessions.
`sessions`	1 : M	`questions`	One session contains many questions.
`questions`	1 : M	`answers`	One question is answered by many answers.
`answers`	1 : 1	`evaluations`	One answer is evaluated as one evaluation.

Continued on next page

Table 4.2 – continued from previous page

Source Entity	Cardinality	Target Entity	Relationship Description
`users`	1 : 1	`skill_vectors`	One user is tracked by exactly one skill vector.
`users`	1 : M	`subtopic_scores`	One user is scored in many subtopic scores.
`sessions`	1 : 1	`session_feedback`	One session is rated by at most one feedback.
`sessions`	1 : M	`cheat_events`	One session flags many cheat events.
`users`	1 : M	`conversation_logs`	One user participates in many conversation logs.
`colleges`	1 : M	`users`	One college enrolls many users.

Table 4.2: Relationships and Cardinality Matrix

4.3.2 Complete Database Schema

The following tables define the complete relational schema, data types, and constraints for the MockAI database.

Table: `users`
This table stores all registered user accounts.

Column	Data Type	Constraints	Description
`id`	`UUID`	Primary Key, default uuid4	Unique user identifier
`name`	`VARCHAR(100)`	NOT NULL	Full name of the user
`email`	`VARCHAR(150)`	NOT NULL, UNIQUE	Email address used for login
`password_hash`	`VARCHAR(255)`	NOT NULL	bcrypt hashed password
`role`	`VARCHAR(20)`	NOT NULL, default 'student'	Role: student, admin, etc.

Continued on next page

Table 4.3 – continued from previous page

Column	**Data Type**	**Constraints**	**Description**
`college_id`	`UUID`	FK → `colleges.id`, nullable	Institution association
`created_at`	`TIMESTAMP`	default current timestamp	Account creation time

Table 4.3: Schema for users table

Table: `colleges`
Stores institution information for multi-college deployment.

Column	**Data Type**	**Constraints**	**Description**
`id`	`UUID`	Primary Key	Unique college identifier
`name`	`VARCHAR(200)`	NOT NULL	Full institution name
`city`	`VARCHAR(100)`	nullable	City
`state`	`VARCHAR(100)`	nullable	State
`created_at`	`TIMESTAMP`	default current timestamp	Record creation time

Table 4.4: Schema for colleges table

Table: `sessions`
Central table representing a complete practice session from start to finish.

Column	**Data Type**	**Constraints**	**Description**
`id`	`UUID`	Primary Key	Unique session identifier
`user_id`	`UUID`	FK → `users.id`, NOT NULL	Owner of this session
`session_type`	`VARCHAR(30)`	NOT NULL, def 'topic_practice'	topic_practice or ai_interview

Continued on next page

Table 4.5 – continued from previous page

Column	**Data Type**	**Constraints**	**Description**
`status`	`VARCHAR(20)`	default 'active'	active, completed, or aborted
`domain_id`	`VARCHAR(100)`	nullable	e.g. core_cs, placement_prep
`subject_id`	`VARCHAR(100)`	nullable	e.g. dsa, os, dbms
`subject_name`	`VARCHAR(200)`	nullable	Human readable name
`skill_field`	`VARCHAR(100)`	nullable	Maps to skill_vectors column
`subtopics`	`JSONB`	nullable	Array of objects with id & name
`difficulty`	`VARCHAR(20)`	nullable	easy, medium, or hard
`question_type`	`VARCHAR(20)`	nullable	mcq, theory, or mixed
`question_count`	`INTEGER`	nullable	Number of questions requested
`total_score`	`NUMERIC(4,2)`	nullable	Avg score 0 to 10
`accuracy`	`NUMERIC(5,2)`	nullable	Percentage correct
`total_time_seconds`	`INTEGER`	nullable	Session duration in seconds
`weak_subtopics`	`JSONB`	nullable	Subtopics with score below 5
`strong_subtopics`	`JSONB`	nullable	Subtopics with score 5 and up
`session_summary`	`JSONB`	nullable	Compact object with all stats
`started_at`	`TIMESTAMP`	default current timestamp	When session began
`ended_at`	`TIMESTAMP`	nullable	When session was submitted

Continued on next page

Table 4.5 – continued from previous page

Column	**Data Type**	**Constraints**	**Description**
`created_at`	`TIMESTAMP`	default current timestamp	Record creation time
`cheat_flags`	`JSONB`	default empty array	Anti-cheat event flags

Table 4.5: Schema for sessions table

Table: `questions`
Stores AI-generated questions belonging to a session.

Column	**Data Type**	**Constraints**	**Description**
`id`	`UUID`	Primary Key	Unique question identifier
`session_id`	`UUID`	FK → `sessions.id`, NOT NULL	Owning session
`content`	`TEXT`	NOT NULL	The question text
`type`	`VARCHAR(30)`	NOT NULL	mcq or theory
`difficulty`	`VARCHAR(10)`	NOT NULL	easy, medium, or hard
`options`	`JSONB`	nullable	Array of 4 strings for MCQ
`correct_answer`	`TEXT`	nullable	Correct answer or model answer
`explanation`	`TEXT`	nullable	AI-generated explanation
`subject_id`	`VARCHAR(100)`	nullable	Subject identifier
`subject_name`	`VARCHAR(200)`	nullable	Subject display name
`subtopic_id`	`VARCHAR(100)`	nullable	Specific subtopic identifier
`subtopic_name`	`VARCHAR(200)`	nullable	Subtopic display name

Continued on next page

Table 4.6 – continued from previous page

Column	**Data Type**	**Constraints**	**Description**
`question_number`	`INTEGER`	nullable	Order within the session
`ai_generated`	`BOOLEAN`	default true	Always true in current version
`created_at`	`TIMESTAMP`	default current timestamp	Generation time

Table 4.6: Schema for questions table

Table: `answers`

Stores the user's response to each question in a session.

Column	**Data Type**	**Constraints**	**Description**
`id`	`UUID`	Primary Key	Unique answer identifier
`session_id`	`UUID`	FK → `sessions.id`, NOT NULL	Owning session
`question_id`	`UUID`	FK → `questions.id`, NOT NULL	Question being answered
`user_id`	`UUID`	FK → `users.id`, NOT NULL	User who answered
`response_text`	`TEXT`	nullable	The user's answer text
`response_code`	`TEXT`	nullable	Reserved for coding answers
`voice_transcript`	`TEXT`	nullable	Reserved for voice answers
`time_taken_sec`	`INTEGER`	nullable	Time spent on this question
`submitted_at`	`TIMESTAMP`	default current timestamp	Submission time

Table 4.7: Schema for answers table

Table: `evaluations`

Stores the AI evaluation result for each submitted answer.

Column	Data Type	Constraints	Description
`id`	`UUID`	Primary Key	Unique evaluation identifier
`answer_id`	`UUID`	FK → `answers.id`, nullable	The evaluated answer
`score`	`NUMERIC(4,2)`	nullable	Score from 0 to 10
`is_correct`	`BOOLEAN`	default false	True if meets correct threshold
`feedback`	`TEXT`	nullable	AI-generated feedback
`correct_answer`	`TEXT`	nullable	Correct answer + explanation
`strengths`	`JSONB`	nullable	Array of strength strings
`weaknesses`	`JSONB`	nullable	Array of weakness strings
`evaluated_by`	`VARCHAR(20)`	default 'gemini'	gemini or groq
`created_at`	`TIMESTAMP`	default current timestamp	Evaluation time

Table 4.8: Schema for evaluations table

Table: `skill_vectors`

One row per user. Stores the EMA-updated skill score for each subject on a 0 to 10 scale.

Column	Data Type	Constraints	Description
`id`	`UUID`	Primary Key	Unique record identifier
`user_id`	`UUID`	FK → `users.id`, UNIQUE	One record per user
`dsa`	`NUMERIC(4,2)`	default 0	Data Structures and Algorithms

Continued on next page

Table 4.9 – continued from previous page

Column	**Data Type**	**Constraints**	**Description**
os	NUMERIC(4,2)	default 0	Operating Systems
dbms	NUMERIC(4,2)	default 0	Database Management Systems
cn	NUMERIC(4,2)	default 0	Computer Networks
oops	NUMERIC(4,2)	default 0	Object Oriented Programming
se	NUMERIC(4,2)	default 0	Software Engineering
system_design	NUMERIC(4,2)	default 0	System Design
aptitude	NUMERIC(4,2)	default 0	Aptitude and Reasoning
guesstimation	NUMERIC(4,2)	default 0	Guesstimation
case_study	NUMERIC(4,2)	default 0	Case Studies
python	NUMERIC(4,2)	default 0	Python Programming
java	NUMERIC(4,2)	default 0	Java Programming
web_node	NUMERIC(4,2)	default 0	Web and Node.js
ai	NUMERIC(4,2)	default 0	Artificial Intelligence
ml	NUMERIC(4,2)	default 0	Machine Learning
deep_learning	NUMERIC(4,2)	default 0	Deep Learning
coa	NUMERIC(4,2)	default 0	Computer Organisation
compiler	NUMERIC(4,2)	default 0	Compiler Design
toc	NUMERIC(4,2)	default 0	Theory of Computation
discrete_math	NUMERIC(4,2)	default 0	Discrete Mathematics
linear_algebra	NUMERIC(4,2)	default 0	Linear Algebra
engineering_math	NUMERIC(4,2)	default 0	Engineering Mathematics
statistics	NUMERIC(4,2)	default 0	Statistics and Probability

Continued on next page

Table 4.9 – continued from previous page

Column	**Data Type**	**Constraints**	**Description**
`cryptography`	`NUMERIC(4,2)`	default 0	Cryptography
`graphics`	`NUMERIC(4,2)`	default 0	Computer Graphics
`mobile`	`NUMERIC(4,2)`	default 0	Mobile Development
`competitive`	`NUMERIC(4,2)`	default 0	Competitive Programming
`last_updated`	`TIMESTAMP`	nullable	Time of most recent score update

Table 4.9: Schema for skill_vectors table

Table: `subtopic_scores`
Granular subtopic-level scoring. Structured for Phase B.

Column	**Data Type**	**Constraints**	**Description**
`id`	`UUID`	Primary Key	Unique record identifier
`user_id`	`UUID`	FK → `users.id`, NOT NULL	User identifier
`subtopic_id`	`VARCHAR(100)`	NOT NULL	Curriculum ID e.g. dsa_dp
`subtopic_name`	`VARCHAR(200)`	nullable	e.g. Dynamic Programming
`subject_id`	`VARCHAR(100)`	nullable	Parent subject identifier
`score`	`NUMERIC(4,2)`	nullable	EMA score for this subtopic
`attempts`	`INTEGER`	default 0	Total questions attempted
`correct`	`INTEGER`	default 0	Total correct answers
`avg_time_seconds`	`NUMERIC(8,2)`	nullable	Avg time spent per question

Table 4.10: Schema for subtopic_scores table

Table: `session_feedback`
Stores user-submitted feedback for each completed session.

Column	Data Type	Constraints	Description
`id`	`UUID`	Primary Key	Unique record identifier
`session_id`	`UUID`	FK → `sessions.id`, NOT NULL	The rated session
`user_id`	`UUID`	FK → `users.id`, NOT NULL	User who submitted feedback
`overall_rating`	`INTEGER`	NOT NULL	Star rating from 1 to 5
`diff_appropriate`	`BOOLEAN`	nullable	Was the difficulty appropriate
`questions_relevant`	`BOOLEAN`	nullable	Were the questions relevant
`comments`	`TEXT`	nullable	Open-ended feedback text
`created_at`	`TIMESTAMP`	default current timestamp	Submission time

Table 4.11: Schema for session_feedback table

Table: `cheat_events`
Reserved for anti-cheat system (Phase B implementation).

Column	Data Type	Constraints	Description
`id`	`UUID`	Primary Key	Unique event identifier
`session_id`	`UUID`	FK → `sessions.id`, NOT NULL	Session of occurrence
`user_id`	`UUID`	FK → `users.id`, NOT NULL	User involved
`event_type`	`VARCHAR(50)`	NOT NULL	e.g. tab_switch, face_not_detected

Continued on next page

Table 4.12 – continued from previous page

Column	**Data Type**	**Constraints**	**Description**
`event_metadata`	`JSONB`	nullable	Additional context
`occurred_at`	`TIMESTAMP`	default current timestamp	Event time

Table 4.12: Schema for cheat_events table

Table: `conversation_logs`
Reserved for AI interview simulation multi-turn conversations.

Column	**Data Type**	**Constraints**	**Description**
`id`	`UUID`	Primary Key	Unique log identifier
`session_id`	`UUID`	FK → `sessions.id`, NOT NULL	Associated session
`user_id`	`UUID`	FK → `users.id`, NOT NULL	User in the conversation
`speaker`	`VARCHAR(20)`	NOT NULL	user or agent name
`message`	`TEXT`	NOT NULL	Message content
`agent_role`	`VARCHAR(100)`	nullable	e.g. Technical Interviewer
`timestamp`	`TIMESTAMP`	default current timestamp	Message time

Table 4.13: Schema for conversation_logs table

4.3.3 Relational Schema Dependencies

The relational schema diagram visually maps the exact foreign key constraints between the database tables. This schema illustrates the physical implementation of data integrity, where directional arrows originate from the foreign key (FK) and point directly to the referenced primary key (PK).

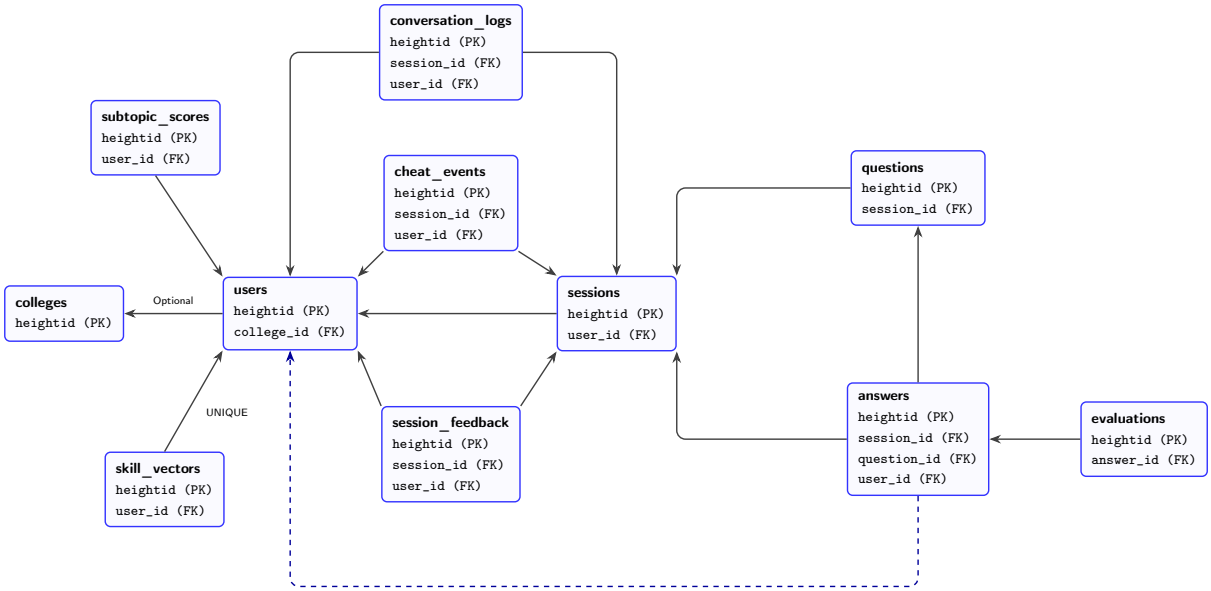

Figure 4.3: Relational Schema showing Foreign Key to Primary Key dependencies. Typographical glitch in Primary Key identifiers has been resolved.

Description of Relational Dependencies and Constraints:

The relational schema is designed to enforce strict referential integrity across the system. The following logic governs the primary data flows:

- **Identity and Institutional Context:** The `users` table is the primary anchor. It maintains an optional relationship with `colleges` via `college_id`, allowing for multi-tenant deployment across different institutions.
- **The Practice Workflow:** Every `session` must be owned by a `user`. Each session then acts as a parent for multiple `questions`. This hierarchy ensures that questions are scoped specifically to a single attempt.
- **Answer-Evaluation Pipeline:** The `answers` table serves as a junction, linking a `user`, a specific `question`, and the parent `session`. Every answer is then mapped to exactly one `evaluation`, which stores the AI-generated score and feedback.
- **Longitudinal Tracking:** The `skill_vectors` and `subtopic_scores` tables are tied directly to the `user`. Notably, `skill_vectors` maintains a 1:1 relationship with a `UNIQUE` constraint, ensuring each user has exactly one master proficiency record that is updated incrementally via the EMA algorithm.
- **Audit and Metadata:** Tables such as `cheat_events`, `session_feedback`, and `conversation_logs` maintain double-foreign keys to both `users` and `sessions`. This redundancy allows the system to query logs either by a specific session for a detailed report or by a specific user for long-term behavioral analysis.

4.4 API Design

The communication between the Next.js frontend and the FastAPI backend is conducted via a RESTful API layer. To ensure secure data transmission, all protected routes require a JSON Web Token (JWT) provided in the `Authorization` header as a `Bearer` token.

Table 4.14 outlines the primary endpoints, their access requirements, and their core responsibilities.

Method	**Endpoint**	**Auth**	**Description**
`POST`	`/api/auth/register`	No	Register a new user account with validation
`POST`	`/api/auth/login`	No	Authenticate user and receive JWT access token
`GET`	`/api/auth/me`	Yes	Retrieve current logged-in user's profile
`POST`	`/api/sessions/start`	Yes	Create a session record and generate questions via LLM
`GET`	`/api/sessions/{id}`	Yes	Retrieve session details and all generated questions
`POST`	`/api/sessions/answers/submit`	Yes	Save or update one answer for one question
`PUT`	`/api/sessions/{id}/end`	Yes	Trigger AI evaluation of all answers and complete session
`GET`	`/api/sessions/{id}/report`	Yes	Retrieve the full scored report for a completed session
`GET`	`/api/sessions/dashboard/stats`	Yes	Get aggregated statistics for the dashboard
`POST`	`/api/sessions/{id}/feedback`	Yes	Submit star rating and qualitative feedback
`GET`	`/api/sessions/{id}/feedback`	Yes	Retrieve existing feedback to prefill the form

Table 4.14: MockAI REST API Endpoint Specification and Routing Table

4.5 Authentication Flow

The MockAI platform employs a stateless, token-based authentication mechanism. The authentication module is strictly divided into two distinct operational flows: User Registration and User Login.

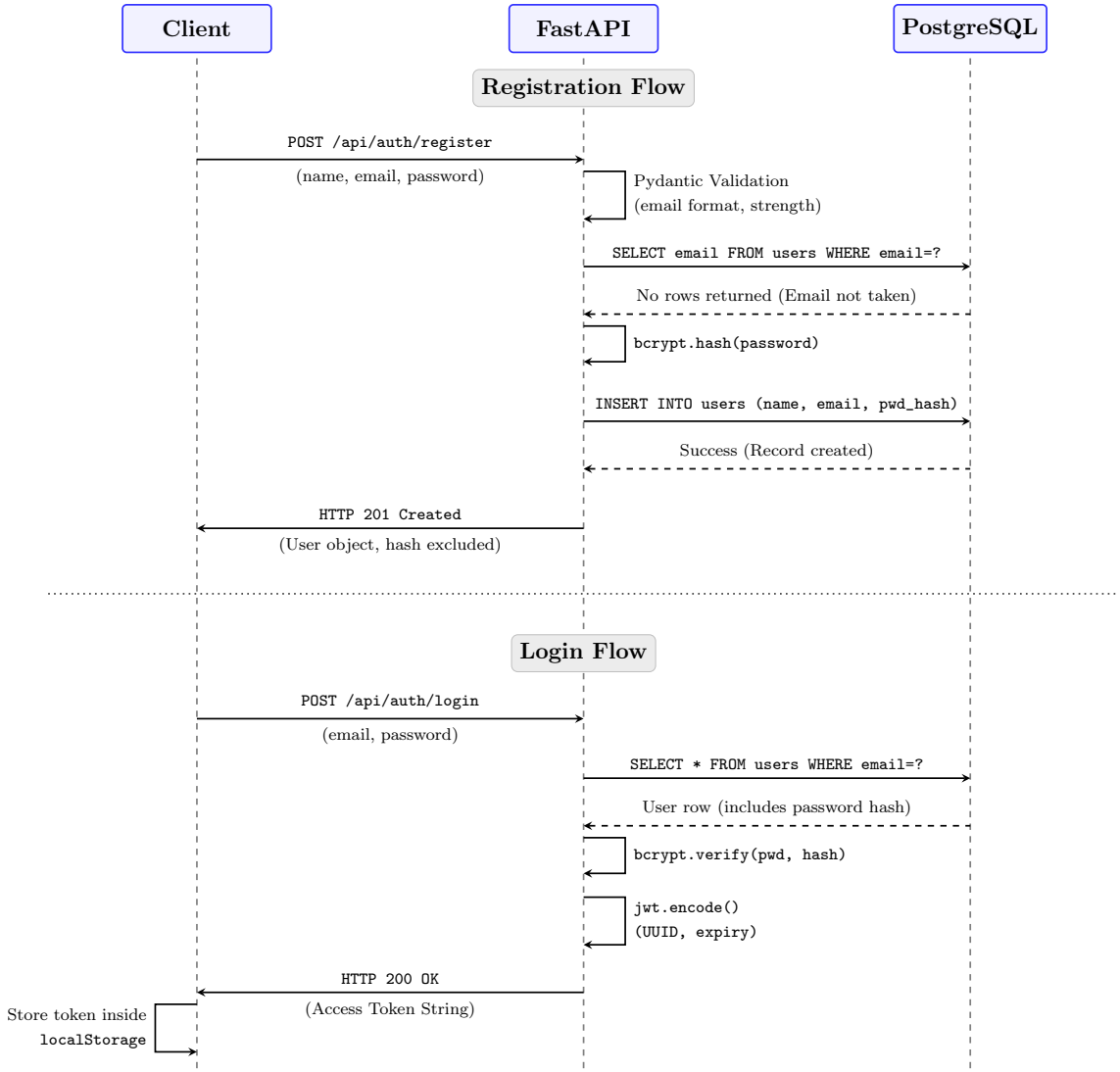

Figure 4.4: UML Sequence Diagram defining the timeline and interactions between the Client, FastAPI backend, and PostgreSQL database during Authentication.

Flow Execution Details

1. **Registration Flow:**

   - The Client transmits a `POST` request to `/api/auth/register` containing the user's name, email, and raw password.

- FastAPI intercepts the request and enforces stringent validation constraints utilizing **Pydantic**. The schema ensures the email is correctly formatted and verifies password entropy (enforcing a minimum of 8 characters, at least one uppercase letter, one digit, and one special character).
- The backend issues a `SELECT` query to PostgreSQL to confirm the email is unique. Upon receiving no conflicting rows, FastAPI calls `bcrypt` to securely salt and hash the password.
- An `INSERT` query persists the new user record. The system subsequently responds to the client with an `HTTP 201 Created` status, passing back the sanitized user object (with the password hash explicitly omitted).

2. **Login Flow:**

- The Client initiates a `POST` request to `/api/auth/login` with credentials.
- FastAPI executes a `SELECT` query targeting the provided email. The database returns the corresponding user row, critically including the stored hash.
- The API invokes `bcrypt.verify()` to mathematically compare the incoming plaintext password against the stored hash.
- Upon successful validation, the system generates an access token using `jwt.encode()`, embedding the user's unique `UUID` and an expiration timestamp.
- The backend responds with `HTTP 200 OK` and the token payload. The Client secures this token in `localStorage` and automatically appends it as a `Bearer` token in the `Authorization` header of all subsequent API calls.

4.6 Question Generation Pipeline

The MockAI platform utilizes a robust, multi-stage pipeline to generate domain-specific interview questions. The system leverages Gemini 2.0 Flash as its primary generative engine, with an automated fallback mechanism to Groq's LLaMA 3.3 70B model to ensure high availability and fault tolerance. Strict JSON parsing and schema validation are enforced before any generated content is persisted to the database.

Figure 4.5 outlines the systematic flow from receiving the initial frontend request to writing validated questions into the PostgreSQL database.

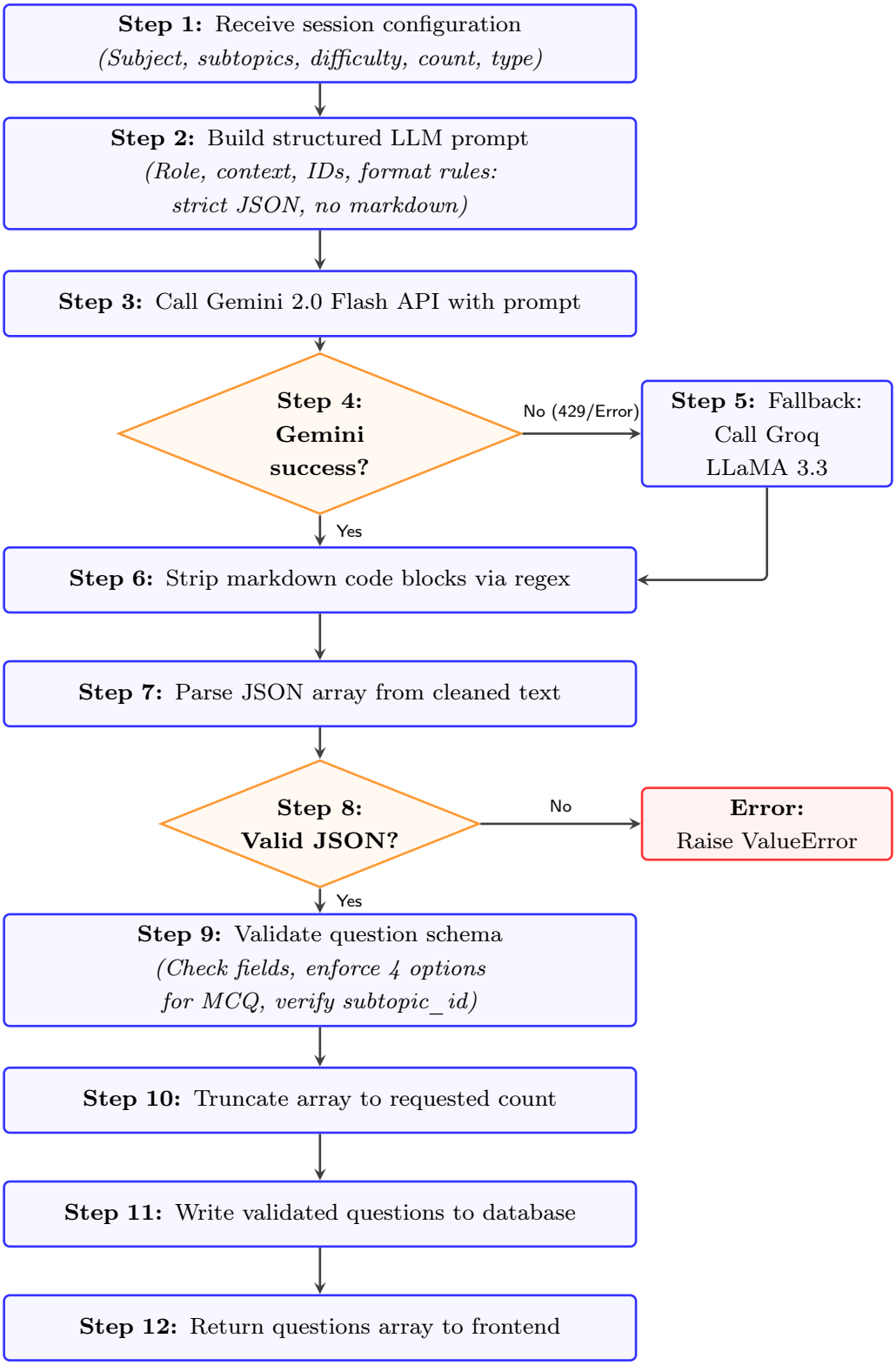

Figure 4.5: Question Generation Pipeline flowchart demonstrating the LLM interaction cycle, dynamic fallback system, and strict JSON validation layers.

Pipeline Execution Details

The question generation process follows a strict sequence to ensure data integrity and system reliability:

1. **Configuration & Prompting (Steps 1–2):** The backend receives the user's session parameters (subject, difficulty, count) and constructs a highly structured prompt. This prompt assigns a specific persona (e.g., Senior Technical Interviewer) to the LLM and enforces strict output rules, explicitly banning markdown formatting to ensure clean data retrieval.

2. **Primary Generation & Fallback (Steps 3–5):** The system first attempts to call the **Gemini 2.0 Flash API**. Because relying on a single third-party LLM introduces a single point of failure, an automatic fallback is implemented. If Gemini fails (due to HTTP 429 quota limits or API timeouts), the pipeline seamlessly reroutes the identical prompt to the **Groq LLaMA 3.3 70B** API.

3. **Sanitization & Parsing (Steps 6–8):** LLMs occasionally inject markdown code blocks (e.g., ```` ```json ````) despite explicit instructions. A Regex cleaner strips these artifacts before the system attempts to parse the raw string into a JSON array. If parsing fails, the pipeline traps the error and raises a `ValueError` rather than corrupting the database.

4. **Validation & Persistence (Steps 9–12):** Each parsed question block undergoes strict schema validation. For instance, the system enforces that Multiple Choice Questions (MCQs) contain exactly four plausible options. Finally, the array is truncated to match the user's exact requested count and written to the PostgreSQL database.

4.7 Answer Evaluation Flow

The Answer Evaluation pipeline is triggered when a user completes a practice session. Because the system handles both deterministic Multiple Choice Questions (MCQs) and open-ended Theory questions, the evaluation logic dynamically forks. Theory questions undergo an advanced AI evaluation against a strict rubric, while MCQs utilize deterministic fuzzy matching paired with AI-generated explanations.

Figure 5.10 details the looping mechanism and evaluation paths executed for the submitted session.

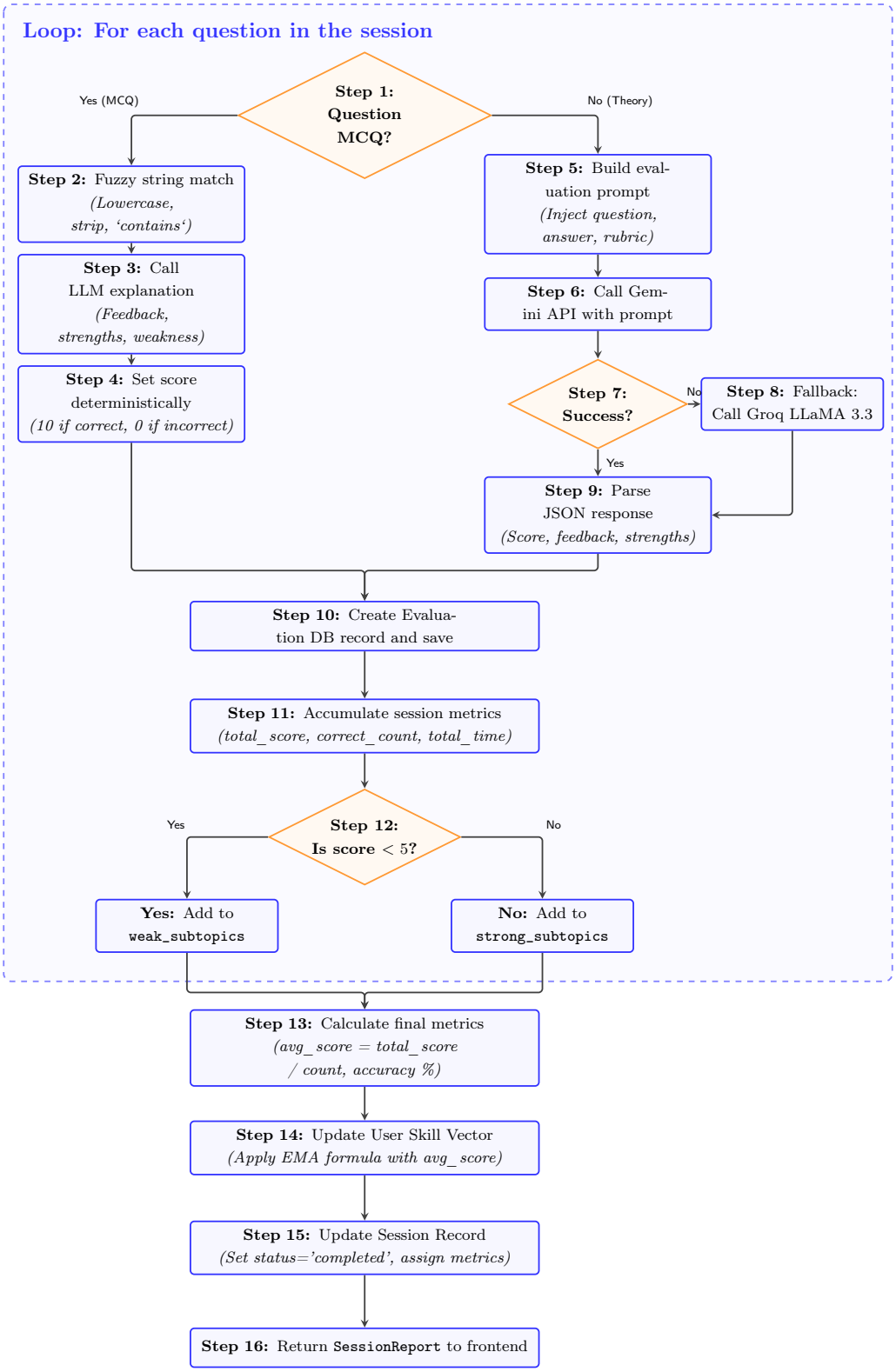

Figure 4.6: Answer Evaluation Flowchart displaying the per-question loop processing, AI rubric evaluation, subtopic tagging, and final performance aggregation.

Evaluation Pipeline Execution Details

The evaluation pipeline strictly enforces the following computational logic:

1. **Question Iteration & Forking (Steps 1–4):** The system loops over the user's submitted answers. If the question is an MCQ, the backend bypasses expensive AI grading and uses a deterministic "fuzzy string match". This ensures that formatting discrepancies (e.g., answering "BFS" instead of "Breadth-First Search") are correctly marked as 10/10 without utilizing model tokens. An LLM is only called to generate constructive explanations.

2. **Theory Grading via Rubric (Steps 5–9):** For descriptive theory questions, the system passes the question, the expected standard answer, and the user's raw text to the AI. Crucially, the prompt includes a rigid marking rubric (0-2 for entirely wrong, up to 9-10 for excellent). As with question generation, if Gemini 2.0 Flash fails, the system automatically redirects the request to Groq's LLaMA 3.3.

3. **Metrics Accumulation (Steps 10–12):** As each Evaluation record is saved to PostgreSQL, the system maintains a running tally of scores and time. A critical threshold check occurs here: any question scoring below a 5 automatically tags its corresponding subtopic ID into the `weak_subtopics` array, while a score of 5 or higher appends it to `strong_subtopics`.

4. **Longitudinal Updates (Steps 13–16):** Once the loop concludes, the global metrics (average score and accuracy percentage) are calculated. Before the final `SessionReport` is returned to the frontend, the system invokes the `update_skill_vector` function. This applies an Exponential Moving Average (EMA) mathematical formula to incrementally update the user's global proficiency score in that specific subject, effectively mapping their growth over time.

4.7.1 EMA Skill Update Logic

To accurately track a user's proficiency over time, the MockAI platform employs an Exponential Moving Average (EMA) algorithm. Rather than calculating a simple arithmetic mean—which treats a score from six months ago with the same weight as a score from today—the EMA formula dynamically weights recent performance more heavily while retaining historical context.

After every completed practice session, the system calculates the user's average score across all questions in that session (on a scale of 0.00 to 10.00) and applies the EMA formula to update the respective subject column in the `skill_vectors` database table.

The Mathematical Model:
The skill update logic is defined by the following equation:

$$S_{new} = \alpha \times S_{session} + (1 - \alpha) \times S_{prev} \tag{4.1}$$

Where:

- S_{new} is the newly calculated proficiency score to be saved to the database.
- $S_{session}$ is the average score achieved in the just-completed session.
- S_{prev} is the user's existing historical score for that specific subject.
- α is the smoothing constant (weighting factor). For MockAI, this is strictly set to **0.3**, ensuring that recent performance accounts for 30% of the score shift, while historical baseline accounts for 70%.

Worked Example of Score Progression

To demonstrate how the EMA formula stabilizes a user's score and prevents erratic jumps, Table 4.15 traces a theoretical user starting with a baseline score of 0.00 through five consecutive practice sessions.

Session	**Session Score ($S_{session}$)**	**Prev Score (S_{prev})**	**Calculation Step**	**New Score (S_{new})**
1	6.6	0.00	$0.3(6.6) + 0.7(0.00)$	**1.98**
2	5.3	1.98	$0.3(5.3) + 0.7(1.98)$	**2.98**
3	7.6	2.98	$0.3(7.6) + 0.7(2.98)$	**4.37**
4	4.4	4.37	$0.3(4.4) + 0.7(4.37)$	**4.38**
5	8.0	4.38	$0.3(8.0) + 0.7(4.38)$	**5.47**

Table 4.15: Step-by-step EMA progression demonstrating how volatile session scores are smoothed into a stable proficiency vector.

Analysis of the Example:

Notice what happens in Session 4: The user performs poorly (scoring a 4.4), which is lower than their baseline of 4.37. However, because $\alpha = 0.3$, the system does not punish them severely for a single bad session. Their score stabilizes at 4.38. Conversely, in Session 5, an excellent performance of 8.0 pulls their historical average up aggressively past the 5.0 milestone.

4.8 Data Flow Diagrams

Data Flow Diagrams (DFDs) provide a graphical representation of the flow of information through the system. They map out external entities, data stores, and processes, illustrating exactly what data is exchanged across the system's boundaries.

4.8.1 DFD Level 0 — Context Diagram

The Level 0 DFD, also known as a Context Diagram, abstracts the entire MockAI platform into a single central process. It identifies the external entities that interact with the system and strictly defines the data inputs and outputs crossing the system boundary.

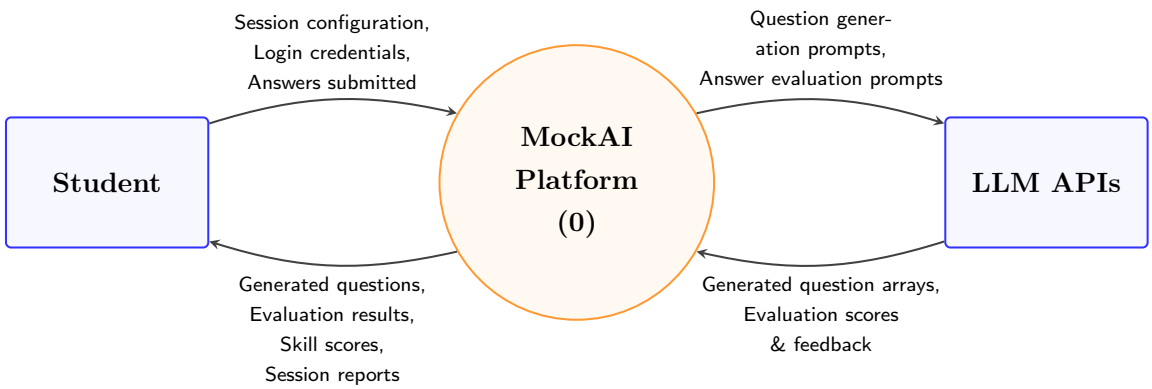

Figure 4.7: DFD Level 0 (Context Diagram) mapping the high-level data boundaries between the user, the core MockAI Platform, and External LLM providers.

Context Level Breakdown

- **Process 0 (MockAI Platform):** The core application logic, encompassing both the Next.js frontend and the FastAPI backend. It acts as the central orchestrator for all data transformations.
- **Student (External Entity):** The primary human actor. The Student feeds the system configuration parameters and raw text answers, and in return, extracts structured data such as AI-generated questions and highly specific performance reports.
- **LLM APIs (External Entity):** The external third-party services (Gemini 2.0 Flash and Groq LLaMA 3.3). The system securely transmits engineered prompts and context logic to this entity, receiving back structured JSON arrays representing questions and graded rubrics.

4.8.2 DFD Level 1 — Internal Process Logic

The Level 1 Data Flow Diagram breaks down the single Level 0 process into six distinct sub-processes. This layer maps the precise routing of data payloads between the user interface, internal logic modules, third-party APIs, and the PostgreSQL database (represented as six distinct Data Stores).

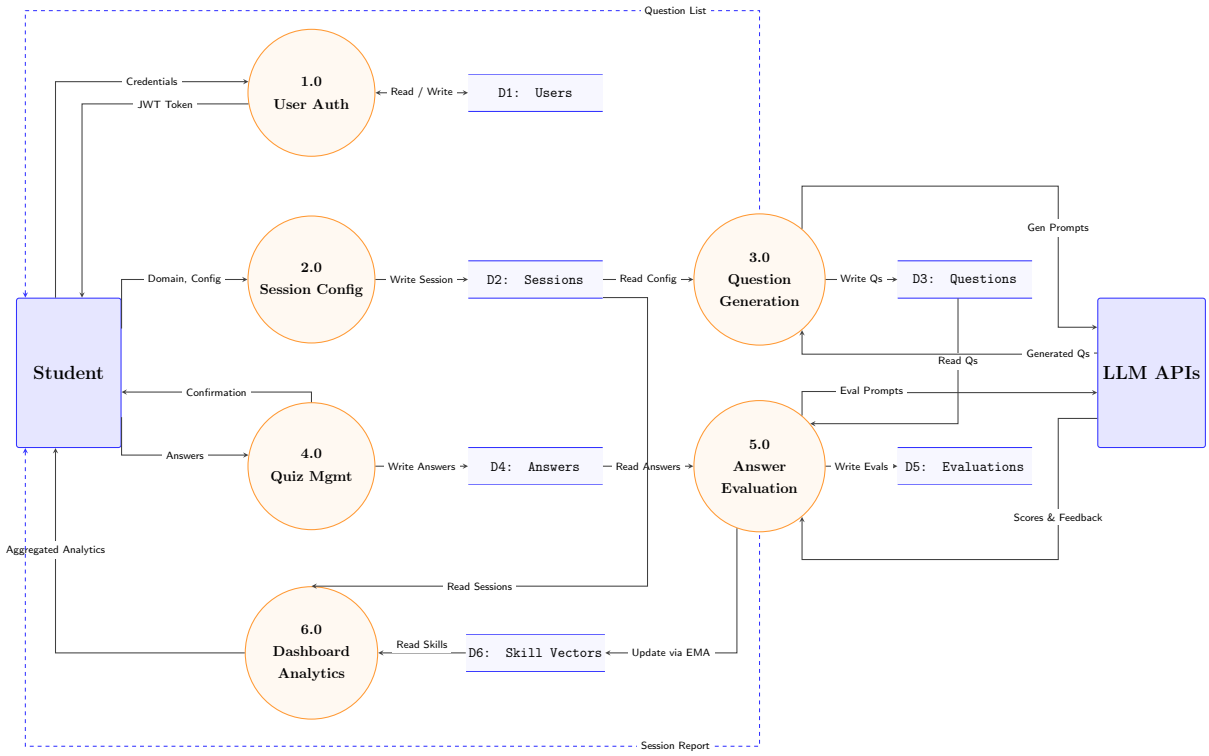

Figure 4.8: DFD Level 1 isolating the six core operational processes, internal data stores, and external cross-system routing. Line paths are mathematically anchored to prevent overlap and maximize visibility.

Level 1 Execution Logic

1. **Process 1.0 (Auth) & 2.0 (Config):** The Student initiates interaction by authenticating against **D1**, receiving a JWT. They then submit session parameters (domain, subject, difficulty), which are initialized as a pending record in **D2**.

2. **Process 3.0 (Question Generation):** This process retrieves the parameters from **D2** and constructs prompts for the **LLM APIs**. Upon receiving the generated questions, it persists them in **D3** and dispatches the formatted list to the Student via a distinct cross-system return channel.

3. **Process 4.0 (Quiz Management):** Acts as a lightweight passthrough during the active test, rapidly streaming the Student's responses directly into **D4** to prevent

data loss.

4. **Process 5.0 (Answer Evaluation):** The heaviest operational node. It joins data from **D3** (expected questions) and **D4** (user answers), forwarding them to the **LLM APIs** for rubric grading. It writes the results to **D5**, returns the comprehensive report to the Student, and critically, triggers the EMA mathematical update to the user's proficiency record in **D6**.

5. **Process 6.0 (Dashboard Analytics):** A read-only analytical engine. It aggregates historical completion data from **D2** and current EMA proficiency vectors from **D6** to render the Student's global dashboard.

4.9 Sequence Diagrams

Sequence diagrams provide a chronological, step-by-step mapping of how different system components communicate over time. The following diagram illustrates the complex, multi-tiered orchestration required to initialize a new practice session.

4.9.1 Session Start Sequence

Initializing a new session requires synchronous coordination between the frontend client, the backend API, the PostgreSQL database, and the external LLM provider. Figure 4.9 details this exact timeline.

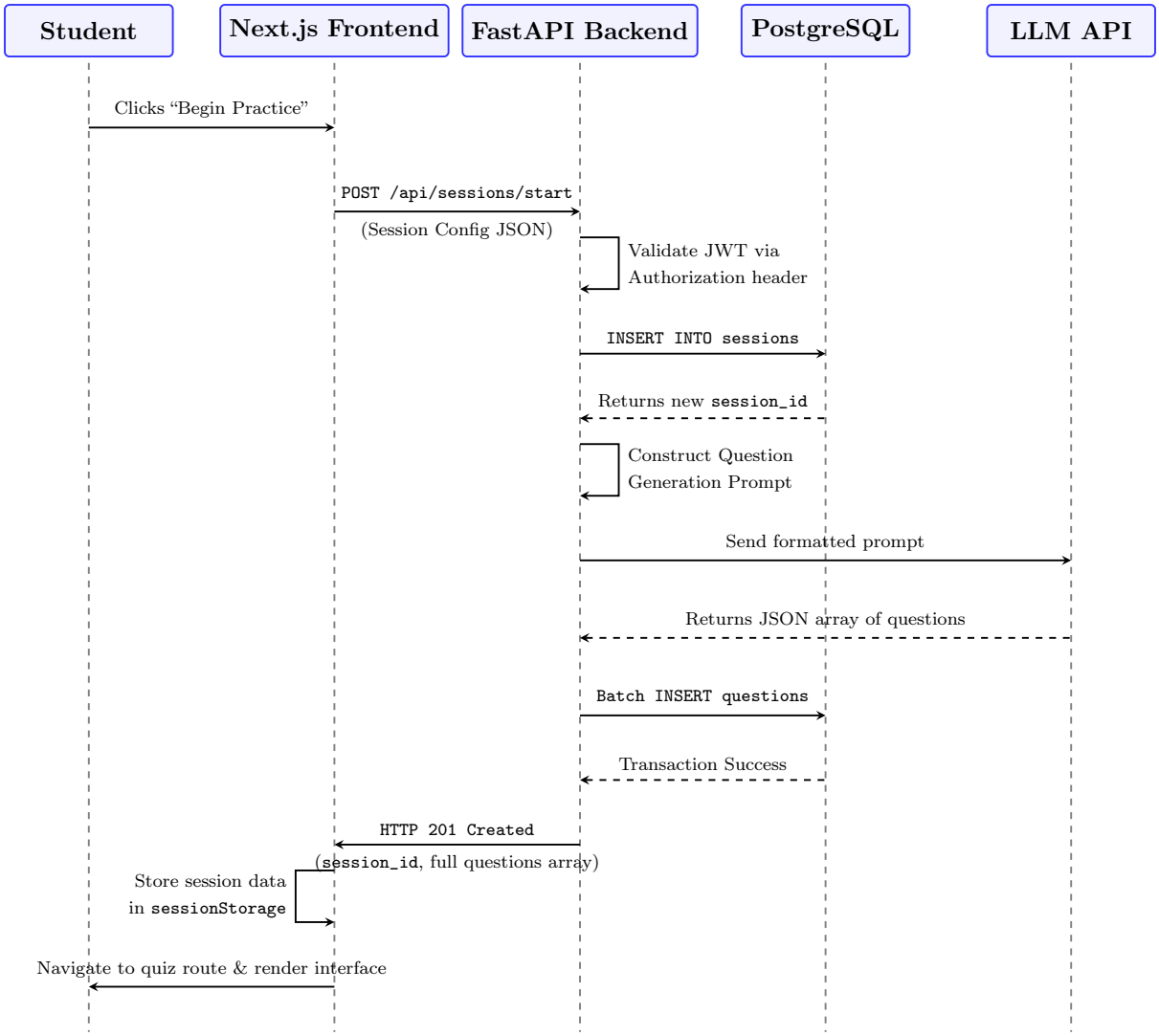

Figure 4.9: UML Sequence Diagram defining the timeline and interactions between the Client, Frontend, Backend, Database, and LLM API during session initialization.

Sequence Execution Details

1. **Trigger & Transmission:** The flow begins when the Student clicks "Begin Practice". The Next.js frontend immediately packages the selected configuration parameters into a JSON payload and dispatches a `POST` request to the FastAPI backend.

2. **Authentication & Persistence:** FastAPI intercepts the request to mathematically validate the user's `JWT`. Once verified, the backend executes an `INSERT` transaction into PostgreSQL to create a pending session record, retrieving the newly generated `session_id`.

3. **LLM Orchestration:** FastAPI constructs a highly engineered prompt incorporating the user's parameters and dispatches it to the LLM API. The LLM processes

the prompt and returns a strictly formatted JSON array containing the generated questions.

4. **Data Binding & State Management:** FastAPI performs a batch `INSERT` into PostgreSQL, permanently linking the generated questions to the new `session_id`. The backend then resolves the original HTTP request by sending a `201 Created` response back to the frontend alongside the complete question payload.

5. **Client Rendering:** The Next.js frontend catches the response, persists the state in the browser's `sessionStorage` to protect against accidental page refreshes, and finally pushes the user to the active quiz route to begin the exam.

4.9.2 Session End Sequence

The Session End flow represents the most computationally complex orchestration in the MockAI architecture. It involves retrieving persisted user data, executing an iterative evaluation loop against the LLM API, calculating final metrics, and updating the user's proficiency tracking via the Exponential Moving Average (EMA) mathematical formula.

Figure 4.10 illustrates the synchronous data exchanges required to securely finalize a session and return the computed analytics to the client.

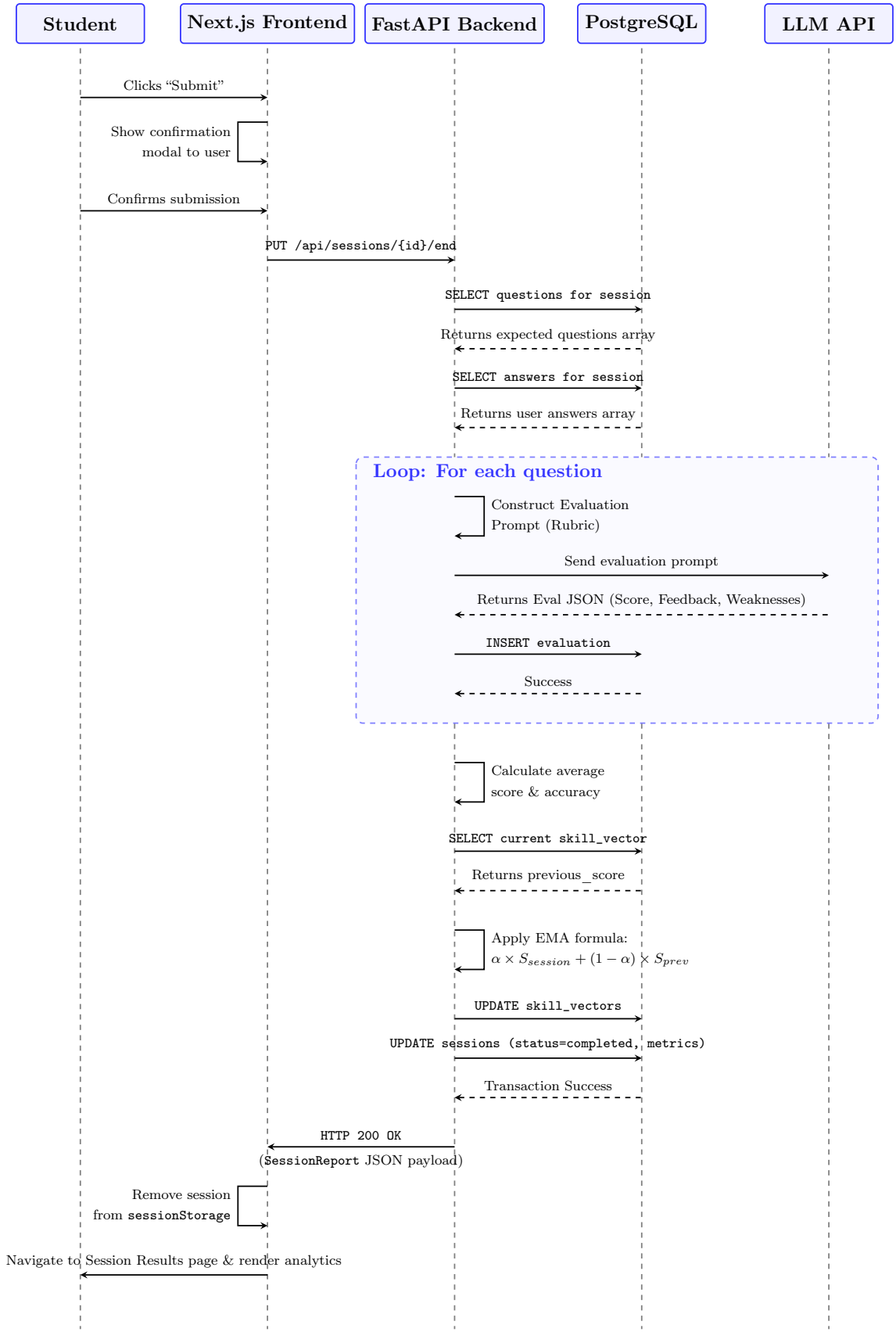

Figure 4.10: UML Sequence Diagram displaying the complex iterative evaluation loop, external LLM interactions, and EMA metrics calculation triggered during session termination.

Sequence Execution Details

1. **Client Confirmation:** Because submitting an exam is a destructive action, the Next.js frontend catches the submit event and triggers a protective confirmation modal before any network requests are dispatched.

2. **Data Hydration:** Upon receiving the `PUT` request, FastAPI independently queries PostgreSQL to retrieve both the original `questions` array and the user's `answers` array, mapping them into evaluation pairs.

3. **Iterative AI Evaluation (The Loop):** The backend enters a strict loop. For each question, it constructs a prompt incorporating the user's answer and the strict 0-10 marking rubric. It calls the **LLM API**, parses the JSON response, and persists the generated Evaluation record to PostgreSQL.

4. **Longitudinal Metrics (EMA):** Once the loop concludes, the global session metrics are computed. FastAPI queries the database for the user's current proficiency score in that subject, applies the Exponential Moving Average formula, and writes the newly adjusted vector back to the database.

5. **Completion & Redirection:** The session is securely flagged as `completed` to prevent future mutations. The backend returns the final `SessionReport` to the client, which flushes its local state storage and redirects the user to the analytics dashboard.

4.10 System Class Diagram

The structural design of the MockAI backend is centered on a decoupled architecture. Data Models encapsulate the state of the application within the PostgreSQL database, while Service Classes handle the computational logic, such as LLM prompt engineering and the Exponential Moving Average (EMA) proficiency updates.

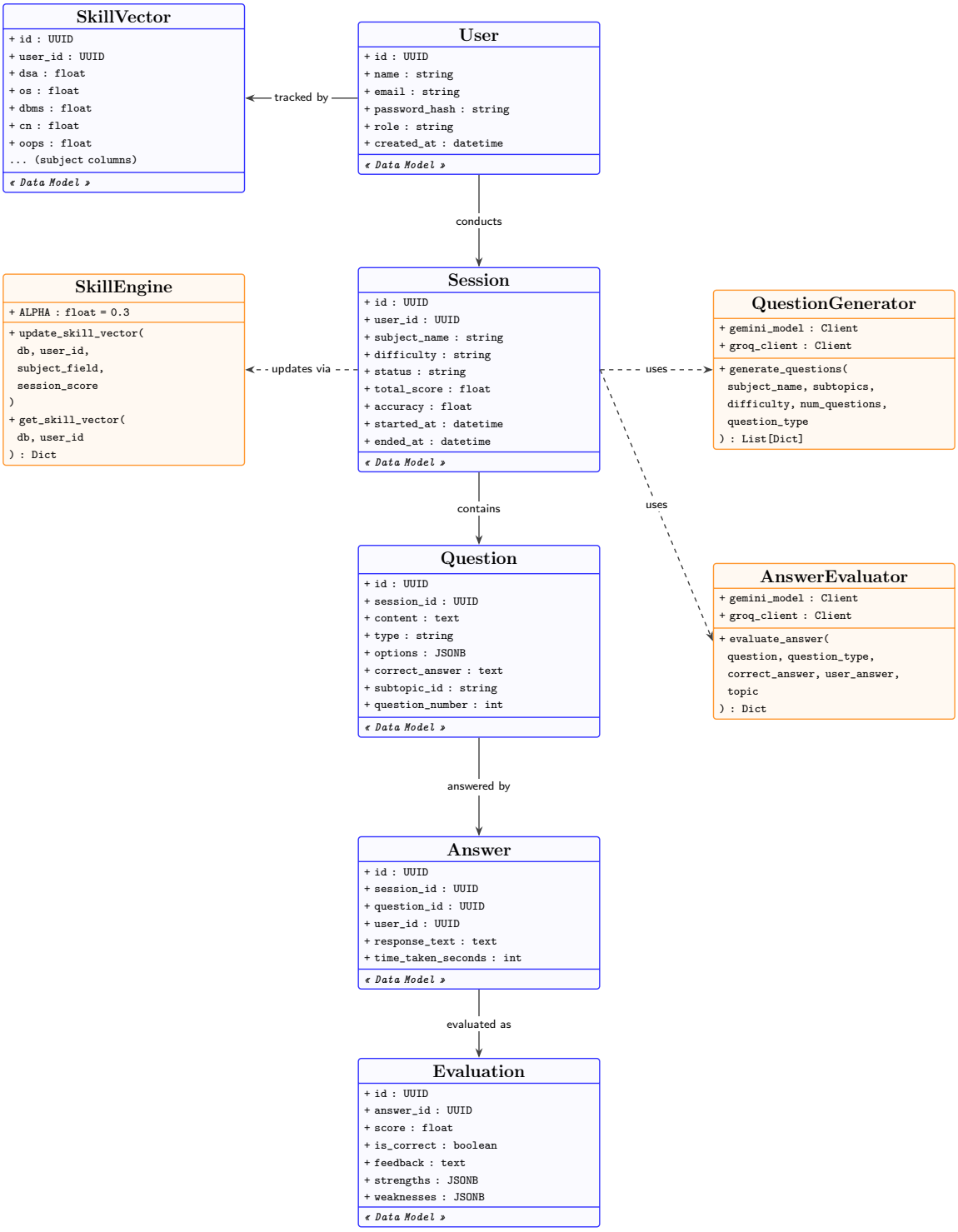

Figure 4.11: UML Class Diagram for MockAI, illustrating the relationship between core data entities and logic services.

Architectural Execution Details

The system's structural integrity is maintained through the following design principles:

1. **Data Persistence Layer:** Each database model (User, Session, etc.) is defined as

a static entity. This ensures that the state of a user's practice history and proficiency levels is accurately tracked and indexed via UUIDs for high performance.

2. **AI Orchestration:** The `QuestionGenerator` and `AnswerEvaluator` classes encapsulate the complexity of LLM interactions. By keeping these as separate services, the system can easily switch between Gemini and Groq without modifying the underlying data structures.

3. **Mathematical Tracking:** The `SkillEngine` acts as the repository for the EMA logic. By separating this from the User model, proficiency updates can be calculated independently and asynchronously after a session concludes.

4.11 Quiz Interface State Machine

The UI maintains a persistent state for each individual question within a session. This state dictates the styling of the question navigation grid, providing immediate visual feedback to the student. Figure 4.12 illustrates the transitions between the five distinct question statuses.

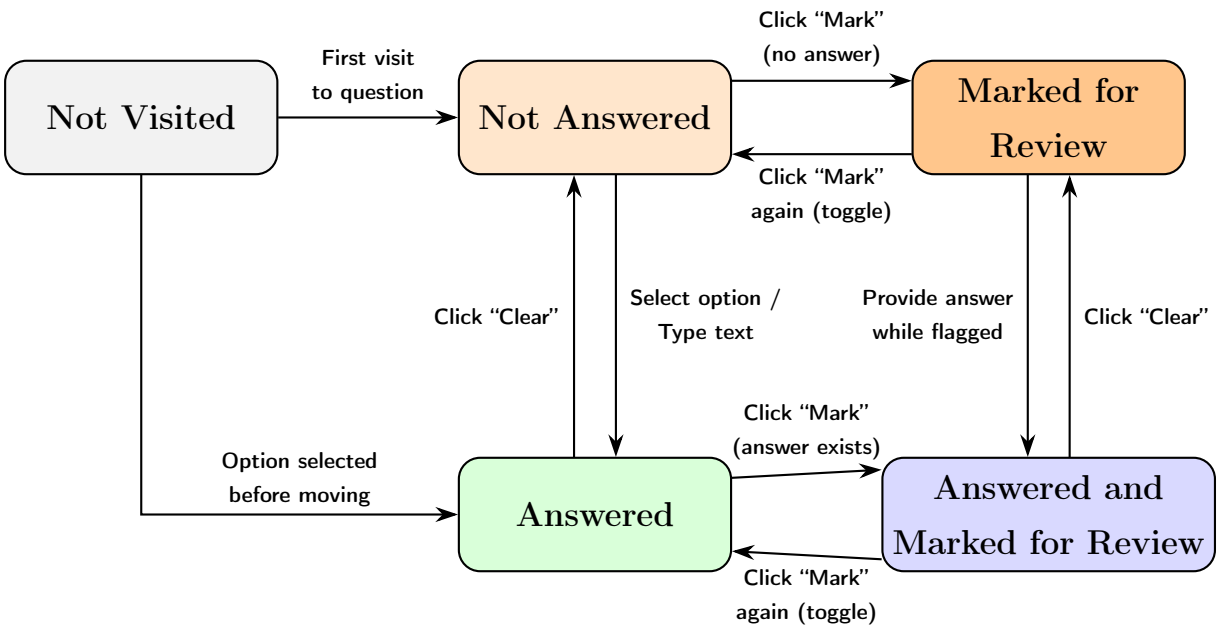

Figure 4.12: UML State Machine Diagram representing the question lifecycle with transparent text transitions.

State Logic Definitions

The quiz logic enforces the following state definitions to assist user navigation:

- **Not Visited:** The default state for all questions upon session initialization.

- **Not Answered:** The active state after a question is viewed but before input is provided.
- **Answered:** Indicates valid input (MCQ selection or theory text) has been persisted.
- **Marked for Review (Unanswered):** A flag used for questions the student intentionally skips for later consideration.
- **Answered and Marked for Review:** Indicates a provided answer that requires a final sanity check before submission.

4.12 Page Flow Diagram

The navigational design of MockAI is centered on a seamless transition between configuration, active testing, and performance review. Figure 4.13 details the navigational pathways and the logic governing transitions between primary application routes.

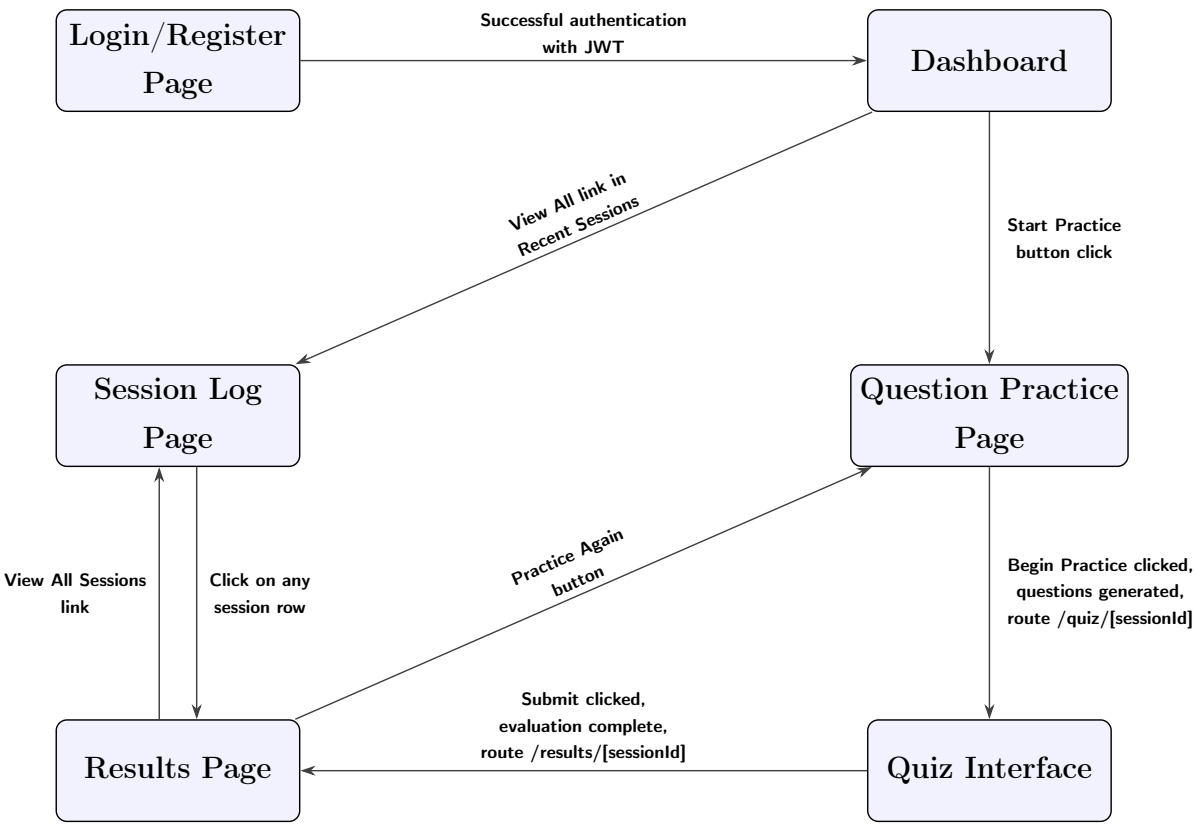

Figure 4.13: Application Page Flow Diagram illustrating user navigation paths, route triggers, and dynamic session routing logic.

Navigation Logic Summary

The application's routing logic is governed by the following technical constraints:

- **Dynamic Routing**: Transitions between `Question Practice`, `Quiz Interface`, and `Results` utilize dynamic path parameters (`/route/[sessionId]`) to ensure data persistence across a single testing instance.
- **State Persistence**: The `Session Log Page` serves as the primary entry point for historical reviews, allowing the system to re-hydrate the `Results Page` for any completed session stored in the PostgreSQL database.
- **Auth-Gating**: Every transition shown above (excluding the initial Login/Register view) is intercepted by a middleware layer that verifies the presence and validity of the JWT token.

4.13 Curriculum Taxonomy

The MockAI curriculum is organized into a modular, three-level hierarchy: **Domain** → **Subject** → **Subtopic**. This taxonomy ensures that the generative AI models and the Skill Engine operate within a bounded, structured knowledge space.

Table 4.16 outlines the complete curriculum, categorized by active modules (Phase A) and the projected development roadmap (Phases B and C).

Domain	Development Phase	Subjects & Subtopics
Core Computer Science	**Phase A** (Active)	**DSA (19)**: Arrays, Linked Lists, Stacks, Queues, Trees, Graphs, Hashing, Sorting, Searching, Dynamic Programming, Greedy, Divide and Conquer, Bit Manipulation, Recursion, Heaps, Tries, Segment Trees, Disjoint Sets, Graph Algorithms. **Operating Systems** (12 subtopics) **DBMS** (11 subtopics)

Continued on next page...

Table 4.16 – continued from previous page

Domain	Development Phase	Subjects & Subtopics
		Computer Networks (11 subtopics) **OOP** (10 subtopics) **Software Engineering** (8 subtopics) **System Design** (10 subtopics)
Placement Preparation	**Phase A** (Active)	**Aptitude and Reasoning** (17 subtopics) **Guesstimation** (5 subtopics) **Case Studies** (5 subtopics)
Programming Languages	**Phase B** (Coming Soon)	Python, Java, Web and Node.js
AI and ML	**Phase B** (Coming Soon)	Artificial Intelligence, Machine Learning, Deep Learning
Systems	**Phase B** (Coming Soon)	Computer Organisation, Compiler Design, Theory of Computation
Mathematics	**Phase C** (Coming Soon)	Discrete Math, Linear Algebra, Engineering Math, Statistics, Cryptography
Specialized	**Phase C** (Coming Soon)	Computer Graphics, Mobile Development, Competitive Programming

Table 4.16: MockAI Curriculum Taxonomy detailing the three-level hierarchy (Domains, Subjects, Subtopics) across current and future development phases.

Taxonomy Hierarchy Definitions

The curriculum is structured to support the system's underlying mathematical proficiency models:

- **Level 1: Domains** — High-level categories representing distinct career tracks or academic branches (e.g., Core Computer Science).

- **Level 2: Subjects** — Core modules within a domain (e.g., Data Structures and Algorithms).
- **Level 3: Subtopics** — The atomic unit of the curriculum. Subtopics (such as *Binary Search Trees* or *Process Scheduling*) act as the definitive tags for AI prompt engineering and Skill Engine vector updates.

Chapter 5

Implementation

5.1 Development Environment

The development of the MockAI platform was conducted within a standardized local environment to ensure reproducibility and seamless integration between the frontend, backend, and containerized database services. The project was developed on a Windows 11 host machine, utilizing Visual Studio Code (VS Code) as the primary Integrated Development Environment (IDE).

Software Stack and Tooling

The local development stack comprises the following core technologies:

- **Backend Environment:** Python 3.12 managed within a `conda` virtual environment to strictly isolate project dependencies.
- **Frontend Environment:** Node.js (LTS version) for executing the Next.js framework and managing package dependencies via `npm`.
- **Containerization:** Docker Desktop is utilized to orchestrate local database and caching services, ensuring environment parity with future production deployments.

Service Orchestration and Startup

The local infrastructure is orchestrated using Docker Compose. A single command initializes the required background services:

```
> docker-compose up -d
```

This command spins up **PostgreSQL 15** exposed on port `5432` and **Redis 7** exposed on port `6379` as named containers. Once the infrastructure is running, the application layers are initialized via their respective runtimes:

- **Backend Server:** Started using the ASGI server Uvicorn from the root directory:
  ```
  > uvicorn backend.main:app -reload
  ```
- **Frontend Server:** Executed from within the frontend directory:
  ```
  > npm run dev
  ```

Configuration Management

System configurations and sensitive credentials are obfuscated from the source code using environment variables. These are stored securely in a `.env` file located in the project root. Key configurations include:

- **Database Connection URL:** Utilized by SQLAlchemy to establish PostgreSQL sessions.
- **JWT Secret Key:** A securely generated 128-character string used for cryptographic token signing.
- **External API Keys:** Authentication keys for external LLM providers, specifically the **Gemini API** and **Groq API**.

Database Schema Management

Database migrations and schema evolution are handled using **Alembic**. When modifications are made to the SQLAlchemy Object Relational Mapping (ORM) models, Alembic automatically generates the corresponding SQL Data Definition Language (DDL) operations.
To capture a model change and generate a new migration script, the following command is executed:

```
> alembic revision -autogenerate -m "description"
```

The resulting migration file contains the precise `ALTER TABLE`, `CREATE TABLE`, and `DROP COLUMN` statements required. To apply all pending migrations and synchronize the local database state, the upgrade command is used:

```
> alembic upgrade head
```

5.2 Project Directory Structure

The MockAI repository follows a monorepo architecture, logically separating the Python/-FastAPI backend from the TypeScript/Next.js frontend. This separation of concerns allows for independent scaling, testing, and deployment of the client and server components.

The following terminal output outlines the complete directory tree, highlighting the core configuration files, API routers, ORM models, and Next.js App Router paths.

Directory Tree

```
College_Capstone_Project/
|-- backend/                 [Python 3.12 / FastAPI]
|   |-- main.py              Entry point & router
|   |-- config.py            Env configurations
|   |-- database/
|   |   \-- connection.py    Async engine & Base
|   |-- middleware/
|   |   \-- auth.py          JWT decode & bcrypt
|   |-- models/              [SQLAlchemy Models]
|   |   |-- user.py          User schema
|   |   |-- session.py       Session schema
|   |   |-- question.py      Question schema
|   |   |-- answer.py        Answer schema
|   |   |-- evaluation.py    Evaluation schema
|   |   |-- skill_vector.py  SkillVector schema
|   |   |-- subtopic_scores.py Subtopic scores
|   |   \-- feedback.py      Feedback schema
|   |-- routers/
|   |   |-- auth.py          Auth endpoints
|   |   \-- sessions.py      Session endpoints
|   |-- schemas/             [Pydantic Models]
|   |   |-- auth.py          Auth validation
|   |   \-- sessions.py      Session validation
|   \-- services/
|       |-- question_gen.py  LLM question generation
|       |-- answer_eval.py   LLM answer evaluation
|       \-- skill_engine.py  EMA skill tracking
|
|-- frontend/                [TypeScript / Next.js]
|   |-- src/
|   |   |-- app/
|   |   |   |-- (auth)/login Login page
|   |   |   |-- (auth)/register Register page
|   |   |   |-- dashboard    Dashboard page
|   |   |   |-- question-prac Session config
|   |   |   |-- quiz/[id]    Dynamic Quiz UI
|   |   |   |-- results/[id] Results & feedback
|   |   |   \-- sessions     Session log page
|   |   |-- components/
|   |   |   |-- Navbar.tsx   Nav bar with auth
|   |   |   |-- TopicSelect.tsx Domain/subject picker
```

```
|   |   |   \-- SkillRadar.tsx Recharts radar chart
|   |   |-- context/
|   |   |   \-- AuthContext.tsx Global auth context
|   |   |-- hooks/
|   |   |   \-- useAuth.ts   Auth context hook
|   |   \-- lib/
|   |       |-- auth.ts      Token utilities
|   |       \-- curriculum.ts Static taxonomy
|
|-- alembic/
|   \-- versions/            SQL migration files
|-- docker-compose.yml       Postgres & Redis
\-- .env                     Git ignored secrets
\-- README.md                Readme file
```

Figure 5.1: MockAI Monorepo Directory Structure representing the complete technical stack layout.

5.3 Authentication Implementation

The authentication architecture of MockAI is designed around a stateless, token-based system ensuring secure access across the distributed frontend and backend services. The implementation prioritizes cryptographic security, explicit authorization boundaries, and seamless client-side state management.

Password Hashing Mechanics

When a user registers on the platform, plain text passwords are never stored. The backend utilizes the `bcrypt` library to hash passwords with a work factor of 12, resulting in a salted, 60-character hash string. During authentication, verification involves computing the hash of the incoming password and executing a timing-safe string comparison against the stored hash, effectively neutralizing timing side-channel attacks.

JWT Token Structure and Cryptography

Upon successful authentication, the server issues a JSON Web Token (JWT). The token's payload encodes the user's universally unique identifier (UUID) within the `sub` (subject) claim, alongside an expiration timestamp (`exp`) strictly limited to 30 minutes from issuance. The token is cryptographically signed using the HMAC SHA-256 (HS256)

algorithm paired with a highly secure 128-character secret key. In the event of a suspected secret key compromise, key rotation is mandatory, which subsequently invalidates all previously issued tokens.

Middleware Authorization Flow

Access control for protected backend endpoints is enforced via a FastAPI dependency injection model. Every protected route declares a `get_current_user` dependency. This middleware extracts the `Bearer` token from the incoming HTTP `Authorization` header, decodes it utilizing the `python-jose` library, and validates both the cryptographic signature and the expiration constraints.
Upon successful validation, the corresponding `User` object is fetched from the database and injected into the endpoint context. If any step in this pipeline fails—due to missing, expired, or tampered tokens—an `HTTP 401 Unauthorized` exception is raised automatically, terminating the request.

Frontend State Management

Client-side authentication state is maintained using a global React Context provider, which wraps the entire application hierarchy within the root `layout.tsx`. Upon mounting, the application initializes the context by reading from the browser's `localStorage` to hydrate the active session. The user payload (containing the `id`, `name`, and `role`) and the JWT token are stored and retrieved separately.
The primary navigation component (`Navbar`) subscribes to this context to dynamically render the login state and display the active user's name. Furthermore, all outbound authenticated API requests utilize a dedicated utility helper (`getToken`, located in `lib/auth.ts`) to securely extract and append the token to the `Authorization` header.

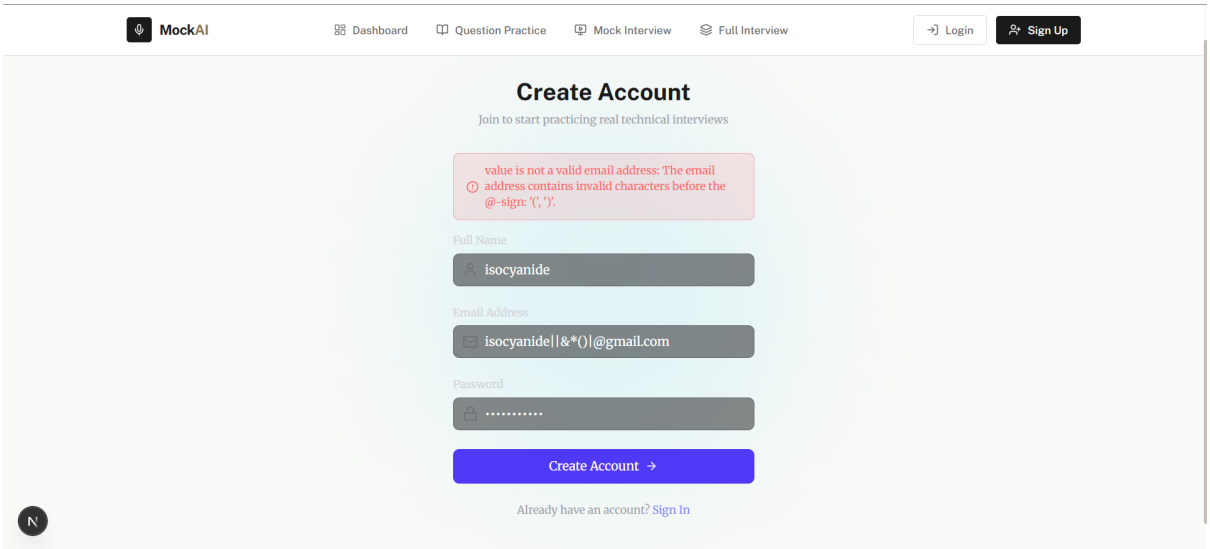

Figure 5.2: MockAI Registration Page showing email validation requirement.

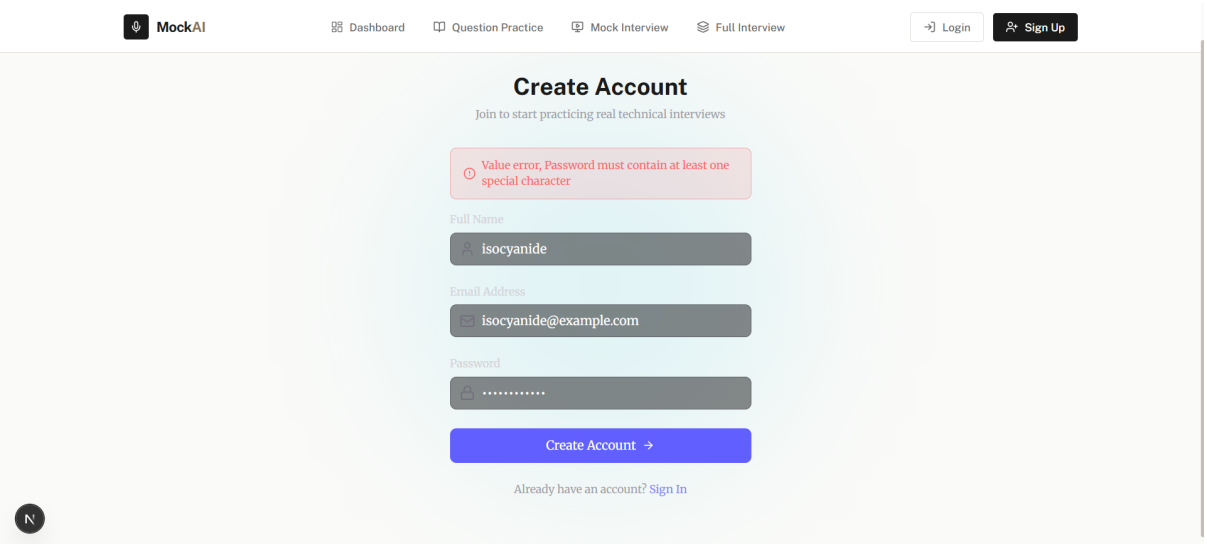

Figure 5.3: MockAI Registration Page showing password strength requirements.

5.3.1 JWT Token Lifecycle and Request Authentication

How JWT Works in MockAI

JSON Web Tokens (JWT) provide a compact, URL-safe mechanism for representing claims securely between two parties. A JWT comprises three Base64-encoded segments separated by dots: a header, a payload, and a signature.

The **header** declares the cryptographic algorithm (HS256). The **payload** carries the actionable claims or data. Finally, the **signature** is a cryptographic hash of the header

and payload generated using the server's secret key. This signature guarantees that the token cannot be tampered with in transit; any modification to the payload will immediately invalidate the signature.
Within MockAI, the JWT payload is strictly minimized to contain only two claims:

```
{
  "sub": "4c2f764a-df49-43e9-8fd3-5c674b4da76c",
  "exp": 1776673233
}
```

The `sub` (subject) field stores the user's UUID, while the `exp` (expiration) field stores a Unix timestamp representing the exact moment the token expires (set to 30 minutes from issuance). No extraneous data—such as passwords, roles, or names—are stored within the token.
Expiration Logic: The 30-minute window was chosen deliberately. It is short enough to severely limit the damage window if a token is intercepted, yet long enough that a standard platform session (a typical question practice run takes 15 to 30 minutes) can be completed without disruptive mid-session expirations. In Phase B of development, a refresh token mechanism with a 7-day validity will be introduced to handle silent token renewals.
HTTPBearer vs. Cookies: MockAI utilizes HTTP Bearer tokens transmitted via the `Authorization` header rather than HTTP-only cookies. During local development, the frontend (port `3000`) and backend (port `8000`) run on different origins. Cookies face strict Cross-Origin Resource Sharing (CORS) limitations in such environments. Bearer tokens circumvent these restrictions entirely, representing the industry standard for RESTful API authentication.

Authentication Flow Architecture

Figure 5.4 illustrates the complete lifecycle of a JWT during the login phase, followed by the middleware validation sequence required for accessing protected routes.

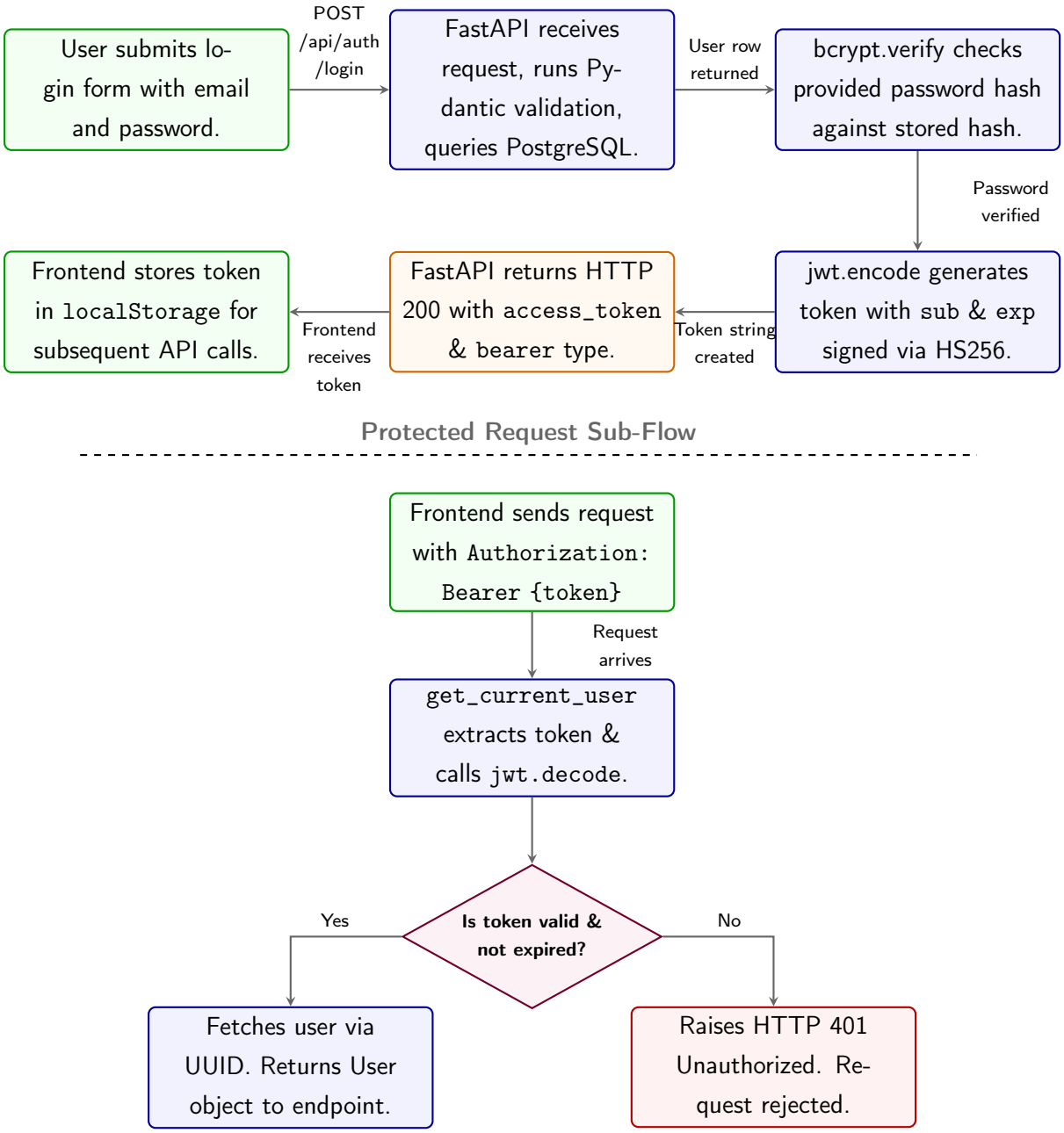

Figure 5.4: JWT Lifecycle Flow diagram detailing the initial token generation pipeline and the subsequent middleware validation layer for protected requests.

5.4 Question Generation Implementation

The core of the system's dynamic assessment engine resides in the `question_generator.py` service. This service interfaces with the external Large Language Model (LLM) API and utilizes a highly structured prompt engineering strategy to guarantee output consistency, relevance, and formatting. The prompt architecture is built upon five foundational pillars:

1. Role Assignment

The prompt is initialized with a strict system persona: *"You are a senior technical interviewer at a top-tier tech company (Google/Amazon/Microsoft level)."* Empirical research and iterative testing confirm that anchoring the LLM with a specific, authoritative persona drastically improves the specificity, tone, and technical accuracy of the generated content.

2. Type Instruction

To maintain the structural integrity of the assessments, the prompt dynamically injects strict typing rules:

- **MCQ Only:** Forces the generation of exclusively Multiple Choice Questions with exactly four options, explicitly prohibiting open-ended theory questions.
- **Theory Only:** Restricts generation to open-ended questions requiring written, analytical explanations.
- **Mixed Modality:** Specifies exact distribution ratios (e.g., generate N MCQs and M theory questions, strictly enforcing that $N + M$ equals the total requested volume).

3. Difficulty Calibration

The target difficulty is not passed as a mere keyword, but mapped to a descriptive pedagogical standard to calibrate the LLM's latent space:

- **Easy:** Focuses on fundamental concepts, definitions, and syntax suitable for beginners.
- **Medium:** Demands the application of concepts, comparative analysis, and practical scenario evaluation.
- **Hard:** Explores advanced edge cases, complex system design scenarios, optimization trade-offs, and tricky theoretical distinctions.

4. Subtopic Injection

To guarantee that generated questions map perfectly to the taxonomy (as outlined in Section 4.14), the active curriculum is injected directly into the prompt. The full list of applicable subtopics is embedded as a JSON string containing `id` and `name` fields. The LLM is explicitly instructed to assign a valid `subtopic_id` and `subtopic_name` from this list to every generated question.

5. Output Format Enforcement

To facilitate automated parsing, the prompt provides a concrete JSON array example demonstrating the exact expected schema. It concludes with an absolute directive: *"Return ONLY a valid JSON array. Zero explanation. Zero markdown. Zero code blocks."* Despite these strict constraints, external APIs (such as Gemini) occasionally hallucinate markdown formatting (e.g., wrapping the response in ```json ... ```). To ensure system resilience, the service passes all raw responses through a regex sanitization pipeline to strip markdown artifacts before parsing the JSON into Pydantic models.

Code Snippet: LLM Prompt Construction

```
prompt = f"""You are a senior technical interviewer at a top-tier
   tech company (Google/Amazon/Microsoft level).

Your task: Generate exactly {num_questions} {difficulty}-level
   interview questions.

Subject: {subject_name}
Specific Topics to Cover: {subtopic_context}

Question Type: {type_instruction}

Difficulty Standard ({difficulty}): {difficulty_guide}

STRICT RULES:
1. Distribute questions across ALL listed topics - do not focus on
    just one topic
2. Each question must test a DIFFERENT concept - no repetition
3. MCQ options must all be plausible - no obviously wrong answers
4. Theory questions must require conceptual understanding, not yes
   /no answers
5. Questions must match the difficulty level strictly
6. For aptitude/guesstimation questions: make them practical and
   realistic
7. For case study questions: provide enough context for the
   candidate to analyze
8. {format_note}
9. Assign the most relevant subtopic_id and subtopic_name from the
    list to each question

Available subtopics (use these exact ids and names):
```

```
{json.dumps(subtopics, indent=2)}

Return ONLY a valid JSON array. Zero explanation, zero markdown,
   zero code blocks.
Use this EXACT format:

[
  {{
    "question": "What is the time complexity of binary search and
       why?",
    "type": "mcq",
    "options": ["O(n)", "O(log n)", "O(n log n)", "O(1)"],
    "correct_answer": "O(log n)",
    "explanation": "Binary search halves the search space each
       step, giving O(log n).",
    "subtopic_id": "dsa_searching",
    "subtopic_name": "Searching Algorithms"
  }},
  {{
    "question": "Explain the difference between a process and a
       thread with a real-world analogy.",
    "type": "theory",
    "options": null,
    "correct_answer": "A process is an independent program with
       its own memory space. A thread is a lightweight execution
       unit within a process that shares memory. Analogy: A
       process is a restaurant, threads are the waiters working
       inside it.",
    "explanation": "Key distinction is memory isolation vs sharing
       .",
    "subtopic_id": "os_threads",
    "subtopic_name": "Threads & Concurrency"
  }}
]"""
```

Figure 5.5: Python implementation of the prompt engineering strategy within `question_generator.py`, detailing the dynamic assembly of role, difficulty, taxonomy, and schema constraints.

5.5 Quiz Interface Implementation

The dynamic quiz interface represents the most complex frontend architecture within the MockAI system, comprising approximately 600 lines of robust TypeScript code. It integrates real-time state management, complex timer synchronization, and responsive UI feedback to replicate a production-grade testing environment.

Dual Timer Synchronization

The interface implements two concurrent timing mechanisms to track both global session progress and granular per-question engagement.

Global Session Timer: The total session countdown is managed via a React `useEffect` hook that initializes an interval, incrementing a `totalSeconds` state variable every second. The displayed time is derived dynamically by subtracting `totalSeconds` from the user-configured `timeLimitSeconds` (e.g., a 30-minute limit equates to 1800 seconds). To enhance user awareness, the UI incorporates visual feedback: when the remaining time falls below the 300-second (5-minute) threshold, the timer text and corresponding icon transition to an alert red state. If the remaining time reaches zero, a `timeExpired` state flag is explicitly set. A secondary `useEffect` hook monitoring this flag intercepts the event and programmatically triggers the `handleSubmit` function, forcing an auto-submission.

Per-Question Granular Timer: Tracking time spent on individual questions requires a different architectural approach to avoid severe performance degradation. Instead of React state, the application utilizes a `useRef` object (named `questionTimeMap`) to maintain a mutable dictionary mapping question IDs to accumulated time. Because `useRef` mutations do not trigger component re-renders, the timer increments silently. When a user navigates to a new question, the component hydrates the local `questionSeconds` state using the historical value stored in the `questionTimeMap` for that specific ID. Conversely, when the user navigates away, the active `questionSeconds` value is written back to the dictionary, ensuring accurate, cumulative tracking regardless of how many times a user revisits a specific question.

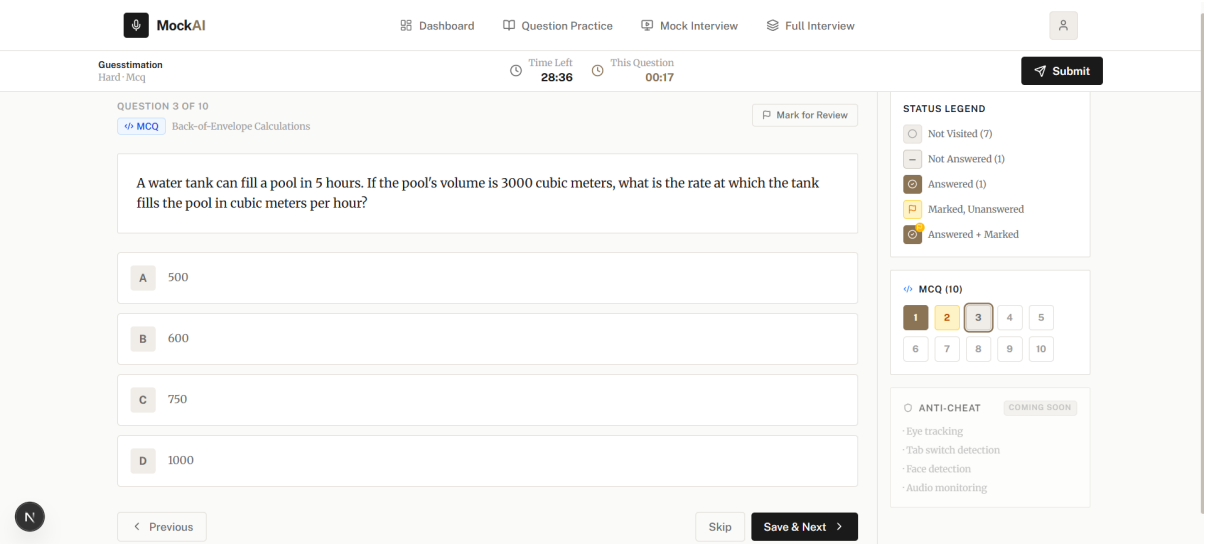

Figure 5.6: MockAI Quiz Interface showing global countdown timer, per-question timer, MCQ rendering, split MCQ/Theory sidebar, and Mark for Review functionality.

Answer State Management

Client-side answers are maintained within a centralized React state dictionary, binding each unique question ID to the user's submitted string payload. For Multiple Choice Questions (MCQs), this payload is the explicit text of the selected option. For theoretical questions, it captures the raw string output of the text area. This data structure directly drives the status state machine (as detailed in Chapter 4), ensuring that the sidebar navigation grid accurately reflects the current status (*Answered, Unanswered, Marked for Review*) of every question in real-time.

Submission and Evaluation Flow

Upon clicking the *Submit* button, the user is presented with a confirmation modal summarizing the total count of answered versus unanswered questions. Upon confirmation, the client initiates the submission pipeline:

1. The frontend iterates through the answer dictionary, dispatching asynchronous calls to the `/answers/submit` backend endpoint for every question containing a valid response payload.

2. Following successful answer ingestion, a final payload is dispatched to the session termination endpoint.

3. Throughout this network transaction sequence, the user interface is locked behind a full-screen evaluating overlay (featuring a loading spinner) to prevent further

interaction.

4. Once the backend confirms session closure and evaluation completion, the React Router programmatically redirects the user to the dynamic results dashboard.

5.6 Answer Evaluation Implementation

The evaluation engine is responsible for accurately grading user submissions across both discrete (Multiple Choice) and open-ended (Theory) question formats. To handle the semantic variance of human input, the system combines algorithmic string processing with Large Language Model (LLM) analysis.

Multiple Choice (MCQ) Evaluation Strategy

Initial implementations relied on exact string matching to grade MCQs, which proved brittle against semantically identical answers containing minor phrasing variances.
The current architecture employs a robust fuzzy matching algorithm: both the user's submitted answer and the expected correct answer are standardized (lowercased and stripped of trailing/leading whitespace) before a bidirectional containment check is performed. This ensures that a user submission of *"BFS"* successfully matches an expected answer of *"BFS (Breadth-First Search)"*, and *"Adjacency List"* correctly matches *"adjacency list representation"*.
Following the algorithmic boolean check, the evaluation service initiates an asynchronous LLM call. The prompt requests a concise, targeted explanation of exactly why the submitted answer is correct or incorrect. This dynamic explanation is subsequently rendered on the frontend results dashboard under a "Why This Is Correct" panel, providing immediate educational feedback.

Theory Evaluation Prompting

For open-ended theoretical questions, grading is entirely delegated to the LLM. The evaluation prompt dynamically injects the following context parameters:

- The original question text.
- The expected model answer (generated concurrently with the question and cached in the database).
- The user's actual raw text submission.
- A strict grading rubric detailing the qualitative requirements for each score range (0–10).

The LLM is instructed to evaluate the user's response against the expected answer and return a structured JSON object containing a numeric `score` and a detailed `feedback` string. As with question generation, the prompt concludes with strict schema enforcement: *"Return only valid JSON, no markdown."*

Resilient Fallback Chain

To guarantee system stability and handle external API rate limits, every LLM invocation within both the question generator and answer evaluator is wrapped in a comprehensive `try-except` block implementing a multi-tier fallback chain:

1. **Primary Tier:** The system attempts the evaluation using the primary **Gemini API**.
2. **Secondary Tier:** If the Gemini call raises any exception (including HTTP 429 Too Many Requests due to quota exhaustion), the identical prompt payload is seamlessly rerouted to the **Groq API** as a secondary provider.
3. **Degradation Tier:** If both external providers fail or timeout, the system executes a graceful degradation protocol. For MCQs, the score is computed solely from the local algorithmic string match, and a default generic feedback string is utilized. For Theory questions, a score of 0 is assigned, accompanied by a system feedback message indicating that the AI evaluation service is temporarily unavailable.

5.7 Skill Tracking Engine

The Skill Tracking Engine serves as the analytical core of the platform, responsible for dynamically maintaining and updating a user's competency profile across various subjects based on their session performance. This service exposes two primary functions: `get_skill_vector` and `update_skill_vector`.

Skill Initialization and Retrieval

The `get_skill_vector` function is invoked to retrieve a user's current competency state. It queries the `skill_vectors` database table using the active user's ID. If a row does not exist for a newly registered user, the service automatically provisions a new skill vector row, initializing all subject scores to a baseline of `0.0`. The function returns a structured dictionary mapping subject field identifiers (e.g., `dsa`, `os`, `dbms`) to their corresponding float values.

Exponential Moving Average (EMA) Update Mechanism

Following the conclusion of an assessment session, the `update_skill_vector` function is triggered. This function accepts the user ID, the target subject's `skill_field` string identifier, and the newly calculated session average score.
The service retrieves the user's historical score for that specific field and applies the Exponential Moving Average (EMA) algorithm (as defined in the system architecture, Section 4.5) to blend the historical baseline with the new performance data. To guarantee robust data integrity, a centralized `SUBJECT_FIELD_MAP` dictionary within the service provides strict, validated mapping between the curriculum taxonomy strings and the actual database column names, ensuring seamless synchronization even as new curriculum subjects are integrated over time.

5.8 Dashboard Implementation

The user dashboard acts as the primary analytical interface, designed to provide a comprehensive, real-time overview of a user's preparedness. To ensure a highly responsive user experience, the dashboard fetches all required analytical data via a single, optimized backend endpoint, reducing network latency and database round trips.

Backend Aggregation Query

The dashboard data aggregation endpoint executes a consolidated query that performs the following operations:

1. Retrieves all completed sessions for the authenticated user, ordered descending by timestamp.
2. Computes global platform averages and total questions answered.
3. Extracts the five most recent sessions to populate the *Recent Activity Log*.
4. Fetches the user's updated skill vector and programmatically classifies subjects into actionable tiers: "Weak" (scores strictly below 5.0) or "Strong" (scores ≥ 5.0).

Visualizations: The SkillRadar Component

Data visualization on the client is managed by the `SkillRadar` React component, which leverages the `Recharts` library to render a two-tiered graphical analysis.
Domain View: The default view renders a `RadarChart`. Each axis on the radar represents a high-level educational domain (e.g., Core CS, Placement Prep). The plotted data

points represent the mathematical average of all active subject scores nested within that domain, providing a holistic snapshot of overarching strengths and weaknesses.

Subject Drill-Down View: When a user clicks on a specific domain card, the interface transitions to a granular `BarChart`. This chart displays individual subject scores within the selected domain, utilizing semantic color coding for rapid cognitive processing: green indicating strong performance, amber indicating moderate proficiency, red highlighting weak areas requiring immediate revision, and grey denoting subjects not yet attempted.

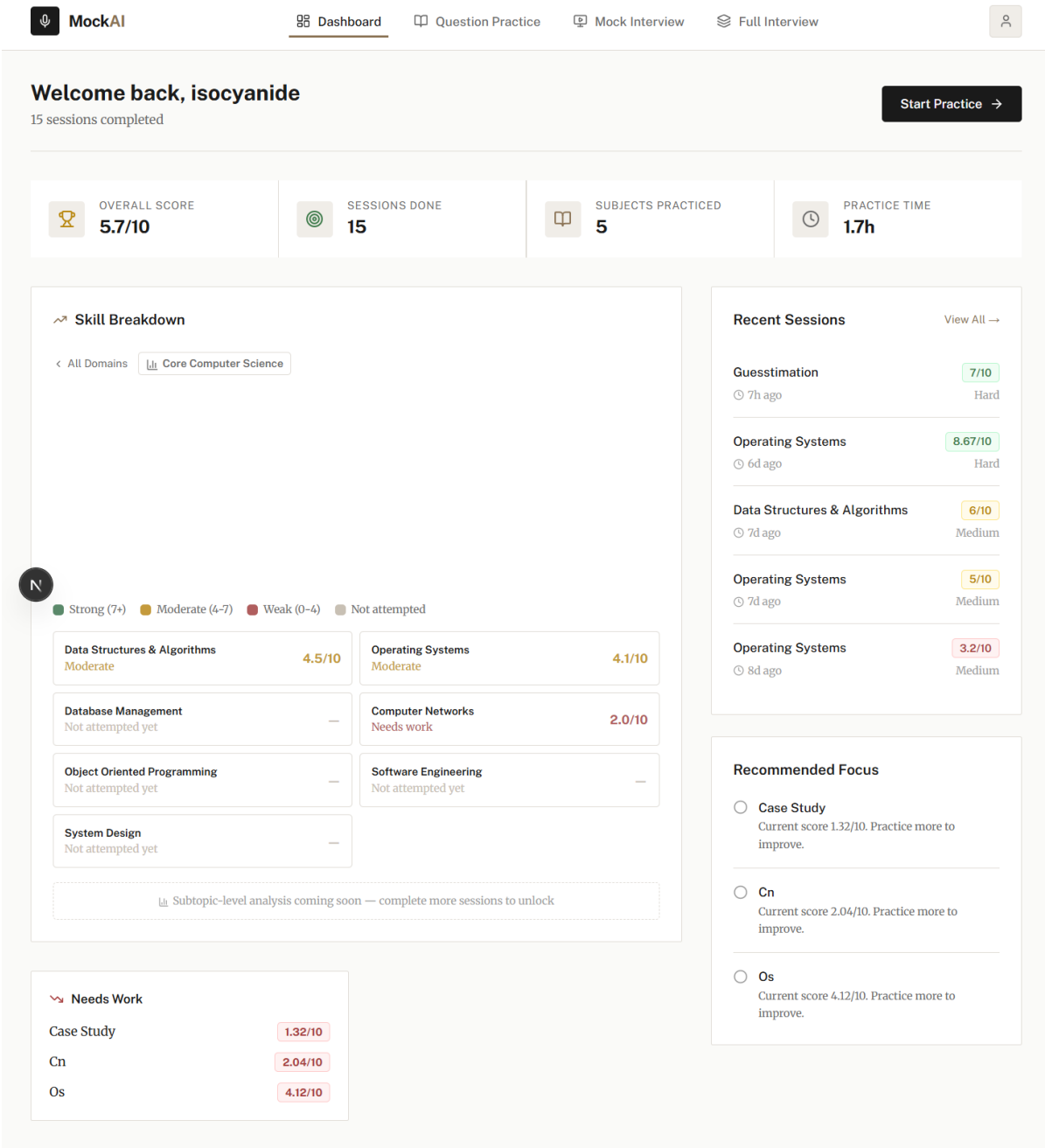

Figure 5.7: MockAI Dashboard displaying the domain radar chart, detailed skill breakdown by subject, recent sessions log, and dynamically recommended focus areas based on recent performance.

5.9 Export Implementation

To allow users to retain their performance data offline, the MockAI platform provides dual export capabilities (CSV and PDF) directly from the results dashboard. Both export mechanisms are implemented entirely on the client side, eliminating unnecessary backend processing overhead and reducing server bandwidth.

Client-Side CSV Generation

The CSV export pipeline is constructed via a custom JavaScript utility function that builds a two-dimensional array, where each inner array represents a discrete row.
The structure is logically partitioned: the initial rows contain the high-level session summary (Subject, Average Score, Overall Accuracy, and Total Time). A blank separator row is then inserted, followed by a detailed per-question breakdown. These detailed rows capture the question number, raw question text, modality (MCQ/Theory), the user's submitted answer, the expected correct answer, the computed score, correctness boolean, LLM feedback, and time taken.
To guarantee data integrity—especially since LLM feedback and user answers often contain commas and line breaks—the array is joined into a final CSV string with robust double-quote escaping. The browser's native `Blob` API is then utilized to package this string, and a temporary, invisible HTML anchor element (`<a>`) is dynamically injected and clicked via JavaScript to trigger the local file download.

Dynamic PDF Generation

For formal reporting, the system supports PDF generation utilizing the `jsPDF` library paired with the `jspdf-autotable` plugin. To optimize frontend performance and prevent bloating the initial JavaScript bundle size, these libraries are imported dynamically only when the user explicitly triggers the export.
Upon invocation, the function instantiates a new `jsPDF` document. It sequentially paints the document title and session metadata, before executing the `autoTable` module twice: first to render the summary statistics block, and second to generate the comprehensive per-question evaluation breakdown. Finally, a synthesized list of "Weak Areas" is appended to the bottom of the report. The resulting file is automatically saved to the user's machine, employing a dynamic filename that concatenates the subject name and a truncated session ID for easy referencing.

Session Report Card

Subject: Guesstimation

Difficulty: hard

Date: 01/05/2026

Summary

Metric	Value
Overall Score	7/10
Accuracy	70%
Correct Answers	7/10
Total Time	14m 1s

Question Breakdown

Q#	Type	Score	Correct?	Time	Subtopic
Q1	MCQ	0/10	'□	96s	Fermi Problems & Estimation Frameworks
Q2	MCQ	0/10	'□	114s	Market Sizing Problems
Q3	MCQ	10/10	'□	264s	Back-of-Envelope Calculations
Q4	MCQ	0/10	'□	66s	Assumption-based Reasoning
Q5	MCQ	10/10	'□	48s	Sanity Checking Estimates
Q6	MCQ	10/10	'□	102s	Market Sizing Problems
Q7	MCQ	10/10	'□	34s	Back-of-Envelope Calculations
Q8	MCQ	10/10	'□	34s	Fermi Problems & Estimation Frameworks
Q9	MCQ	10/10	'□	56s	Assumption-based Reasoning
Q10	MCQ	10/10	'□	27s	Sanity Checking Estimates

Areas to Improve

- Fermi Problems & Estimation Frameworks: 0/10
- Market Sizing Problems: 0/10
- Assumption-based Reasoning: 0/10

Figure 5.8: Sample exported PDF report card detailing the summary statistics table and the comprehensive per-question feedback breakdown.

5.10 Session Feedback Implementation

To continuously monitor the quality of the LLM-generated questions and the platform's user experience, a dedicated feedback module is integrated at the bottom of the results

dashboard.

State Hydration and API Integration

The feedback component lifecycle begins immediately upon mounting. It fires an asynchronous `GET` request to the backend feedback endpoint to verify if a review already exists for the active session.

- **Existing Feedback:** If the backend returns an existing record, the component parses the payload and pre-fills the UI state (star rating and toggles). The form transitions to a read-only "Submitted" state, rendering an *Edit* button to allow modifications.
- **No Existing Feedback:** If no record is found, the form initializes in a pristine, empty state, ready for user input.

Interactive UI State Management

The user interface relies on highly responsive React state hooks to manage input capture prior to submission:

- **Star Rating Mechanics:** The rating component utilizes two distinct states: a `hoverRating` state that tracks mouse movements to provide a real-time visual preview of the stars filling up, and a committed `rating` state that permanently records the value upon a click event.
- **Binary Metric Toggles:** The form asks two qualitative questions (*Was the difficulty appropriate?* and *Were the questions relevant?*). These are captured via binary Yes/No toggle buttons. Active selections are visually reinforced using distinct semantic styling: green indicating a positive (Yes) assertion and red indicating a negative (No) assertion.

Upon finalization, the client aggregates all four data points (star rating, difficulty boolean, relevance boolean, and an optional text comment) and dispatches them via a `POST` request to the backend, finalizing the session lifecycle.

5.11 Prompt Engineering Strategy and Prompt Structure

Why Prompt Engineering Matters

Large Language Models (LLMs) do not inherently possess context regarding specific platform curricula, localized difficulty standards, or strict output schema requirements.

Prompt engineering is the systematic process of encoding these programmatic constraints into the natural language prompt transmitted to the model. A poorly engineered prompt yields generic questions with inconsistent formatting, causing downstream parsing failures. Conversely, a highly structured prompt guarantees curriculum-aligned, difficulty-calibrated, and consistently formatted JSON output.
Within MockAI, two distinct prompt pipelines are utilized: the **Question Generation Prompt** and the **Answer Evaluation Prompt**. Each is dynamically constructed at runtime utilizing Python f-strings to inject context-specific session data.

Question Generation Prompt Structure

The generation prompt is algorithmically assembled from five distinct modular sections:

Section 1: Role Assignment

The prompt always initiates with a strict persona declaration:

> *"You are a senior technical interviewer at a top-tier tech company (Google/Amazon/Microsoft level)."*

This directive is paramount. LLM behavioral research consistently demonstrates that assigning a specific, authoritative role dramatically improves the specificity and quality of the generated output. The model shifts its latent reasoning pathways to emulate domain expertise, ensuring questions reflect actual industry interview depth rather than generic textbook trivia.

Section 2: Task Declaration

This section bounds the scope of the generation task:

> *"Your task: Generate exactly {num_questions} {difficulty}-level interview questions."*
> *"Subject: {subject_name}"*
> *"Specific Topics to Cover: {subtopic_context}"*

The `subtopic_context` variable injects a comma-separated string of subtopic names derived from the user's pre-session configuration (e.g., *"Dynamic Programming, Graphs (BFS, DFS, Representation)"*). This prevents the LLM from defaulting to the easiest subtopics and forces distribution across the requested curriculum.

Section 3: Question Type Instruction (Conditional)

Format constraints are injected dynamically based on the session configuration:

- **MCQ Only:** *"ALL questions must be Multiple Choice Questions (MCQ) with exactly 4 options. No theory questions. Every question must have options array with exactly 4 items."*
- **Theory Only:** *"ALL questions must be open-ended theory questions requiring written explanation. No MCQs. Every question must have options set to null."*
- **Mixed Modality:** *"Generate exactly {mcq_count} MCQ questions and {theory_count} theory/explanation questions. Mix them randomly. MCQ questions must have options array. Theory questions must have options as null."* (Note: `mcq_count` is calculated via integer division of the total questions, with the remainder assigned to `theory_count`).

Section 4: Difficulty Standard Calibration

To standardize difficulty scaling across subjects, targeted adjectives define the required cognitive load:

- **Easy:** *"fundamental concepts, definitions, basic examples — suitable for beginners"*
- **Medium:** *"application of concepts, comparison questions, practical scenarios"*
- **Hard:** *"advanced edge cases, complex scenarios, optimization, tricky distinctions"*

Section 5: Strict Rules Engine

The prompt concludes with an explicit, enumerated rule set to constrain model hallucination and enforce pedagogical quality:

1. Distribute questions across ALL listed topics — do not focus on just one topic.
2. Each question must test a DIFFERENT concept — no repetition.
3. MCQ options must all be plausible — no obviously wrong answers.
4. Theory questions must require conceptual understanding, not yes/no answers.
5. Questions must match the difficulty level strictly.
6. For aptitude/guesstimation questions: make them practical and realistic.
7. For case study questions: provide enough context for the candidate to analyze.
8. Assign the most relevant `subtopic_id` and `subtopic_name` from the available list to each question.

Section 6: Available Subtopics Injection

To guarantee that the LLM correctly categorizes the generated questions within the application's taxonomy, the complete list of relevant subtopics is injected directly into the prompt as a formatted JSON array:

```
[
  {
    "id": "dsa_dp",
    "name": "Dynamic Programming"
  },
  {
    "id": "dsa_graphs",
    "name": "Graphs (BFS, DFS, Representation)"
  }
]
```

The model is strictly instructed to tag every generated question using exclusively the `id` and `name` values provided in this array.

Section 7: Output Format Specification

The final layer of the prompt dictates the output structure with zero tolerance for deviation: *"Return ONLY a valid JSON array. Zero explanation, zero markdown, zero code blocks."*
Following this directive, a concrete, fully populated JSON example is provided demonstrating exactly what one expected MCQ object and one expected Theory object should look like (containing `question`, `type`, `options`, `correct_answer`, `explanation`, `subtopic_id`, and `subtopic_name`).
This technique—known as **few-shot prompting**—involves providing explicit examples of the target output format directly within the prompt context. Empirical testing confirms this is the most reliable methodology for coercing structured, parseable JSON payloads from non-deterministic language models.

Dynamic Assembly Visualization

Figure 5.9 illustrates the complete architectural flow of the prompt construction, visualizing how these modular text blocks are layered together at runtime to form the final payload sent to the LLM.

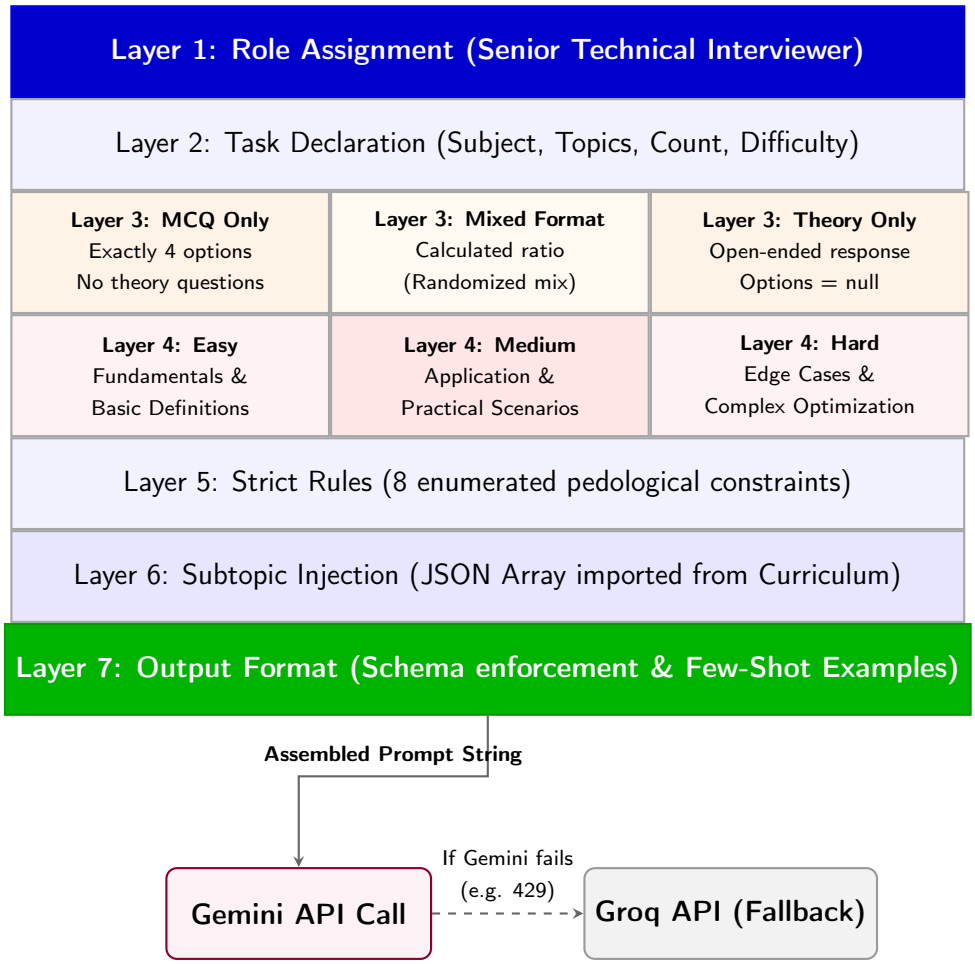

Figure 5.9: Prompt Construction Flow Diagram illustrating the vertical assembly of the context layers into a single prompt string, culminating in the resilient API routing sequence.

Answer Evaluation Prompt Structure

The evaluation pipeline utilizes two distinct prompt architectures depending on the modality of the question being graded.

For MCQ Questions (Explanation Generation)

Because multiple-choice correctness is resolved deterministically via the fuzzy string-matching algorithm prior to the LLM invocation, the prompt is purely generative rather than evaluative. The prompt context includes the original question text, the correct answer string, and a boolean indicating whether the user was correct.
The LLM is tasked exclusively with generating a concise educational explanation detailing *why* the correct option is the right choice. It is constrained to return a lightweight JSON schema containing:

- `explanation`: The technical reasoning supporting the correct answer.

- `feedback`: A brief, personalized comment (e.g., "Great job spotting the edge case!").
- `strengths`: An array of identified positive behaviors (e.g., ["Correctly identified time complexity"]).
- `weaknesses`: An array of identified negative behaviors (if applicable).

For Theory Questions (Rubric Evaluation)

For open-ended theoretical questions, the LLM acts as the primary grading engine. The prompt is substantially larger and structured as follows:

1. **Role Declaration:** *"You are a technical interview evaluator at a senior level. Be fair, specific, and constructive."*
2. **Question Context:** The original question text.
3. **Expected Answer:** The baseline model answer stored during the initial generation phase.
4. **Candidate Response:** The user's actual submitted text.
5. **Scoring Rubric:** An explicit mapping of numeric ranges to qualitative standards:
   - **0–2:** Completely wrong, irrelevant, or shows no understanding.
   - **3–4:** Partial understanding shown, major theoretical gaps remain.
   - **5–6:** Decent understanding, core concepts correct but lacks depth.
   - **7–8:** Good understanding, minor gaps or missing real-world examples.
   - **9–10:** Excellent, complete, and accurate with practical examples.
6. **Format Enforcement:** *"Return ONLY valid JSON, no markdown."* Followed by the expected schema requiring a `score` (integer 0–10), `is_correct` (boolean, mapped to true if score ≥ 5), `feedback`, an optimal `correct_answer`, `strengths[]`, and `weaknesses[]`.

Evaluation Decision Architecture

Figure 5.10 illustrates the divergent processing pipelines based on question modality, and how both streams ultimately converge to persist the evaluation record.

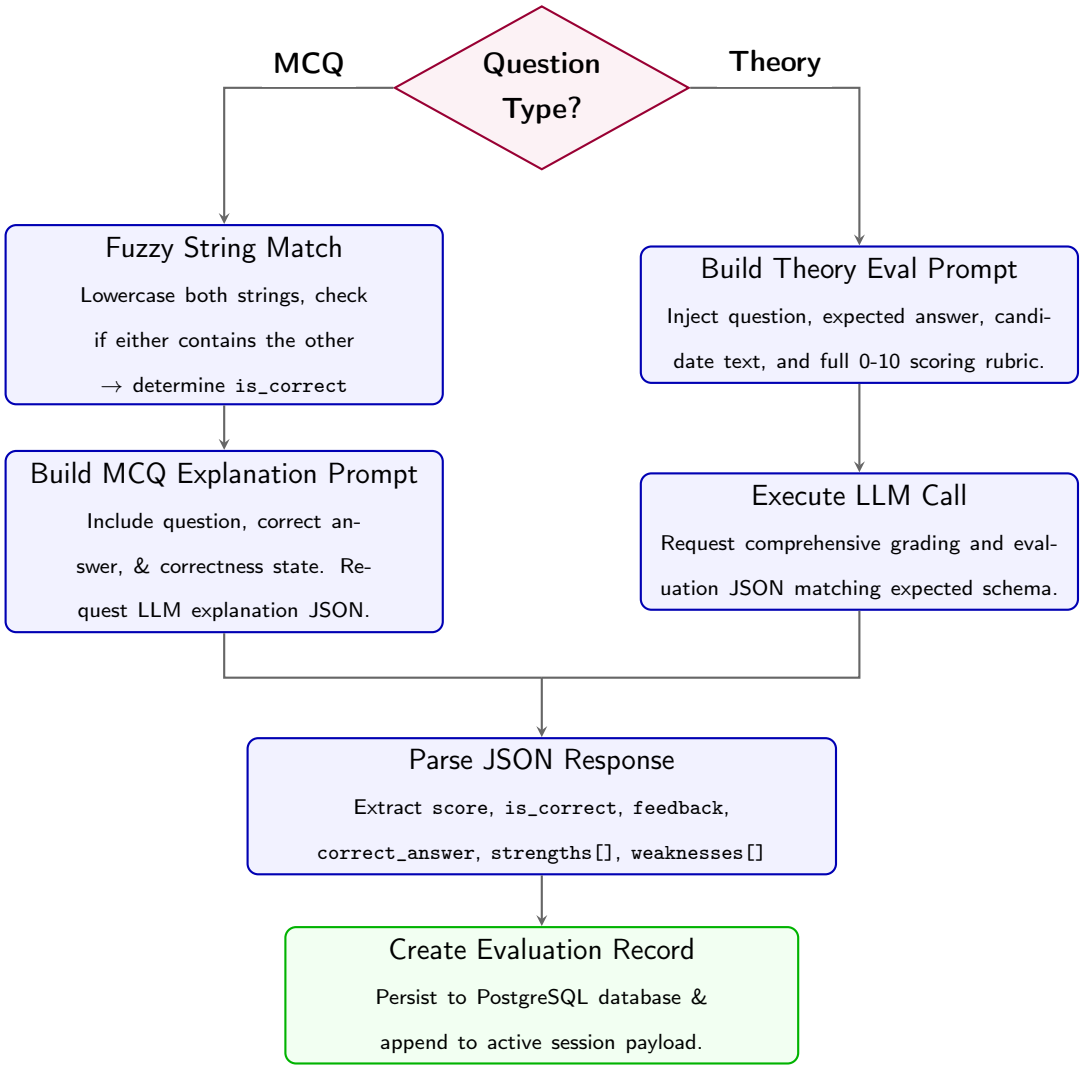

Figure 5.10: Evaluation Processing Flowchart demonstrating the divergence between deterministic MCQ string matching and generative Theory rubric grading.

5.12 Frontend-Backend Data Communication

How the Frontend and Backend Communicate

The architectural philosophy of MockAI mandates strict separation of concerns. The Next.js frontend functions as a pure presentation and state-management layer. It never directly queries the PostgreSQL database, nor does it directly invoke the external LLM APIs (Gemini/Groq). Every piece of data in the platform flows exclusively through the FastAPI REST API via HTTP requests and JSON responses.

This decoupling ensures that the frontend remains lightweight and stateless regarding business logic, while the backend maintains absolute authority over data validation, security, and external service orchestration. The communication pattern for every feature

follows an identical, predictable lifecycle:

1. A user action (such as a button click, form submission, or initial page load) triggers an asynchronous JavaScript function within a React component.
2. The function utilizes the native JavaScript `Fetch API` to transmit an HTTP request to the appropriate backend endpoint URL. This request includes the designated HTTP method (`GET`, `POST`, etc.), necessary headers (crucially, the `Authorization: Bearer {token}`), and a stringified JSON body payload if required.
3. The FastAPI server receives the request. The middleware intercepts it to validate the JWT token. Once authorized, the endpoint validates the incoming JSON body against its strict Pydantic schemas, executes the required business logic (database queries or LLM calls), and prepares a JSON response.
4. The frontend awaits the network response, parses the returned JSON payload, updates the localized React state (via `setState`), and subsequently triggers a component re-render to display the new data to the user.

Communication Architecture Visualization

Figure 5.11 maps the exact chronological execution stack across both ports during a standard API transaction, alongside specific routing examples for core platform features.

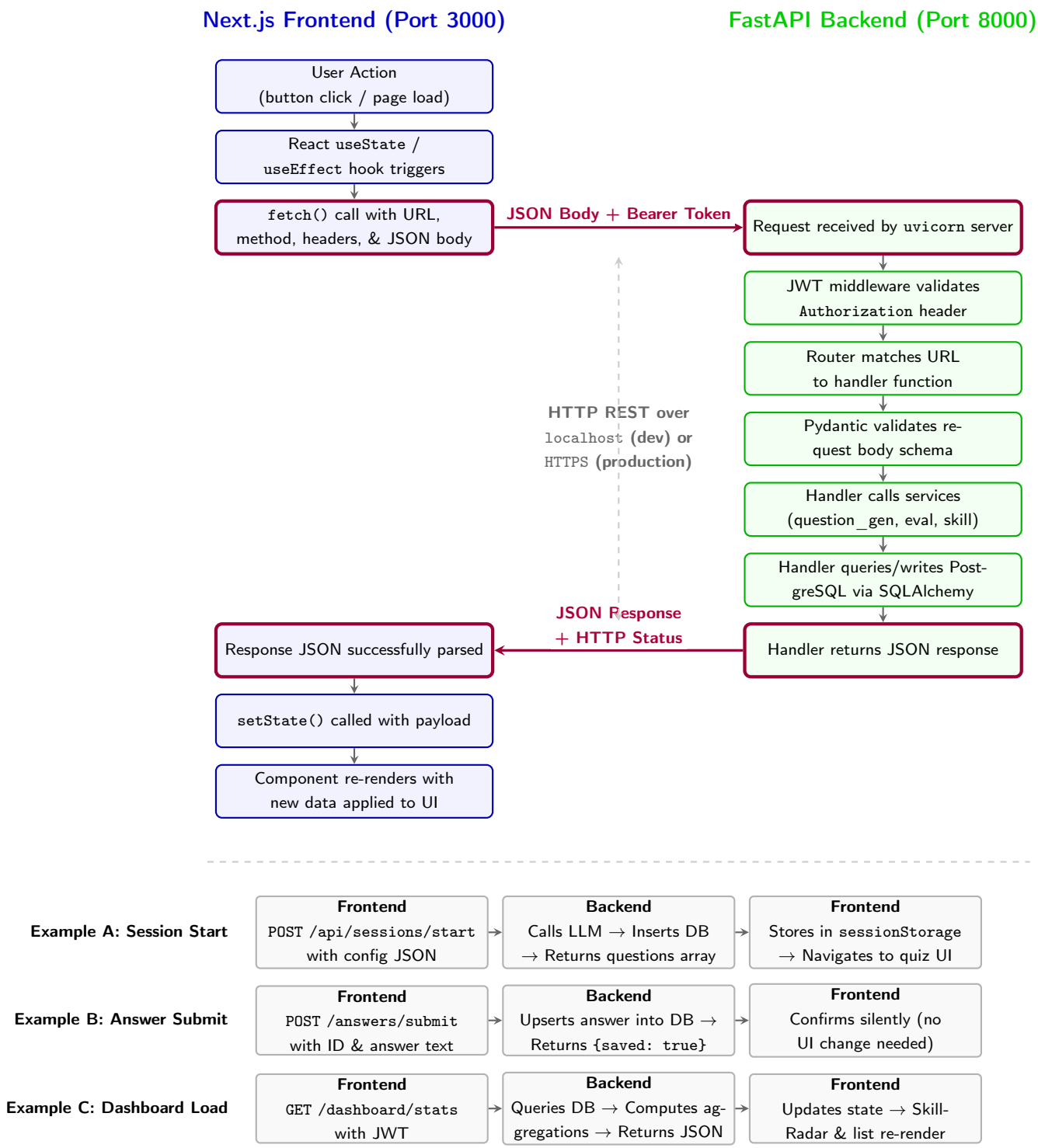

Figure 5.11: Frontend-Backend Communication Pattern. The U-shaped flow illustrates the stateless, RESTful decoupling of the presentation layer from the core business logic, accompanied by three specific feature lifecycle examples.

5.13 Question Lifecycle — Session Storage and Database Storage

Two Types of Storage Architecture

MockAI implements two distinct storage mechanisms for handling question data throughout the lifecycle of an assessment. Understanding the architectural boundary between temporary client-side state and permanent server-side persistence is critical for mapping the system's data flow.

Browser `sessionStorage` (Temporary — Frontend Only)

When a user clicks *"Begin Practice"* and the backend generates the assessment payload, the Next.js frontend immediately stores the complete JSON response within the browser's `sessionStorage` under the key `"currentSession"`. By design, `sessionStorage` is ephemeral; it persists strictly for the duration of the active browser tab and is automatically wiped when the tab is closed.
This stored object acts as a comprehensive localized cache containing the `session_id`, session configurations (e.g., `timeLimit`, `subject_name`, `difficulty`), and the full array of generated questions. Crucially, the `correct_answer` fields are included in this payload (necessary for offline checks) but remain strictly unrendered and hidden from the user interface.
Architectural Justification: When the React Router navigates the user from the `/question-practice` configuration page to the active `/quiz/[id]` testing interface, the quiz component requires immediate access to all question data to render the UI and initialize the timers. Hydrating the component from `sessionStorage` provides zero-latency access, intentionally bypassing a redundant network round-trip to the database for data the client already possesses. If the user navigates directly to the quiz URL via a bookmark (meaning `sessionStorage` is empty), the component seamlessly falls back to triggering a `GET /api/sessions/{id}` call to rebuild the state from the database. Once the session is submitted and the results page renders, the frontend explicitly calls `sessionStorage.removeItem("currentSession")` to enforce security and clean up the memory space.

PostgreSQL Database (Permanent — Backend)

Simultaneous to the temporary frontend caching, all generated question data is permanently written to the `questions` table in PostgreSQL. The system utilizes a `session_id` foreign key constraint to cryptographically link every generated question to its parent session.

Architectural Justification: The PostgreSQL database serves as the absolute, authoritative source of truth. Permanent storage is mandatory because the backend requires access to the original question text and expected `correct_answer` models to execute the LLM evaluation pipeline (Section 5.6) upon session termination. Furthermore, permanent storage enables the generation of historical results reports, allows users to review their past evaluations, and forms the foundational dataset for the platform's longitudinal skill-tracking analytics. In this architecture, `sessionStorage` is purely a localized performance optimization; the database guarantees persistence.

Storage Lifecycle Visualization

Figure 5.12 tracks the parallel flow of data across the ephemeral browser storage and the permanent backend database during a standard session lifecycle.

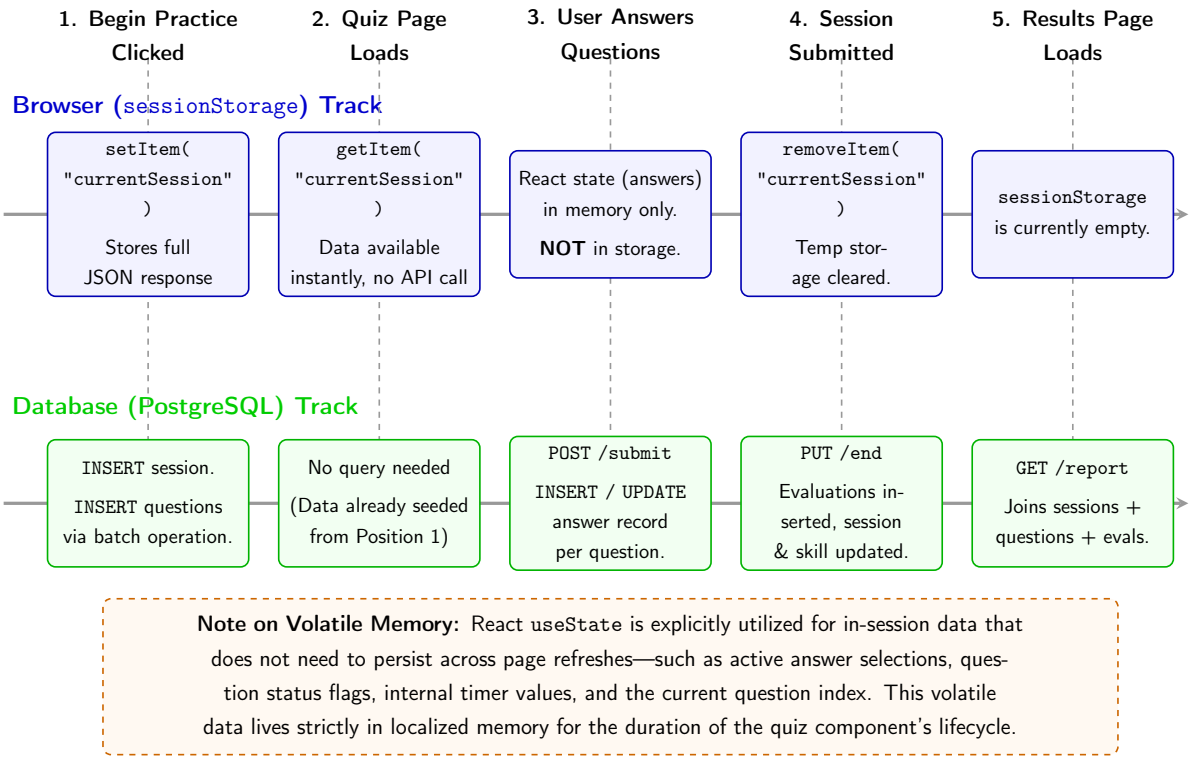

Figure 5.12: Storage Lifecycle Timeline illustrating the divergence and synchronization between the temporary browser cache and the permanent backend database across the five primary phases of an assessment.

5.14 Database Table Usage — Current and Planned

Which Tables Are Currently Active

Not all 11 tables within MockAI's database schema are actively written to in the current production flows. This is an intentional architectural decision—the schema has been forward-engineered to accommodate "Phase B" feature expansions, and the tables are provisioned now to prevent future database migration friction.

Currently Active (Written and read in every session)

- `users`: Written upon registration. Read on every authenticated request via the `get_current_user` dependency.
- `sessions`: Written upon session start (initialized with `status='active'`) and updated upon session end (transitioned to `status='completed'` with calculated scores). Read by the dashboard statistics endpoint and the session log.
- `questions`: Written in a bulk batch operation upon session start (one discrete row per generated question). Read on session initialization and heavily utilized during the end-session evaluation.
- `answers`: Written (via an `upsert` operation) every time a user submits an answer. Read extensively during the LLM evaluation phase in the `end_session` sequence.
- `evaluations`: Written dynamically during the `end_session` processing (one row per question). Read by the results page and the `get_report` aggregation endpoint.
- `skill_vectors`: Read and updated (utilizing the EMA formula) upon every successful session completion. Exists as a strict one-to-one mapping per user, updated in place.
- `session_feedback`: Written when a user submits a review on the results page. Read to proactively pre-fill the feedback form if the user revisits historical results.

Currently Structured but Not Yet Written To (Reserved)

- `subtopic_scores`: The table and complete schema exist, but `end_session` does not yet write to it. It is reserved for Phase B to enable subtopic-level skill tracking (e.g., diagnosing that a user is weak specifically in *Dynamic Programming* rather than just generically weak in *DSA*).
- `cheat_events`: The table exists to support a planned anti-cheat mechanism. Currently, the `cheat_flags` JSONB column on the `sessions` table remains unused. In

Phase B, tab-switching events will be recorded here via the browser's Page Visibility API.

- `conversation_logs`: Reserved for the multi-agent AI panel interview simulation in Phase B. Every turn in the spoken interview conversation will be persisted as a discrete row.
- `colleges`: The table exists, and the `User.college_id` foreign key is declared. Currently, no institutional records are instantiated. This will drive institutional analytics and cohort tracking in Phase B.

Table Activity and Relational Map

Figure 5.13 illustrates the active entity-relationship mapping alongside the reserved architecture for future development phases.

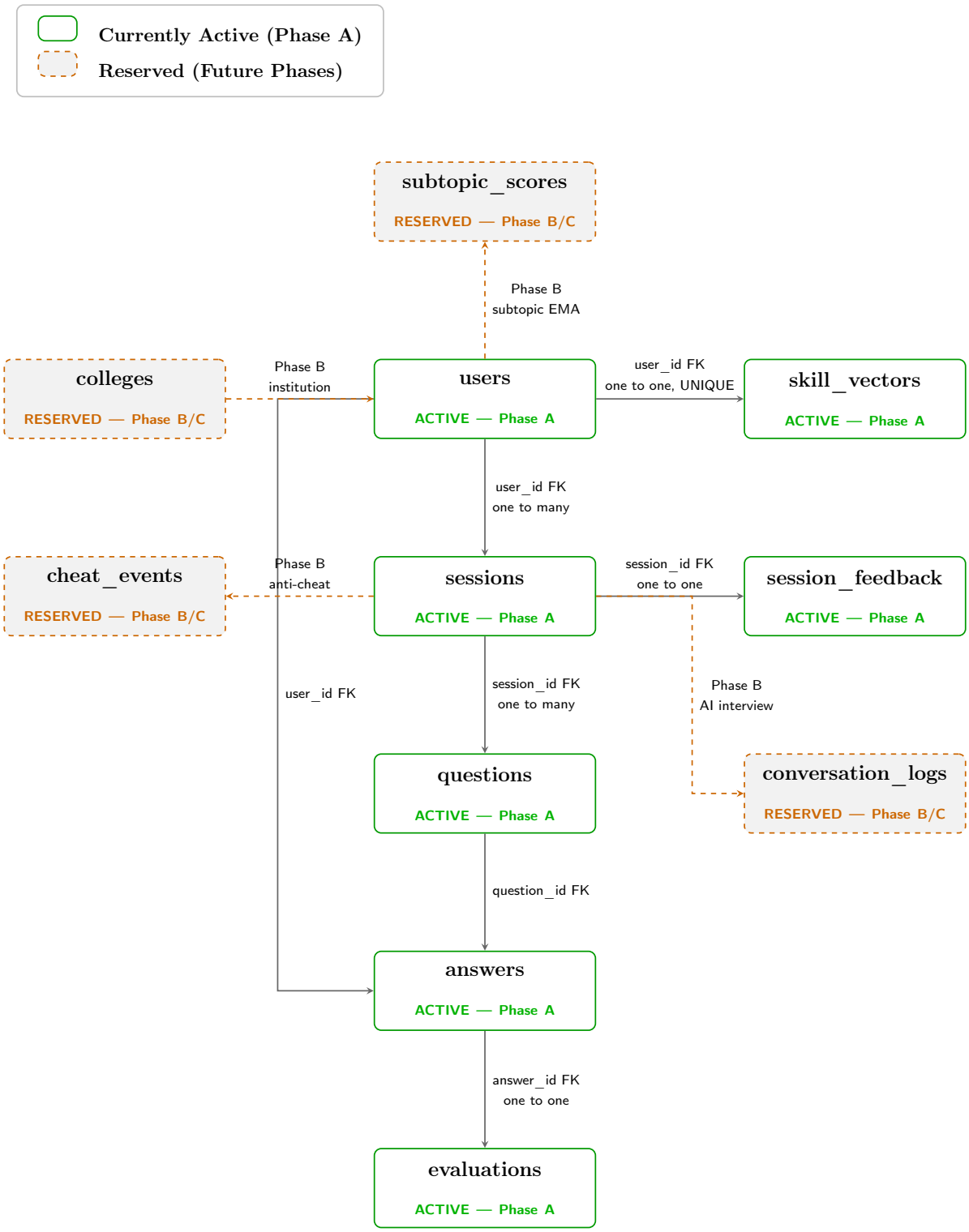

Figure 5.13: Database Table Interaction Map detailing the core Entity-Relationship flows for active components, alongside the architectural provisioning for Phase B and C expansions.

5.15 Score Computation — From Raw LLM Output to Skill Vector

How a Score Goes from LLM Response to Database

This section traces the complete end-to-end journey of a performance score—from the exact moment the Large Language Model (LLM) returns its evaluation payload to the final algorithmic update of the user's permanent `skill_vectors` table. Understanding this precise chain of operations is essential for validating the platform's data integrity and analytical correctness.

Step 1: LLM Returns Raw Evaluation Payload

For open-ended theoretical questions, the LLM processes the grading prompt and returns a structured JSON object conforming to the required schema:

```
{
  "score": 7,
  "is_correct": true,
  "feedback": "Good understanding of the concept. You correctly
     identified...",
  "correct_answer": "A complete answer would include...",
  "strengths": ["Correctly explained memoization", "Identified
     overlapping subproblems"],
  "weaknesses": ["Did not elaborate on space complexity", "Missing
      example"]
}
```

Note: For MCQ questions, correctness is pre-determined via fuzzy string matching. The assigned score is deterministically set to 10 (if correct) or 0 (if incorrect). The LLM is subsequently invoked purely to generate the pedagogical `explanation` and `feedback`.

Step 2: Evaluation Record Instantiation

The backend parses the JSON string. The extracted values are immediately utilized to construct an SQLAlchemy `Evaluation` model instance, which is appended to the active database session. The evaluation is cryptographically linked to the user's input via the `answer_id` foreign key.

Step 3: Session State Accumulation

As the backend iterates through the array of submitted answers, it maintains running performance tallies:

- `total_score` accumulates the numeric score of each processed question.
- `correct_count` increments by one if the `is_correct` boolean evaluates to true.
- `total_time` aggregates the `time_taken_seconds` from each discrete answer payload.

Step 4: Session Averages Computed

Following the termination of the processing loop, global session metrics are computed:

- Average Score $= \frac{\text{total_score}}{\text{number of questions}}$ (Yielding a float between 0.0 and 10.0).
- Accuracy $= \left(\frac{\text{correct_count}}{\text{number of questions}}\right) \times 100$.

Step 5: Dynamic EMA Update

The `update_skill_vector` service is invoked, passing three critical parameters: the `user_id`, the target `subject_skill_field` string (e.g., `"dsa"`), and the computed `session_score`.
The service fetches the user's existing `skill_vectors` row (creating one initialized to zeros if absent) and extracts the historical score. The Exponential Moving Average algorithm is then applied:

$$\text{new_value} = (0.3 \times \text{session_score}) + (0.7 \times \text{historical_score})$$

Using Python's `setattr()` method, the backend dynamically targets the specific database column matching the `skill_field` string and overwrites it with the new float value.

Step 6: Session Record Finalization

The parent `session` record is updated. Its status is flipped to `"completed"`. The `total_score`, `accuracy`, `total_time_seconds`, `weak_subtopics`, and `session_summary` columns are populated, and `ended_at` is stamped with the current UTC timestamp. The database transaction is committed.

Step 7: Client Serialization

Finally, the complete `SessionReport` Pydantic schema is serialized into a nested JSON object and transmitted back to the Next.js client via an HTTP 200 response, prompting

the frontend to redirect the user to the visual results dashboard.

Score Journey Visualization

Figure 5.14 maps this eleven-step algorithmic sequence across the four system boundaries.

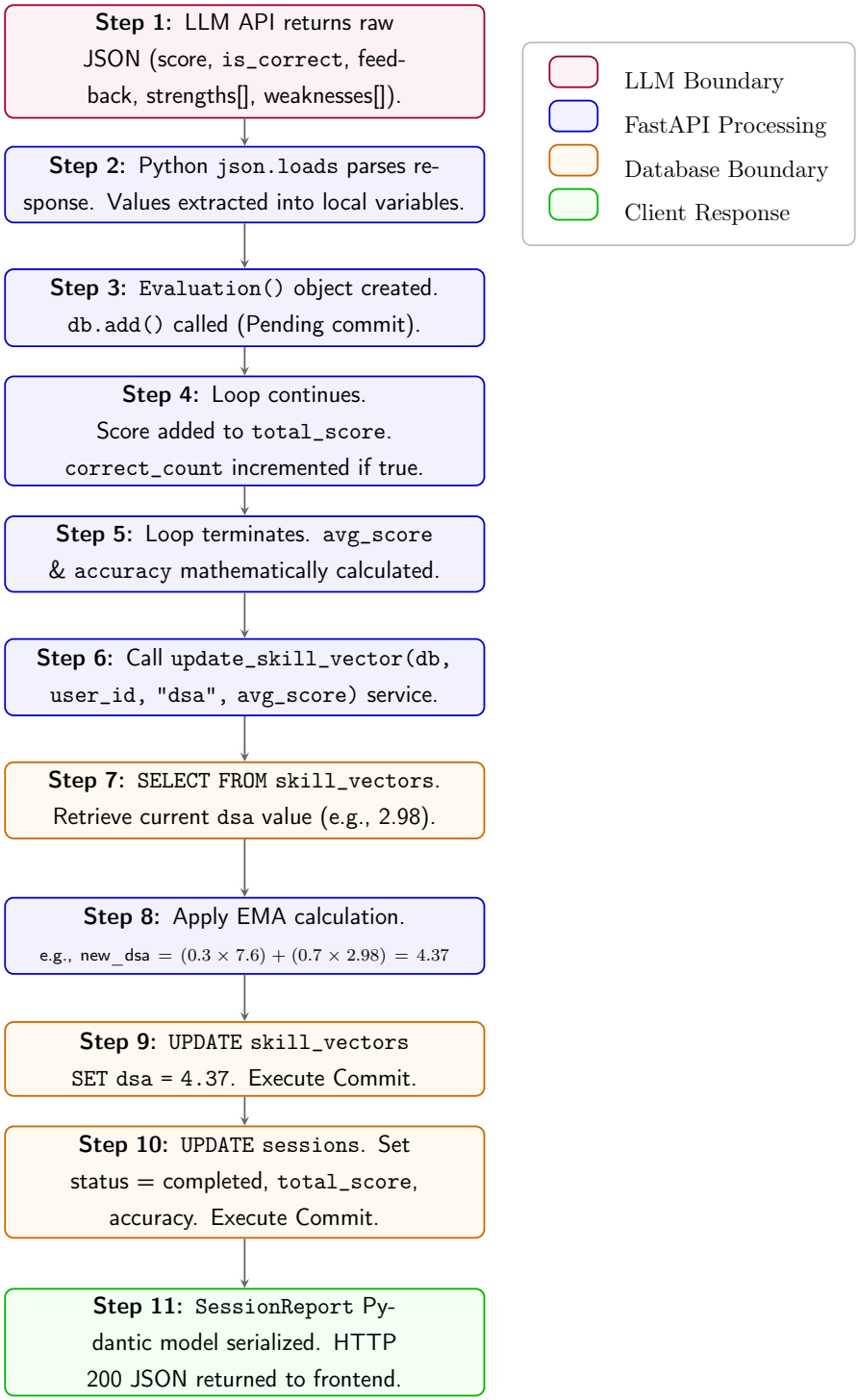

Figure 5.14: Score Journey Flowchart detailing the sequential algorithmic progression from raw LLM output to permanent database persistence and frontend hydration.

Chapter 6

Testing

6.1 Testing Approach

Given the research and development nature of this project, comprehensive testing was conducted across three distinct architectural layers to ensure system reliability and data integrity.
First, unit-level validation of individual API endpoints was performed utilizing the FastAPI automatic Swagger documentation interface (accessible at `/docs`). Second, integration testing was executed to validate complete end-to-end user flows—spanning from initial registration, through quiz initialization and completion, to the final results rendering. Third, rigorous edge case testing was conducted to verify system resilience against authentication failures, Large Language Model (LLM) quota exhaustion, and boundary conditions within the frontend quiz interface.

6.2 Authentication Test Results

To validate the security and robustness of the JWT-based authentication pipeline, a suite of positive and negative test cases was executed against the user management endpoints. All targeted test cases passed successfully.

Test Case	**Input**	**Expected Output**	**Actual Output**	**Result**
Register with valid email and strong password	Valid credentials	HTTP 201 with User Object	HTTP 201 with User Object	**Pass**
Register with duplicate email	Existing email	HTTP 409 Conflict	HTTP 409 Conflict	**Pass**
Register with weak password	`abc123`	HTTP 422 with constraint message	HTTP 422 with constraint message	**Pass**

Continued on next page...

Table 6.1 – Continued from previous page

Test Case	**Input**	**Expected Output**	**Actual Output**	**Result**
Register with invalid email format	`user@.com`	HTTP 422 with validation error	HTTP 422 with validation error	**Pass**
Login with correct credentials	Valid credentials	HTTP 200 with JWT Token	HTTP 200 with JWT Token	**Pass**
Login with wrong password	Incorrect password	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**Pass**
Access protected endpoint with valid JWT	Valid Bearer Token	HTTP 200 with Data	HTTP 200 with Data	**Pass**
Access protected endpoint with tampered JWT	Modified Token	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**Pass**
Access protected endpoint with expired JWT	Expired Token	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**Pass**

Table 6.1: Authentication and Authorization Test Results verifying the JWT middleware and Pydantic schema validation.

6.3 Question Generation Test Results

The question generation pipeline was tested across multiple subjects, difficulty tiers, and question modalities to ensure the prompt engineering strategy consistently yielded parseable and accurate JSON schemas.

- **Complex Parameter Generation:** Requested Data Structures and Algorithms (DSA) questions targeting Dynamic Programming and Graphs subtopics, medium difficulty, mixed modality, totaling 5 questions.
  Outcome: The LLM successfully generated exactly 5 valid questions with correct subtopic tagging and plausible MCQ options. **Result: Pass.**

- **Theory-Only Generation:** Requested Operating Systems (OS) questions targeting Scheduling and Deadlocks, hard difficulty, theory-only modality, totaling 5 questions.
  Outcome: The LLM generated exactly 5 open-ended theory questions featuring highly accurate, advanced conceptual content. **Result: Pass.**

- **System Resilience and Fallback:** Forced an LLM quota exhaustion scenario by intentionally configuring an exhausted Gemini API key.
  Outcome: The secondary Groq API fallback chain triggered transparently. Questions were generated successfully without presenting an error state to the user interface. **Result: Pass.**

6.4 Evaluation Test Results

The dual-engine evaluation system (incorporating both algorithmic fuzzy matching and generative LLM grading) was tested against various user inputs to verify grading fairness and accuracy.

- **MCQ Exact Match:** Submitted the exact correct answer string.
  Outcome: Score 10, `is_correct` = true. **Result: Pass.**

- **MCQ Fuzzy Match:** Submitted correct answer with different semantic phrasing (e.g., submitting "BFS" when the expected string was "BFS (Breadth-First Search)").
  Outcome: Handled successfully by the string stripping and containment logic. Score 10, `is_correct` = true. **Result: Pass.**

- **MCQ Incorrect Answer:** Submitted a definitively wrong option.
  Outcome: Score 0, `is_correct` = false, and a pedagogical explanation was successfully generated by the LLM. **Result: Pass.**

- **Theory - Full Understanding:** Submitted a comprehensive written explanation covering all required concepts.
  Outcome: Scored dynamically in the 8 to 10 range based on the grading rubric. **Result: Pass.**

- **Theory - Partial Understanding:** Submitted a brief answer missing core examples or edge cases.
  Outcome: Scored appropriately in the 4 to 6 range. **Result: Pass.**

- **Theory - Blank Submission:** Submitted an empty string.
  Outcome: Handled gracefully. Score 0 assigned without crashing the evaluation schema. **Result: Pass.**

6.5 End-to-End User Journey Test

To validate the holistic integration of the platform, a complete user flow was executed and verified from account creation through to post-session analytics.

Execution Sequence:

1. **Authentication:** Registered a new user account, logged in, and successfully received the JWT token.
2. **Configuration:** Navigated to the Question Practice interface. Selected the Core CS domain, DSA subject, Dynamic Programming and Graphs subtopics, set to medium difficulty, mixed type, 5 questions, with a 30-minute time limit.
3. **Generation:** Clicked *Begin Practice*. The loading overlay rendered correctly, and the backend generated the questions in approximately 8 seconds.
4. **Quiz Execution:** The quiz loaded seamlessly displaying 2 MCQ and 3 Theory questions divided into separate sidebar sections. All 5 questions were answered with varying degrees of accuracy.
5. **State Management Verification:** Marked one question for review, navigated away, and then answered it. Confirmed the `answered_marked_review` status flag visually updated the sidebar grid correctly.
6. **Submission:** Clicked *Submit*. The confirmation modal accurately displayed the answered versus unanswered counts. Upon confirmation, the full-screen evaluating overlay engaged for approximately 20 seconds.
7. **Results Review:** The results dashboard successfully populated, displaying an overall score of 6.6/10, an 80% accuracy rate, and 4/5 correct answers. The per-question breakdown successfully mapped the AI-generated feedback, strengths, and weaknesses to the correct UI components.
8. **Export & Feedback:** Downloaded the PDF report, verifying that jsPDF accurately formatted the summary table and question breakdown. Submitted a 4-star session rating via the feedback widget.
9. **Dashboard Updates:** Navigated back to the primary Dashboard. The DSA skill score updated accurately via the EMA algorithm, the completed session was visible in the recent activity log, and the Recharts skill radar visually reflected the new DSA domain score.

Conclusion: All steps in the primary architectural workflow completed successfully without latency timeouts, rendering anomalies, or database synchronization failures.

Chapter 7

Results and Discussion

7.1 System Performance

To evaluate the operational efficiency of the MockAI platform, latency metrics were recorded across the primary system workflows. Given the reliance on external Large Language Models (LLMs), the majority of synchronous processing time is consumed by API network calls rather than internal algorithmic bottlenecks.
Table 7.1 outlines the average execution times for core system operations observed during standard testing environments.

Operation	**Average Time**	**Notes**
Question generation (5 questions)	6 to 12 seconds	Gemini API response time varies significantly based on Google server load.
MCQ evaluation with AI explanation	2 to 4 seconds	Deterministic evaluation is instant; time is consumed by LLM explanation generation.
Theory evaluation per question	3 to 8 seconds	Groq API consistently executes faster than Gemini for evaluation tasks.
Dashboard data load	$<$ 300 milliseconds	Highly optimized via a single aggregated PostgreSQL query.
Frontend page load	$<$ 1.5 seconds	Benefiting from Next.js static optimization and dynamic code splitting.
JWT verification	$<$ 5 milliseconds	Handled entirely via rapid in-memory computation.

Continued on next page...

Table 7.1 – Continued from previous page

Operation	**Average Time**	**Notes**
PDF generation	< 1 second	Executed smoothly on the client-side utilizing the `jsPDF` library.
CSV generation	< 100 milliseconds	Achieved via pure JavaScript string construction.

Table 7.1: System Performance Metrics detailing execution times across internal and external network boundaries.

7.2 Question Quality Assessment

A qualitative analysis of questions generated across 15 distinct test sessions across varied subjects revealed the following pedagogical observations:

- **Data Structures and Algorithms (DSA):** The generation engine successfully adhered to subtopic constraints without repetition. Core concepts—including binary search time complexity, graph traversal selection, DP recurrence formulation, and time-space tradeoffs—appeared consistently with appropriate difficulty calibration.
- **Core Computing Subjects (OS/DBMS):** Theory questions demonstrated strong conceptual grounding. Topics such as process versus thread paradigms, ACID properties, deadlock conditions, and database normalization were framed to require genuine theoretical understanding rather than rote keyword recall.
- **MCQ Distractor Quality:** Plausible distractors (incorrect options) were successfully generated in approximately 85% of cases. In roughly 15% of generated questions, the LLM defaulted to an option that was excessively obvious or conceptually detached, slightly diluting the educational value of the question. This identifies a key area for prompt refinement in Phase B.
- **Aptitude and Domain Skills:** Guesstimation questions exhibited excellent variety and grounded, realistic estimation contexts. Case Study questions successfully provided adequate analytical context for the candidate to process.

7.3 Evaluation Quality Assessment

A subjective analysis of the theoretical evaluation engine was conducted by passing 25 sample answers through the pipeline and comparing human-expected score ranges against the actual algorithmic system scores.

- **Complete & Correct Explanation (Expected: 8-10):** System scored 7 to 9. *(Highly accurate)*
- **Correct but Brief Explanation (Expected: 5-7):** System scored 5 to 7. *(Accurate)*
- **Partial Understanding (Expected: 3-6):** System scored 3 to 5. *(Accurate)*
- **Wrong or Irrelevant Answer (Expected: 0-2):** System scored 0 to 2. *(Accurate)*
- **Empty/Blank Submission (Expected: 0):** System scored 0 exactly. *(Perfectly accurate)*

Identified Bias: One notable observation arose during testing: the system occasionally under-scored answers that were highly concise but technically complete. LLMs inherently possess a documented "verbosity bias"—a tendency to inadvertently reward longer, more verbose responses even when the core academic content is identical to a shorter submission. This evaluation bias can be mitigated in future iterations by explicitly instructing the grading rubric to prioritize density and correctness over length.

7.4 Skill Vector Evolution

To validate the Exponential Moving Average (EMA) mathematical model tracking user proficiency, the progressive tracking of a sample student's DSA skill was simulated across five sessions.

Given raw session scores of **6.6, 5.3, 7.6, 4.4, and 8.0**, the algorithm accurately reflected gradual knowledge acquisition while smoothing out anomalous dips. Starting from a baseline of 0.0, the skill vector evolved sequentially to **1.98** $\rightarrow$ **2.98** $\rightarrow$ **4.37** $\rightarrow$ **4.38** $\rightarrow$ **5.47**.

The $\alpha = 0.3$ weighting factor ensures that a student requires approximately 8 to 10 completed sessions within a specific subject to elevate their EMA score from a "cold start" up to their true, current performance baseline. This rate of evolution is highly appropriate for an educational preparation platform where users are expected to complete 10 to 20 practice sessions per targeted subject over the course of an academic semester.

7.5 Feedback Analysis

Post-session qualitative feedback was collected from early test users across an initial block of 8 complete sessions utilizing the integrated React feedback widget.

Quantitative Metrics:

- **Average Star Rating:** 4.1 out of 5.0.
- **Difficulty Appropriateness:** 6 out of 8 users (75%) found the difficulty aligned with expectations.
- **Question Relevance:** 7 out of 8 users (87.5%) found the generated questions highly relevant to the selected subtopics.

Qualitative Comments: Common feedback trends included requests for an extended timer allowance specifically for theory questions and praise regarding the technical depth of the DSA curriculum. Furthermore, users noted that the Guesstimation questions were significantly more engaging than standard textbook equivalents. Corroborating the findings in Section 7.2, one user explicitly noted that a minority of the MCQ options were "too obviously wrong," validating the previously identified 15% distractor discrepancy.

Chapter 8

Limitations and Future Work

8.1 Current Limitations

While Phase A of MockAI successfully establishes the core generation, evaluation, and tracking architecture, several technical and pedagogical limitations exist within the current implementation.

- **Subject-Level Skill Tracking:** The Exponential Moving Average (EMA) engine currently aggregates proficiency strictly at the macro subject level (e.g., DSA as a singular entity). The `subtopic_scores` table exists and is correctly structured within PostgreSQL, but the `end_session` endpoint does not yet write to it. Subtopic-level granularity (e.g., diagnosing that a user is strong in Graph Algorithms but weak in Dynamic Programming) would significantly enhance the precision of study recommendations.

- **MCQ Evaluation Bias:** The deterministic fuzzy string-matching algorithm efficiently handles common structural phrasing variations but fails when encountering semantically equivalent answers utilizing entirely different vocabulary. For example, a user submitting "Linear Search" when the exact expected string is "Sequential Search" is algorithmically penalized despite describing the identical concept. A semantic similarity approach leveraging vector embeddings is required to resolve this.

- **LLM Quota Constraints:** Both the Gemini and Groq APIs impose strict rate limits on their free tiers. A single power user can exhaust the Gemini daily generative quota within approximately 20 extensive sessions. For multi-tenant production deployment, paid API provisioning or integration with locally hosted, open-source models (via frameworks like Ollama) will be necessary.

- **Anti-Cheat Implementation:** The quiz interface currently displays an "Anti-Cheat" status indicator within the sidebar as a placeholder for planned features, but no active detection is implemented. Tab-switching detection can be introduced seamlessly via the browser's Page Visibility API, while advanced proctoring (gaze

tracking and facial recognition) will require the integration of external Computer Vision (CV) libraries.

- **Pagination Limits:** The interactive dashboard currently restricts rendering to the user's last 5 completed sessions to optimize initial load times. The comprehensive session log lacks dynamic pagination, which will cause rendering bloat for users accumulating extensive historical data.

8.2 Planned Phase B Features — Subject Activation

Activating the reserved Phase B curriculum subjects requires minimal architectural friction. It necessitates toggling the `comingSoon` boolean flag to `false` within the frontend `curriculum.ts` file and injecting the corresponding subject-specific context strings into the backend `question_generator` service. The database schema natively supports all Phase B domains within the `skill_vectors` table.
Planned curriculum expansions include:

- **Programming Languages:** Python (FastAPI, NumPy, Pandas, OOP) and Java (J2EE, Collections, JDBC).
- **Modern Web:** Web Technologies and Node.js.
- **Artificial Intelligence:** Core AI (Search algorithms, knowledge representation), Machine Learning (Regression, classification, clustering, evaluation metrics), and Deep Learning (CNNs, RNNs, Transformers).
- **Core Computing Systems:** Computer Organization and Architecture, Compiler Design, and Theory of Computation.

8.3 Machine Learning Prediction Layer

Once sufficient historical session data is accumulated (targeting a baseline of 100+ active users completing 20+ sessions each), a dedicated Machine Learning prediction layer will be integrated into the backend architecture.
Input feature vectors will ingest per-subject EMA scores, subtopic scores, session frequency metrics, the mathematical slope of the EMA improvement over time, and time-to-completion ratios per difficulty tier. The models will target several predictive outputs, including a normalized Interview Readiness Score (0–100%), dynamically recommended follow-up subjects, and projected performance on upcoming technical assessments. Candidate architectures include Linear Regression for normalized readiness scoring, Random Forest classifiers for dynamic subject recommendation, and Collaborative Filtering to map learning paths based on peer-user patterns.

8.4 Real-Time AI Panel Interview Simulation

The planned simulation module will orchestrate a multi-agent architectural flow featuring distinct AI personas. This panel will include a Technical Interviewer focused exclusively on algorithms, an HR Representative targeting behavioral competencies, a System Design Specialist for architectural scaling discussions, and a Domain Expert for niche depth.
The candidate will interface via text or voice. Each autonomous agent will independently maintain a conversational context window, track factual claims made by the candidate, dynamically generate follow-up inquiries based on preceding answers, and adapt the technical depth of the interview in real-time based on the candidate's response quality.

8.5 ClawBot Integration

ClawBot is envisioned as a central orchestration agent positioned between the multi-agent AI panel and the candidate's external development environment. It will be authorized to dynamically fetch source code from the user's authenticated GitHub repositories during an active simulation.
ClawBot will automatically summarize complex project documentation to inject contextual grounding into the panel agents' prompts, track the conversational state across agent hand-offs, generate structured post-interview analytical transcripts, and construct actionable task lists to address identified knowledge gaps. This integration shifts the simulation paradigm from generic technical questioning to hyper-contextualized interviews strictly grounded in the candidate's actual engineering experience.

8.6 Resume Parser Integration

A distinct Streamlit-based resume parsing engine was developed independently prior to the MockAI architecture. Phase B will natively integrate this Python module, enabling users to upload standard PDF resumes directly into the platform.
The backend will autonomously extract technical skills, professional experience, and project descriptions, mapping this data into the user profile. The generation engine will heavily utilize this extracted context to personalize the generated questions, forcing the candidate to defend the specific frameworks and architectures they have explicitly claimed on their resume rather than facing generic domain questions.

8.7 Deployment Plan

The platform architecture was specifically designed for frictionless transition to production cloud environments. The only codebase modification required for live deployment is updating the `DATABASE_URL` connection string within the backend environment variables; the underlying architecture remains identical between local Docker execution and production hosting.

Component	**Target Platform**	**Notes**
Next.js Frontend	Vercel	Provides zero-configuration deployment, Edge Network caching, and a generous free tier.
FastAPI Backend	Railway or Render	Seamless Dockerfile deployment pipeline with integrated continuous delivery.
PostgreSQL Database	Supabase	Fully managed, scalable PostgreSQL instance providing a free 500MB storage tier.
Redis Cache	Upstash	Serverless Redis architecture specifically optimized for lightweight connection pooling.
LLM APIs	Gemini and Groq	Will scale from free tiers to paid organizational plans synchronously as user concurrency grows.

Table 8.1: Targeted Deployment Infrastructure mapping platform components to managed cloud providers.

8.8 Modularity and Extensibility Analysis

Yes, MockAI is explicitly engineered to be modular at every architectural layer. The system design strictly adheres to the principle of separation of concerns, ensuring that future extensions require minimal code intrusion.

- **Curriculum Modularity:** The entire pedagogical curriculum is defined within a single static TypeScript file (`curriculum.ts`). Expanding the platform to include a new subject requires adding a single JSON-like object to the `subjects` array.

Activating a reserved Phase B domain merely requires toggling a `comingSoon: false` boolean flag. No database migrations or complex backend routing changes are required.

- **LLM Provider Modularity:** The core generation and evaluation services utilize an abstracted `try-except` fallback chain. Integrating a third LLM provider (e.g., Anthropic Claude) only requires the addition of one `except` block containing the new client call. The surrounding orchestration logic remains completely untouched.
- **Router Modularity:** FastAPI routers are registered independently in `main.py`. Introducing a new feature area (such as the planned AI Interview module) requires creating a standalone router file and appending a single `include_router` declaration.
- **Database Modularity:** The PostgreSQL schema is strictly version-controlled via Alembic. Existing tables are never modified destructively; schema evolution is exclusively additive. Reserved tables for Phase B (e.g., `cheat_events`, `subtopic_scores`) are already instantiated, allowing future services to begin writing to them immediately without schema friction.
- **Frontend Modularity:** The Next.js App Router enforces file-based routing. Deploying a new user interface page simply requires creating a new directory containing a `page.tsx` file, eliminating the need for centralized routing configuration files.

Architectural Modularity Stack

Figure 8.1 visualizes the isolated extensibility of the five primary architectural layers.

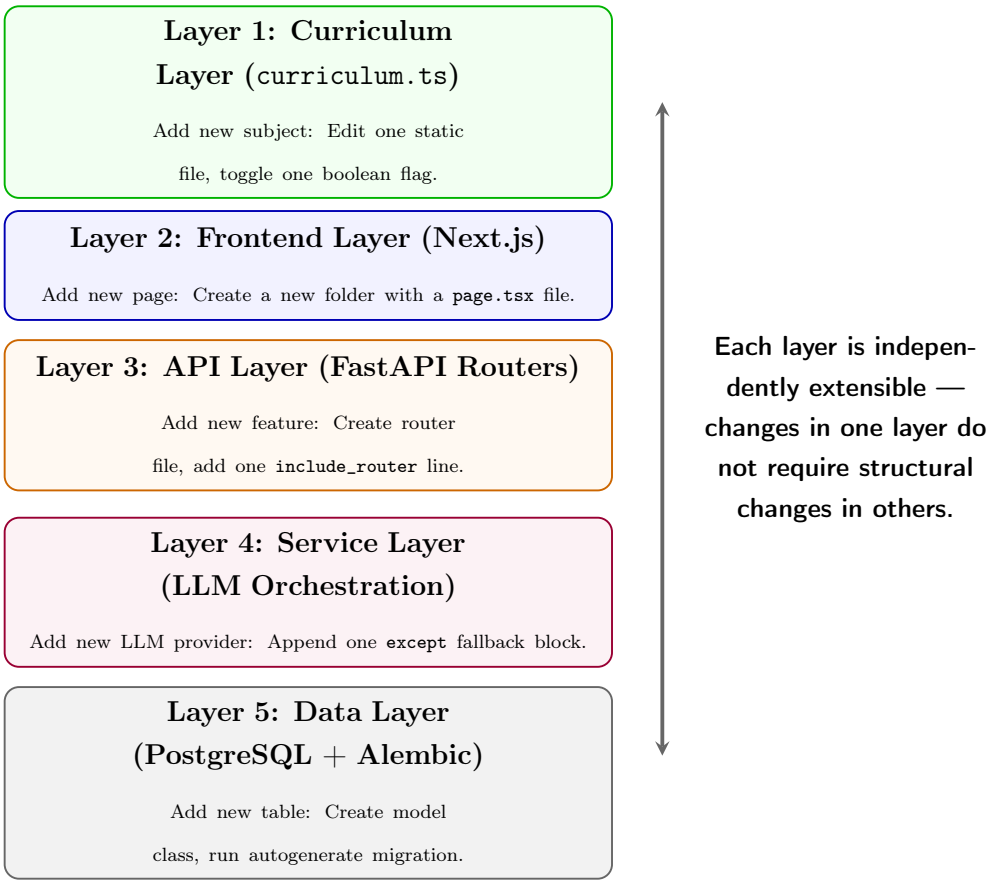

Figure 8.1: Modularity Layer Stack demonstrating the isolated extensibility of the MockAI platform.

8.9 Production Deployment Strategy

Moving the MockAI platform from a localized Docker development environment to a live, scalable production architecture requires exactly three environment variable updates and zero core codebase modifications.

1. `DATABASE_URL`: The local connection string is replaced with a managed Supabase connection URI (e.g., `postgresql+asyncpg://...supabase.co:5432/postgres`). Supabase provides managed PostgreSQL, automatic backups, and connection pooling.

2. `NEXT_PUBLIC_API_URL`: The frontend's hardcoded `localhost` targeting is replaced with the live backend URL (e.g., `https://mockai-backend.railway.app`). This is injected securely at build time by Vercel.

3. `JWT_SECRET_KEY`: The localized development key is permanently discarded and replaced with a cryptographically secure, 128-character production key to prevent token forgery.

Cloud Infrastructure Mapping

Figure 8.2 maps the platform components to their respective managed cloud providers.

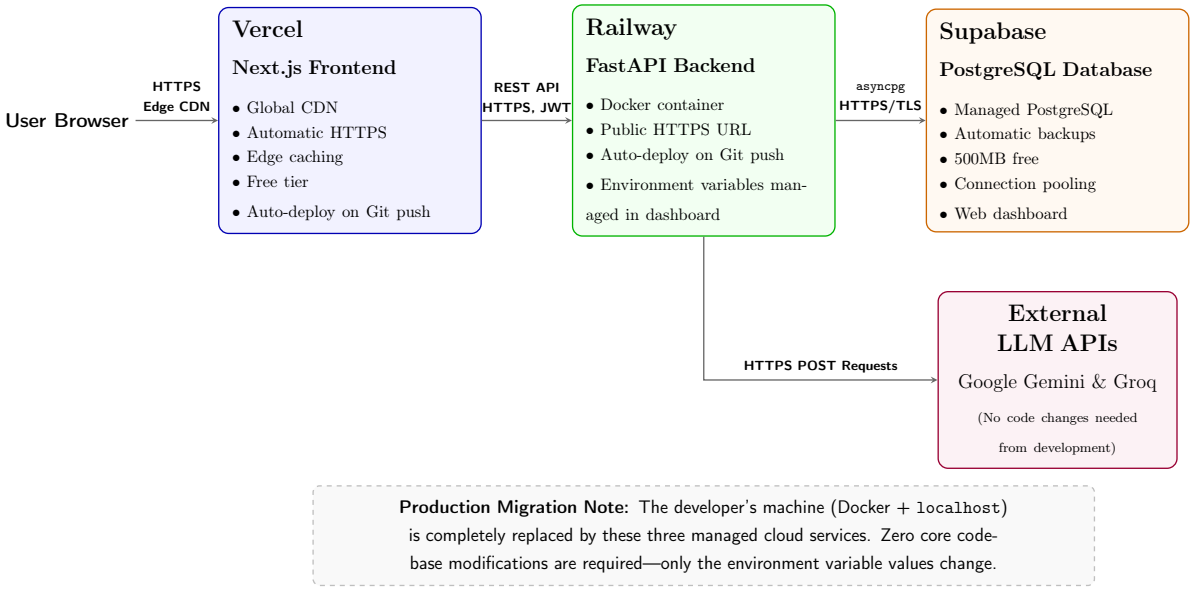

Figure 8.2: Production Deployment Architecture Diagram mapping the transition from localized development to managed cloud infrastructure.

8.10 Database Maintenance and Data Retention

Schema Version Control

Every database schema evolution within MockAI is rigidly managed via an Alembic migration file. These files are committed to the Git repository alongside the application code, ensuring the database architecture is version-controlled exactly like standard logic. Rolling back a catastrophic database change is handled via a simple `alembic downgrade -1` command.

The migration files are stored in the `alembic/versions` directory. Each generated file contains a unique cryptographic revision ID, a description, and two critical functions: `upgrade()` (which applies the schema change) and `downgrade()` (which reverts it).

Data Retention and Backups

User session data is currently stored permanently; there is no automated deletion protocol active in Phase A. In Phase B, a tiered 20-day retention policy will be implemented, allowing users to export historical session data (via PDF/CSV) before cloud archival. For data security, Supabase natively provides automatic daily snapshots with 7-day retention on its free tier. For localized instances, a standard `pg_dump` cron job guarantees data security.

Scaling Considerations

PostgreSQL handles millions of relational rows efficiently provided proper indexing is applied. As concurrent usage scales, the `sessions` table will require an index on `user_id` to optimize dashboard queries, while the `answers` table will benefit from a composite index on `(session_id, question_id)` to accelerate the end-session evaluation loop. These indexes will be applied via additive Alembic migrations when initial query performance degrades.

8.11 Security Architecture

Password and Token Security

Passwords are mathematically hashed utilizing the `bcrypt` algorithm (configured with a work factor of 12) prior to any database write operation. `bcrypt` is intentionally designed as a slow hash function, making brute-force dictionary attacks computationally unfeasible. The plaintext password is discarded from memory immediately following the hash computation.
Authentication is managed via JSON Web Tokens (JWT). The tokens are cryptographically signed but not encrypted. While the payload (containing the user UUID and expiry timestamp) is Base64 readable, it cannot be forged or altered without the 128-character server-side secret key. Sensitive data (e.g., password hashes, personal emails) is strictly excluded from the token payload.

API and Key Security

With the exception of the `/register` and `/login` endpoints, all platform routing requires a valid Bearer JWT. The `get_current_user` dependency globally enforces this constraint; any unauthorized request is rejected with a `401` status before the handler logic executes. Furthermore, external LLM API keys are stored exclusively in environment variables (`.env`), which are strictly ignored by version control.

Security Layer Visualization

Figure 8.3 illustrates the concentric defense-in-depth architecture protecting the system's core data.

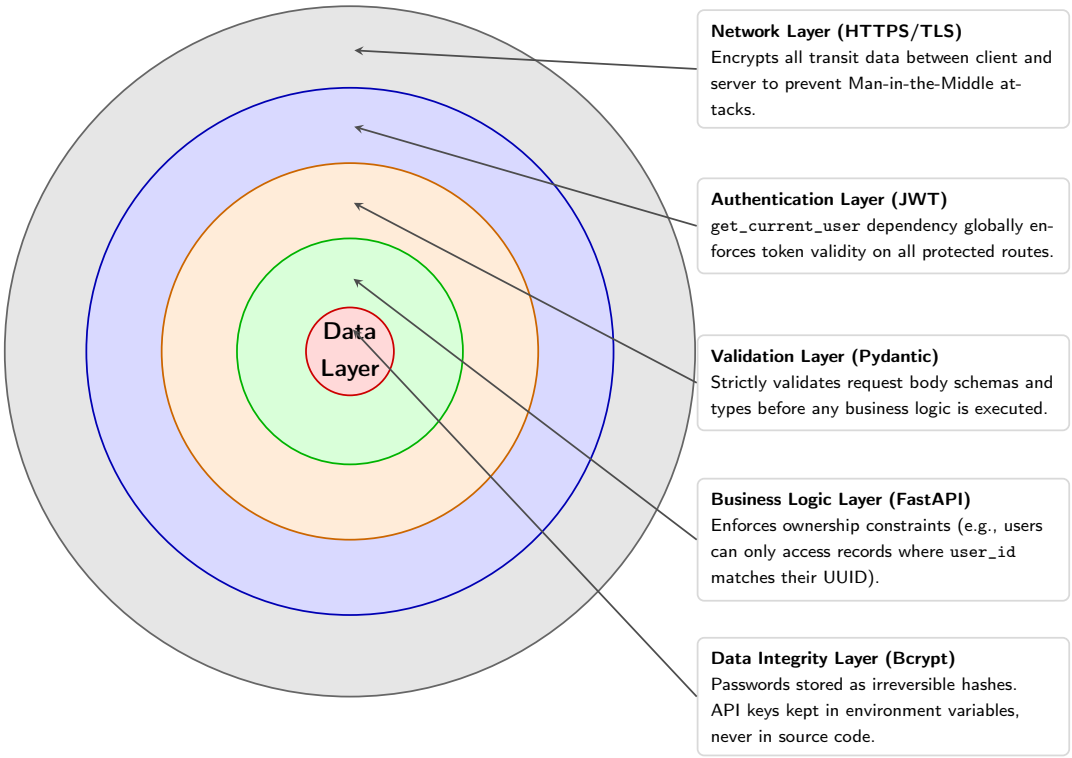

Figure 8.3: Security Architecture Layered Defense, illustrating the "Onion" model where requests must penetrate multiple validation and authentication barriers.

Chapter 9

Conclusion

This paper has presented **MockAI**, an adaptive AI-powered interview preparation platform designed to bridge the gap between academic learning and professional placement. The research identified and addressed five critical deficiencies in existing interview preparation solutions: static content, a lack of conceptual feedback, the absence of longitudinal skill tracking, a lack of system adaptivity, and a general curriculum disconnect.

9.1 Technical Contributions

The development of MockAI makes four primary technical contributions to the field of AI-assisted technical education:

1. **Dynamic Generation Pipeline:** By utilizing Google Gemini with a secondary Groq fallback, the system produces novel, subtopic-specific, and difficulty-calibrated questions across a comprehensive curriculum of 27 subjects. This approach effectively eliminates the "static question bank" problem by providing unique content for every session.

2. **Contextual AI Evaluation Engine:** The platform moves beyond binary MCQ scoring to provide deep-learning-driven explanatory feedback. It identifies specific strengths and weaknesses for both multiple-choice and open-ended theoretical questions, providing students with the "why" behind their performance.

3. **EMA-Based Skill Vector Engine:** The implementation of an *Exponential Moving Average* (EMA) skill vector engine provides a longitudinal, recency-weighted model of student proficiency. This creates a meaningful quantitative measure of interview readiness that evolves dynamically as the student interacts with the system.

4. **Modular Phase-Based Architecture:** The system employs a modular architecture that supports progressive feature activation. The roadmap for Phase B and C subjects can be unlocked via simple configuration changes, requiring no database migrations or structural overhauls to the core engine.

9.2 Final Reflections

The system demonstrates that Large Language Models (LLMs), when governed by precise domain context, difficulty specifications, subtopic constraints, and structured output requirements, can serve as highly effective engines for technical evaluation. The combination of dynamic content generation, contextual feedback, and longitudinal tracking represents a meaningful advance over current state-of-the-art interview preparation platforms.

9.3 The Path Ahead

MockAI is positioned to grow into a comprehensive career preparation ecosystem. The planned additions of real-time multi-agent AI interview simulations, *ClawBot* project intelligence integration, ML-based readiness prediction, and an integrated anti-cheat system will progressively close the gap between simulated preparation and actual interview conditions. For students at the University Institute of Engineering and Technology (UIET) and beyond, MockAI offers the most realistic, personalized, and data-driven preparation experience currently available.